

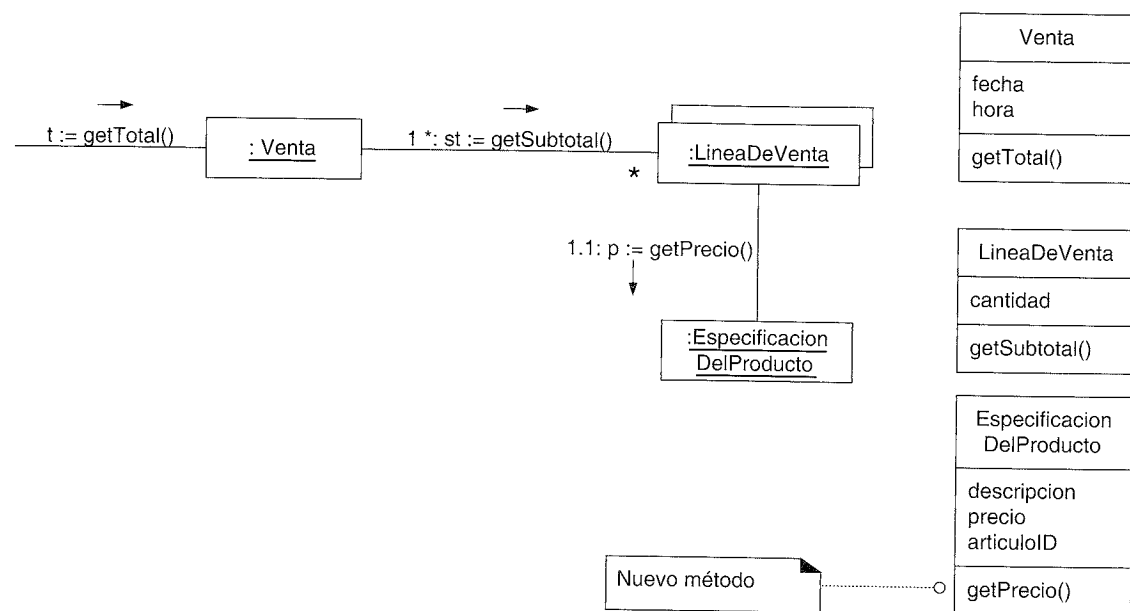


Figura 16.6. Cálculo del total de una Venta.

que la información se encuentre esparcida por objetos diferentes, necesitarán interactuar mediante el paso de mensajes para compartir el trabajo.

Por lo general, el Experto nos conduce a diseños donde los objetos del software realizan aquellas operaciones que normalmente se hacen a los objetos inanimados del mundo real que representan; Peter Coad denomina a esto la estrategia “Hacerlo yo mismo” [Coad95]. Por ejemplo, en el mundo real, sin el uso de ayudas mecánicas, una venta no te dice su total; es un objeto inanimado. Alguien calcula el total de la venta. Pero en el campo del software orientado a objetos, todos los objetos software están “vivos” o “animados”, y pueden tener responsabilidades y hacer cosas. Fundamentalmente, hacen cosas relacionadas con la información que conocen. Yo lo denomino el principio “de animación” en el diseño de objetos; es como estar en unos dibujos animados donde todo está vivo.

El patrón Experto en Información —como muchas cosas en la tecnología de objetos— tiene una analogía en el mundo real. Normalmente, otorgamos responsabilidades a los individuos con la información necesaria para llevar a cabo una tarea. Por ejemplo, en un negocio, ¿quién debería ser el responsable de la creación de una declaración de ganancias y pérdidas? La persona que tiene acceso a toda la información necesaria para crearla —quizás el director financiero—. Y, del mismo modo en que los objetos colaboran porque la información se encuentra dispersa, así pasa con las personas. El director financiero de la compañía podría solicitar a los contables que generen informes sobre los créditos y débitos.

En algunas ocasiones la solución que sugiere el Experto no es deseable, normalmente debido a problemas de acoplamiento y cohesión (estos principios se discutirán más tarde en este capítulo).

Por ejemplo, ¿quién debería ser el responsable de almacenar una *Venta* en una base de datos? Ciertamente, mucha de la información que se tiene que almacenar se encuen-

tra en el objeto *Venta* y, por tanto, siguiendo el patrón Experto se podría argumentar la inclusión de la responsabilidad en la clase *Venta*. La extensión lógica de esta decisión es que cada clase contiene sus propios servicios para almacenarse ella misma en una base de datos. Pero esto nos lleva a problemas de cohesión, acoplamiento y duplicación. Por ejemplo, la clase *Venta* debe ahora contener la lógica relacionada con la gestión de la base de datos, como la relacionada con SQL y JDBC (Conexión de Base de Datos de Java, *Java Database Connectivity*). La clase ya no está centrada únicamente en la lógica de la aplicación de “ser una venta” simplemente; ahora tiene otro tipo de responsabilidades, lo que disminuye su cohesión. La clase debe acoplarse a los servicios técnicos de base de datos de otro subsistema, como los servicios JDBC, en lugar de acoplarse únicamente con otros objetos en la capa del dominio de los objetos del software, lo que eleva su acoplamiento. Y es probable que se dupliquen lógicas de bases de datos similares en muchas clases persistentes.

Todos estos problemas indican la violación de un principio arquitectural básico: diseñe separando los principales aspectos del sistema. Mantenga la lógica de la aplicación en un sitio (como en los objetos software del dominio), mantenga la lógica de la base de datos en otro sitio (como en un subsistema de servicios de persistencia separado), y así sucesivamente, en lugar de entremezclar aspectos del sistema diferentes en el mismo componente⁴.

Separando los aspectos importantes se mejora el acoplamiento y la cohesión del diseño. Por tanto, aunque siguiendo el Experto se podría justificar la asignación de la responsabilidad para el servicio de la base de datos a la clase *Venta*, por otros motivos (normalmente cohesión y acoplamiento), resulta un diseño pobre.

Beneficios

- Se mantiene el encapsulamiento de la información, puesto que los objetos utilizan su propia información para llevar a cabo las tareas. Normalmente, esto conlleva un bajo acoplamiento, lo que da lugar a sistemas más robustos y más fáciles de mantener. (Bajo Acoplamiento también es un patrón GRASP que se estudiará en una de las secciones siguientes.)
- Se distribuye el comportamiento entre las clases que contienen la información requerida, por tanto, se estimula las definiciones de clases más cohesivas y “ligeras” que son más fáciles de entender y mantener. Se soporta normalmente una alta cohesión (otro patrón que se estudiará más tarde).
- Bajo Acoplamiento.
- Alta Cohesión.

Patrones o Principios Relacionados

También conocido como; Parecido a

“Colocar las responsabilidades con los datos”, “Eso que conoces, hazlo”, “Hacerlo yo mismo”, “Colocar los Servicios con los Atributos con los que trabaja”.

16.7. Creador

Solución

Asignar a la clase B la responsabilidad de crear una instancia de clase A si se cumple uno o más de los casos siguientes:

⁴ En el Capítulo 32 se presentará una discusión sobre la separación de intereses.

- B *agrega* objetos de A.
- B *contiene* objetos de A.
- B *registra* instancias de objetos de A.
- B *utiliza más estrechamente* objetos de A.
- B *tiene los datos de inicialización* que se pasarán a un objeto de A cuando sea creado (por tanto, B es un Experto con respecto a la creación de A).

B es un *creador* de los objetos A.

Si se puede aplicar más de una opción, inclínese por una clase B que *agregue* o *contenga* la clase A.

Problema

¿Quién debería ser el responsable de la creación de una nueva instancia de alguna clase?

La creación de instancias es una de las actividades más comunes en un sistema orientado a objetos. En consecuencia, es útil contar con un principio general para la asignación de las responsabilidades de creación. Si se asignan bien, el diseño puede soportar un bajo acoplamiento, mayor claridad, encapsulación y reutilización.

Ejemplo

En la aplicación del PDV, ¿quién debería ser el responsable de la creación de una instancia de *LineaDeVenta*? Según el patrón Creador, deberíamos buscar las clases que agregan, contienen, etcétera, instancias de *LineaDeVenta*. Considere el modelo del dominio parcial de la Figura 16.7.

Puesto que una *Venta* contiene (de hecho, agrega) muchos objetos de *LineaDeVenta*, el patrón Creador sugiere que *Venta* es un buen candidato para tener la responsabilidad de la creación de las instancias de *LineaDeVenta*.

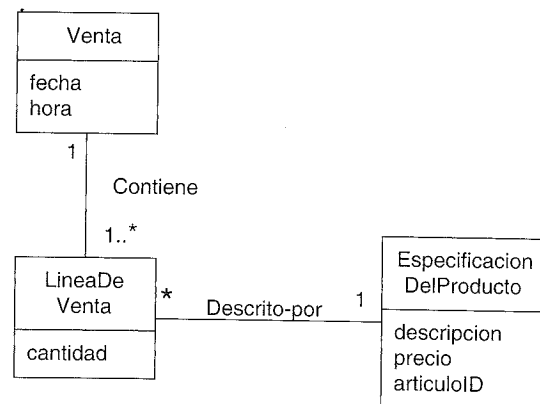


Figura 16.7. Modelo del Dominio Parcial.

Esto nos lleva al diseño de las interacciones entre los objetos que se muestran en la Figura 16.8.

Esta asignación de responsabilidades requiere que se defina en la clase *Venta* un método *crearLineaDeVenta*.

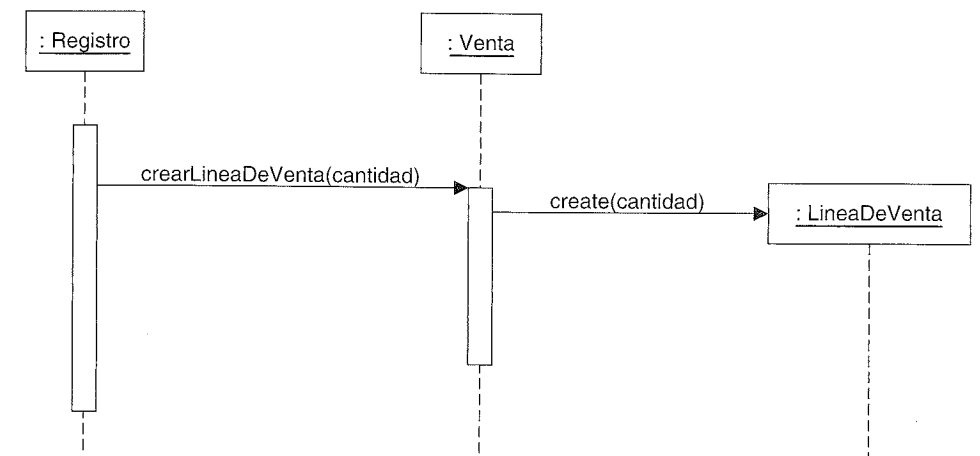


Figura 16.8. Creación de una LineaDeVenta.

Una vez más, el contexto en el que se consideraron y se optaron por estas responsabilidades fue durante la elaboración de un diagrama de interacción. La sección de los métodos de un diagrama de clases puede mostrar los resultados de la asignación de responsabilidades, materializadas concretamente como métodos.

Discusión

El patrón Creador guía la asignación de responsabilidades relacionadas con la creación de objetos, una tarea muy común. La intención básica del patrón Creador es encontrar un creador que necesite conectarse al objeto creado en alguna situación. Elijiéndolo como el creador se favorece el bajo acoplamiento.

El Agregado *agrega* Partes, el Contenedor *contiene* Contenido, y el Registro *registra* Registros, son todas ellas relaciones comunes entre las clases en un diagrama de clases. El Creador sugiere que la clase contenedor o registro es una buena candidata para asignarle la responsabilidad de crear lo que contiene o registra. Por su puesto, esto es sólo una guía.

Nótese que se ha utilizado el concepto de **agregación** al considerar el patrón Creador. La agregación se examinará en el Capítulo 27; una definición breve es que una agregación involucra cosas que se encuentran en una relación Todo-Parte o Ensamblaje-Parte, como un Cuerpo agrega Pierna o un Párrafo agrega Frase.

Algunas veces se encuentra un creador buscando las clases que tienen los datos de inicialización que se pasarán durante la creación. Se trata, en realidad, de un ejemplo del patrón Experto. Los datos de inicialización se pasan durante la creación por medio de algún tipo de método de inicialización, como un constructor Java que tiene parámetros. Por ejemplo, asuma que una instancia de *Pago* necesita inicializarse, cuando se crea, con el total de la *Venta*. Puesto que la *Venta* conoce el total, la *Venta* es un creador candidato del *Pago*.

Contraindicaciones

A menudo, la creación requiere una complejidad significativa, como utilizar instancias recicladas por motivos de rendimiento, crear condicionalmente una instancia a partir de una familia de clases similares basado en el valor de alguna propiedad externa, etcétera. En estos casos, es aconsejable delegar la creación a una clase auxiliar denominada *Factoría* [GHJV95] en lugar de utilizar la clase que sugiere el *Creador*. Las factorías se estudiarán en el Capítulo 23.

Beneficios	• Se soporta bajo acoplamiento (descrito a continuación), lo que implica menos dependencias de mantenimiento y mayores oportunidades para reutilizar. Probablemente no se incrementa el acoplamiento porque la clase <i>creada</i> es presumible que ya sea visible a la clase <i>creadora</i>, debido a las asociaciones existentes que motivaron su elección como creador.
Patrones o Principios Relacionados	• Bajo Acoplamiento. • Factoría. • Todo-Parte [BMRSS96] describe un patrón para definir objetos agregados que favorece la encapsulación de componentes.

16.8. Bajo Acoplamiento

Solución	Asignar una responsabilidad de manera que el acoplamiento permanezca bajo.
Problema	¿Cómo soportar bajas dependencias, bajo impacto del cambio e incremento de la reutilización?

El **acoplamiento** es una medida de la fuerza con que un elemento está conectado a, tiene conocimiento de, confía en, otros elementos. Un elemento con bajo (o débil) acoplamiento no depende de demasiados otros elementos; “demasiados” depende del contexto, pero se estudiará. Estos elementos pueden ser clases, subsistemas, sistemas, etcétera.

Una clase con alto (o fuerte) acoplamiento confía en muchas otras clases. Tales clases podrían no ser deseables; algunas adolecen de los siguientes problemas:

- Los cambios en las clases relacionadas fuerzan cambios locales.
- Son difíciles de entender de manera aislada.
- Son difíciles de reutilizar puesto que su uso requiere la presencia adicional de las clases de las que depende.

Ejemplo	Considere el siguiente diagrama de clases parcial del caso de estudio NuevaEra:
---------	---



Asuma que tenemos la necesidad de crear una instancia de *Pago* y asociarla con la *Venta*. ¿Qué clase debería ser la responsable de esto? Puesto que un *Registro* “registra” un *Pago* en el dominio del mundo real, el patrón Creador sugiere el *Registro* como candidata para la creación del *Pago*. La instancia de *Registro* podría enviar entonces el mensaje *añadirPago* a la *Venta*, pasando el nuevo *Pago* como parámetro. La Figura 16.9 muestra un posible diagrama de interacción que refleja esto.

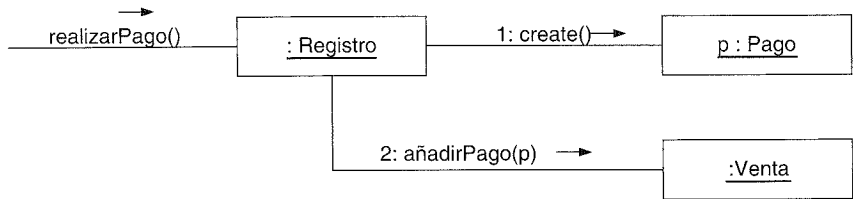


Figura 16.9. El Registro crea un Pago.

Esta asignación de responsabilidades acopla la clase *Registro* con el conocimiento de la clase *Pago*.

Notación UML: Nótese que la instancia de *Pago* se nombra explícitamente con una *p* de manera que se puede referenciar como parámetro en el mensaje 2.

La Figura 16.10 muestra una solución alternativa a crear el *Pago* y asociarlo con la *Venta*.

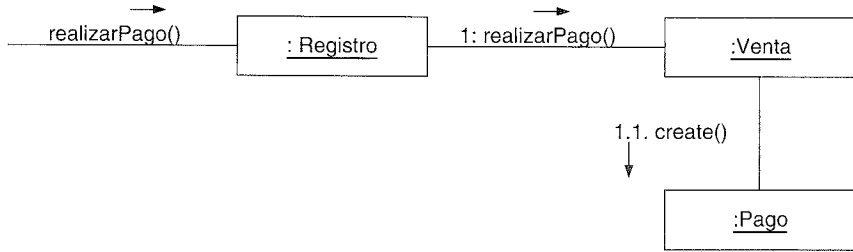


Figura 16.10. La Venta crea el Pago.

¿Qué diseño, basado en la asignación de responsabilidades, soporta Bajo Acoplamiento? En ambos casos asumiremos que la *Venta* debe finalmente acoplarse con el conocimiento del *Pago*. El Diseño 1, en el que el *Registro* crea el *Pago*, añade acoplamiento entre el *Registro* y el *Pago*, mientras que el Diseño 2, en el que la *Venta* lleva a cabo la creación del *Pago*, no incrementa el acoplamiento. Desde el punto de vista puramente del acoplamiento, es preferible el Diseño 2 porque mantiene el acoplamiento global más bajo. Éste es un ejemplo en el que dos patrones —Bajo Acoplamiento y Creador— podrían sugerir soluciones diferentes.

En la práctica, el nivel de acoplamiento no se puede considerar de manera aislada a otros principios como el Experto o Alta Cohesión. Sin embargo, es un factor a tener en cuenta para mejorar un diseño.

Discusión

El patrón de Bajo Acoplamiento es un principio a tener en mente en todas las decisiones de diseño; es un objetivo subyacente a tener en cuenta continuamente. Es un **principio evaluativo** que aplica un diseñador mientras evalúa todas las decisiones de diseño.

En los lenguajes orientados a objetos como C++, Java y C#, algunas de las formas comunes de acoplamiento entre el *TipoX* y el *TipoY* son:

- El *TipoX* tiene un atributo (miembro de datos, o variable de instancia) que hace referencia a una instancia de *TipoY*, o al propio *TipoY*.
- Un objeto de *TipoX* invoca los servicios de un objeto de *TipoY*.
- El *TipoX* tiene un método que referencia a una instancia de *TipoY*, o al propio *TipoY*, de algún modo. Esto, generalmente, comprende un parámetro o variable local de *TipoY*, o que el objeto de retorno de un mensaje sea una instancia de *TipoY*.
- El *TipoX* es una subclase, directa o indirecta, del *TipoY*.
- El *TipoY* es una interfaz y el *TipoX* implementa esa interfaz.

El patrón Bajo Acoplamiento impulsa la asignación de responsabilidades de manera que su localización no incremente el acoplamiento hasta un nivel que nos lleve a los resultados negativos que puede producir un acoplamiento alto.

El Bajo Acoplamiento soporta el diseño de clases que son más independientes, lo que reduce el impacto del cambio. No se puede considerar de manera aislada a otros patrones como el Experto o el de Alta Cohesión, sino que necesita incluirse como uno de los diferentes principios de diseño que influyen en una elección al asignar una responsabilidad.

Una subclase está fuertemente acoplada a su superclase. Se debe estudiar cuidadosamente la decisión de derivar a partir de una superclase debido a esta forma fuerte de acoplamiento. Por ejemplo, suponga que necesitamos almacenar los objetos de manera persistente en una base de datos relacional u objetual. En este caso, un diseño relativamente común consiste en crear una superclase abstracta denominada *ObjetoPersistente* a partir de la cual derivan otras clases. El inconveniente de esta creación de subclases es que acopla altamente los objetos del dominio a un servicio del nivel tecnológico particular y mezcla diferentes aspectos de la arquitectura, mientras que la ventaja es que se hereda de manera automática el comportamiento persistente.

No existe una medida absoluta de cuando el acoplamiento es demasiado alto. Lo que es importante es que un desarrollador pueda medir el grado de acoplamiento actual, y evaluar si aumentarlo le causará problemas. En general, las clases que son inherentemente muy genéricas por naturaleza, y con una probabilidad de reutilización alta, debería tener un acoplamiento especialmente bajo.

El caso extremo de Bajo Acoplamiento es cuando no existe acoplamiento entre clases. Esto no es deseable porque una metáfora central de la tecnología de objetos es un sistema de objetos conectados que se comunican mediante el paso de mensajes. Si el Bajo Acoplamiento se lleva al extremo, producirá un diseño pobre porque dará lugar a unos pocos objetos inconexos, saturados, y con actividad compleja que hacen todo el trabajo, con muchos objetos muy pasivos, sin acoplamiento que actúan como simples repositorios de datos. Es normal y necesario un grado moderado de acoplamiento entre las clases para crear un sistema orientado a objetos en el que las tareas se llevan a cabo mediante la colaboración de los objetos conectados.

No suele ser un problema el acoplamiento alto entre objetos estables y elementos generalizados. Por ejemplo, una aplicación Java J2EE puede acoplarse con seguridad con las librerías de Java (*java.util*, etc.), porque son estables y extendidas.

Escoja sus batallas

El alto acoplamiento en sí no es un problema; es el alto acoplamiento entre elementos que no son estables en alguna dimensión, como su interfaz, implementación o su mera presencia.

Éste es un punto importante: como diseñadores, podemos añadir flexibilidad, encapsular detalles e implementaciones, y, en general, diseñar para disminuir el acoplamiento en muchas áreas del sistema. Pero, si nos esforzamos en “futuras necesidades” o en disminuir el acoplamiento en algunos puntos donde de hecho no hay motivos realistas, el tiempo no se está empleando de manera adecuada.

Contraindicaciones

Los diseñadores tienen que escoger sus batallas para disminuir el acoplamiento y encapsular información. Centrándose en los puntos en los que sea realista pensar en una inestabilidad o evolución alta. Por ejemplo, en el proyecto NuevaEra, se sabe que se necesitan conectar al sistema diferentes sistemas calculadores de impuestos de terceras partes (con interfaces únicas). Por tanto, es práctico diseñar para disminuir el acoplamiento en este punto de variación.

Beneficios

- No afectan los cambios en otros componentes.
- Fácil de entender de manera aislada.
- Conveniente para reutilizar.

Antecedentes

El acoplamiento y la cohesión (descrita a continuación) son principios realmente fundamentales en el diseño y todos los desarrolladores de software deberían apreciarlos y aplicarlos por sí mismos. Larry Constantine, también un creador del diseño estructurado en los años setenta y que actualmente propugna una mayor atención a la ingeniería de usabilidad [CL99], fue el principal responsable en los sesenta de la identificación y comunicación del acoplamiento y la cohesión como principios básicos [Constantine68, CMS74].

Patrones Relacionados

- Variaciones Protegidas.

16.9. Alta Cohesión

Solución

Asignar una responsabilidad de manera que la cohesión permanezca alta.

Problema

¿Cómo mantener la complejidad manejable?

En cuanto al diseño de objetos, la **cohesión** (o de manera más específica, la cohesión funcional) es una medida de la fuerza con la que se relacionan y del grado de focalización de las responsabilidades de un elemento. Un elemento con responsabilidades altamente relacionadas, y que no hace una gran cantidad de trabajo, tiene alta cohesión. Estos elementos pueden ser clases, subsistemas, etcétera.

Una clase con baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo. Tales clases no son convenientes; adolecen de los siguientes problemas:

- Difíciles de entender.
- Difíciles de reutilizar.
- Difíciles de mantener.
- Delicadas, constantemente afectadas por los cambios.

A menudo, las clases con baja cohesión representan un “grano grande” de abstracción, o se les han asignado responsabilidades que deberían haberse delegado en otros objetos.

Podemos analizar para la Alta Cohesión el mismo ejemplo que se utilizó para el patrón de Bajo Acoplamiento.

Ejemplo

Asuma que necesitamos crear una instancia de *Pago* (en efectivo) y asociarla con la *Venta*. ¿Qué clase debería ser responsable de esto? Puesto que el *Registro* registra un *Pago* en el dominio del mundo real, el patrón *Creador* sugiere el *Registro* como candidato para la creación del *Pago*. La instancia de *Registro* podría entonces enviar un

mensaje *añadirPago* a la *Venta*, pasando como parámetro el nuevo *Pago*, como se muestra en la Figura 16.11.

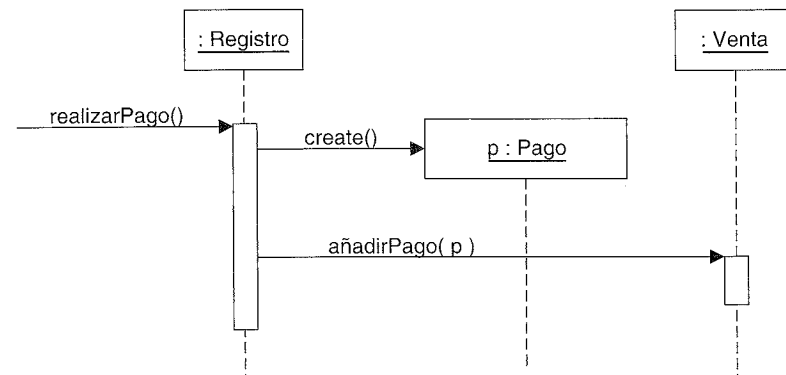


Figura 16.11. El Registro crea el Pago.

Esta asignación de responsabilidades sitúa la responsabilidad de realizar un pago en el *Registro*. El *Registro* toma parte en la responsabilidad de llevar a cabo la operación del sistema *realizarPago*.

En este ejemplo aislado, esto es aceptable; pero si continuamos haciendo responsable a la clase *Registro* de realizar parte o la mayoría del trabajo relacionado con más y más operaciones del sistema, se irá sobrecargando incrementalmente con tareas y llegará a perder la cohesión.

Imagine que hubiera cincuenta operaciones del sistema, todas recibidas por *Registro*. Si hace el trabajo relacionado con cada una, se convertirá en un objeto "saturado" y sin cohesión. El hecho no es que esta simple tarea de creación de un *Pago* en sí misma haga que el *Registro* no sea cohesivo, sino que como parte de un cuadro mayor de asignación de responsabilidades global, podría sugerir una tendencia a una baja cohesión.

Y más importante en cuanto a las habilidades de desarrollo como diseñadores de objetos, independientemente de la elección de diseño final, lo importante es que por lo menos un desarrollador sepa tener en cuenta el impacto de la cohesión.

En cambio, como se muestra en la Figura 16.12, el segundo diseño delega la responsabilidad de creación del pago a la *Venta*, que favorece una cohesión más alta en el *Registro*.

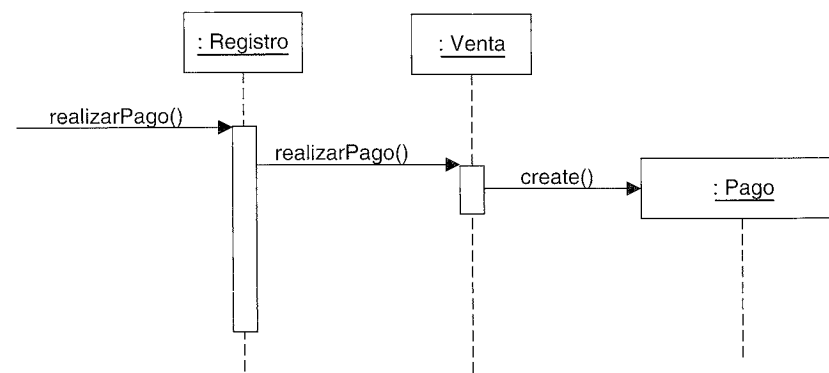


Figura 16.12. La Venta crea el Pago.

Es deseable el segundo diseño puesto que soporta tanto alta cohesión como bajo acoplamiento.

En la práctica, el nivel de cohesión no se puede considerar de manera aislada a otras responsabilidades y otros principios como los patrones Experto y Bajo Acoplamiento.

Discusión

Igual que el Bajo Acoplamiento, el patrón de Alta Cohesión es un principio a tener en mente durante todas las decisiones de diseño; es un objetivo subyacente a tener en cuenta continuamente. Es un principio evaluativo que aplica un diseñador mientras evalúa todas las decisiones de diseño.

Grady Booch establece que existe alta cohesión funcional cuando los elementos de un componente (como una clase) "trabajan todos juntos para proporcionar algún comportamiento bien delimitado" [Booch94].

A continuación presentamos algunos escenarios que ilustran diferentes grados de cohesión funcional:

1. *Muy baja cohesión.* Una única clase es responsable de muchas cosas en áreas funcionales muy diferentes.
 - Asuma que existe una clase denominada *Interfaz-BDR-RPC* que es completamente responsable de la interacción con bases de datos relacionales y la gestión de las llamadas remotas a procedimientos (*Remote Procedure Calls*). Éstas son dos áreas funcionales muy diferentes, y cada una requiere mucho código de soporte. La responsabilidad debería dividirse en una familia de clases relacionadas con el acceso BDR y una familia relacionada con el soporte de RPC.
2. *Baja cohesión.* Una única clase tiene la responsabilidad de una tarea compleja en un área funcional.
 - Asuma que existe una clase denominada *InterfazBDR* que es completamente responsable de interactuar con las bases de datos relacionales. Los métodos de la clase están todos relacionados, pero hay muchos, y una gran cantidad de código de soporte; podría haber cientos o miles de métodos. La clase debería dividirse en una familia de clases ligeras que comparten el trabajo de proporcionar el acceso BDR.
3. *Alta cohesión.* Una clase tiene una responsabilidad moderada en un área funcional y colabora con otras clases para llevar a cabo las tareas.
 - Asuma que existe una clase denominada *InterfazBDR* que es sólo parcialmente responsable de interactuar con las bases de datos relacionales. Interactúa con una docena de otras clases relacionadas con el acceso BDR para recuperar y almacenar objetos.
4. *Moderada cohesión.* Una clase tiene responsabilidades ligeras y únicas en unas pocas áreas diferentes que están lógicamente relacionadas con el concepto de la clase, pero no entre ellas.
 - Asuma que existe una clase denominada *Compañía* que es completamente responsable de (a) conocer a sus empleados y (b) conocer su información fi-

nanciera. Estas dos áreas no están fuertemente relacionadas entre ellas; aunque ambas están lógicamente relacionadas con el concepto de compañía. Además, el número total de métodos públicos es pequeño, al igual que la cantidad de código de soporte.

Como regla empírica, una clase con alta cohesión tiene un número relativamente pequeño de métodos, con funcionalidad altamente relacionada, y no realiza mucho trabajo. Colabora con otros objetos para compartir el esfuerzo si la tarea es extensa.

Una clase con alta cohesión es ventajosa porque es relativamente fácil de mantener, entender y reutilizar. El alto grado de funcionalidad relacionada, combinada con un número pequeño de operaciones, también simplifica el mantenimiento y las mejoras. El grano fino de la funcionalidad altamente relacionada también aumenta el potencial de reutilización.

El patrón de Alta Cohesión —como muchas cosas en la tecnología de objetos— tiene una analogía en el mundo real. Una observación típica es que si una persona tiene demasiadas responsabilidades no relacionadas —especialmente unas que deberían delegarse de manera adecuada en otras— entonces la persona no es efectiva. Se observa en algunos directores que no han aprendido a delegar. Estas personas padecen baja cohesión; están preparados para llegar a ser personas “despegadas”.

Otro principio clásico: diseño modular

El acoplamiento y la cohesión son viejos principios del diseño de software; diseñar con objetos no implica que se ignoren los fundamentos bien establecidos. Otro de estos principios —que está fuertemente relacionado con el acoplamiento y la cohesión— es promover el **diseño modular**. Citando textualmente:

La modularidad es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos cohesivos y débilmente acoplados [Booch94].

Fomentamos el diseño modular creando métodos y clases con alta cohesión. Al nivel básico de objetos, la modularidad se alcanza diseñando cada método con un único y claro objetivo, y agrupando un conjunto de aspectos relacionados en una clase.

Cohesión y acoplamiento; el yin y el yang

Una mala cohesión causa, normalmente, un mal acoplamiento, y viceversa. Llamo a la cohesión y el acoplamiento el *yin y el yang de la ingeniería del software* debido a que influye el uno en el otro. Por ejemplo, considere una clase que implementa un elemento gráfico GUI que representa y pinta dicho elemento gráfico, almacena datos en una base de datos, e invoca el servicio de objetos remotos. No sólo es profundamente no cohesiva, sino que está acoplada a muchos (y con nada en común) elementos.

Existen pocos casos en los que esté justificada la aceptación de baja cohesión.

Un caso es la agrupación de responsabilidades o código en una clase o componente para simplificar el mantenimiento por una persona —aunque queda advertido de que tal agrupación podría también empeorar el mantenimiento—. Pero por ejemplo, suponga

que una aplicación contiene sentencias SQL embebidas que siguiendo otros buenos principios de diseño deberían distribuirse por diez clases, tales como diez clases encargadas de la “conversión objeto-tupla de base de datos”. Ahora bien, es normal que sólo uno o dos expertos en SQL conozcan la mejor manera para definir y mantener este SQL, incluso si hay docenas de programadores orientados a objetos (OO) en el proyecto; pocos programadores OO podrían dominar SQL. Suponga que incluso el experto en SQL no se encuentra cómodo como programador OO. El arquitecto de software podría decidir agrupar todas las sentencias SQL en una clase, *OperacionesBDR*, de manera que sea fácil para el experto en SQL trabajar en SQL en un único lugar.

Otro caso de componentes con baja cohesión lo constituyen los objetos servidores distribuidos. Debido a implicaciones de costes fijos y rendimientos asociados con los objetos remotos y comunicaciones remotas, a veces, es deseable crear menos objetos servidores, de mayor tamaño y menos cohesivos que proporcionen una interfaz para muchas operaciones. Esto también está relacionado con el patrón denominado **Interfaz Remota de Grano Grueso**, en el cual, las operaciones remotas se hacen de grano más grueso con el objeto de realizar o solicitar más trabajo en las llamadas a operaciones remotas, debido a las penalizaciones de rendimiento de las llamadas remotas en la red. Como un simple ejemplo, en vez de un objeto remoto con tres operaciones de grano fino *setNombre*⁵, *setSalario*, y *setFechaAlquiler*, hay una operación remota *setDatos* que recibe un conjunto de datos. Esto da lugar a menos llamadas remotas y mejor rendimiento.

Beneficios

- Se incrementa la claridad y facilita la comprensión del diseño.
- Se simplifican el mantenimiento y las mejoras.
- Se soporta a menudo bajo acoplamiento.
- El grano fino de funcionalidad altamente relacionada incrementa la reutilización porque una clase cohesiva se puede utilizar para un propósito muy específico.

16.10. Controlador

Solución

Asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase que representa una de las siguientes opciones:

- Representa el sistema global, dispositivo o subsistema (*controlador de fachada*).
- Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo denominado <NombreDelCasoDeUso>Manejador, <NombreDelCasoDeUso>Coordinador o <NombreDelCasoDeUso>Sesion (*controlador de sesión o de caso de uso*).
 - Utilice la misma clase controlador para todos los eventos del sistema en el mismo escenario de caso de uso.
 - Informalmente, una sesión es una instancia de una conversación con un actor. Las sesiones pueden tener cualquier duración, pero se organizan a menudo en función de los casos de uso (sesiones de casos de uso).

Contraindicaciones

⁵ N. del T.: Para nombrar los métodos de una clase que asignan un valor a una propiedad denominada <X>, utilizaremos la habitual terminología set<X>.

Corolario: Nótese que las clases “ventana”, “applet”, “widget”, “vista” y “documento” no están en esta lista. Tales clases *no* deberían abordar las tareas asociadas con los eventos del sistema, normalmente, reciben estos eventos y los delegan a un controlador.

Problema ¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?

Un **evento del sistema** de entrada es un evento generado por un actor externo. Se asocian con **operaciones del sistema** —operaciones del sistema como respuesta a los eventos del sistema—, tal como se relacionan los mensajes y los métodos.

Por ejemplo, cuando un cajero utiliza un terminal PDV y presiona el botón “Finalizar Venta”, está generando un evento del sistema que indica que la “venta ha terminado”. Igualmente, cuando un escritor utiliza un procesador de texto y presiona el botón “comprobar ortografía”, está generando un evento del sistema que indica que se “ejecute una comprobación de la ortografía”.

Un **Controlador** es un objeto que no pertenece a la interfaz de usuario, responsable de recibir o manejar un evento del sistema. Un Controlador define el método para la operación del sistema.

Ejemplo En la aplicación NuevaEra, hay varias operaciones del sistema, como se ilustra en la Figura 16.13, que muestran al propio sistema como una clase o componente (que es legal en UML).

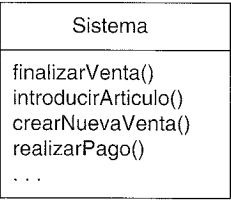


Figura 16.13. Operaciones del sistema asociadas con los eventos del sistema.

Durante el análisis, las operaciones del sistema pueden asignarse a la clase *Sistema*, para indicar que se trata de operaciones del sistema. Sin embargo, esto *no* significa que una clase software denominada *Sistema* las lleve a cabo durante el diseño. Más bien, durante el diseño, se asigna la responsabilidad de las operaciones del sistema a una clase Controlador (ver Figura 16.14).

¿Quién debería ser el controlador de los eventos del sistema como *introducirArticulo* y *finalizarVenta*?

Siguiendo el patrón Controlador, a continuación presentamos algunas de las opciones:

- | | |
|--|---|
| representa el “sistema” global,
dispositivo o subsistema | <i>Registro, SistemaPDV</i> |
| representa un receptor o manejador
de todos los eventos del sistema
de un escenario de caso de uso | <i>ProcesarVentaManejador</i>
<i>ProcesarVentaSesion</i> |

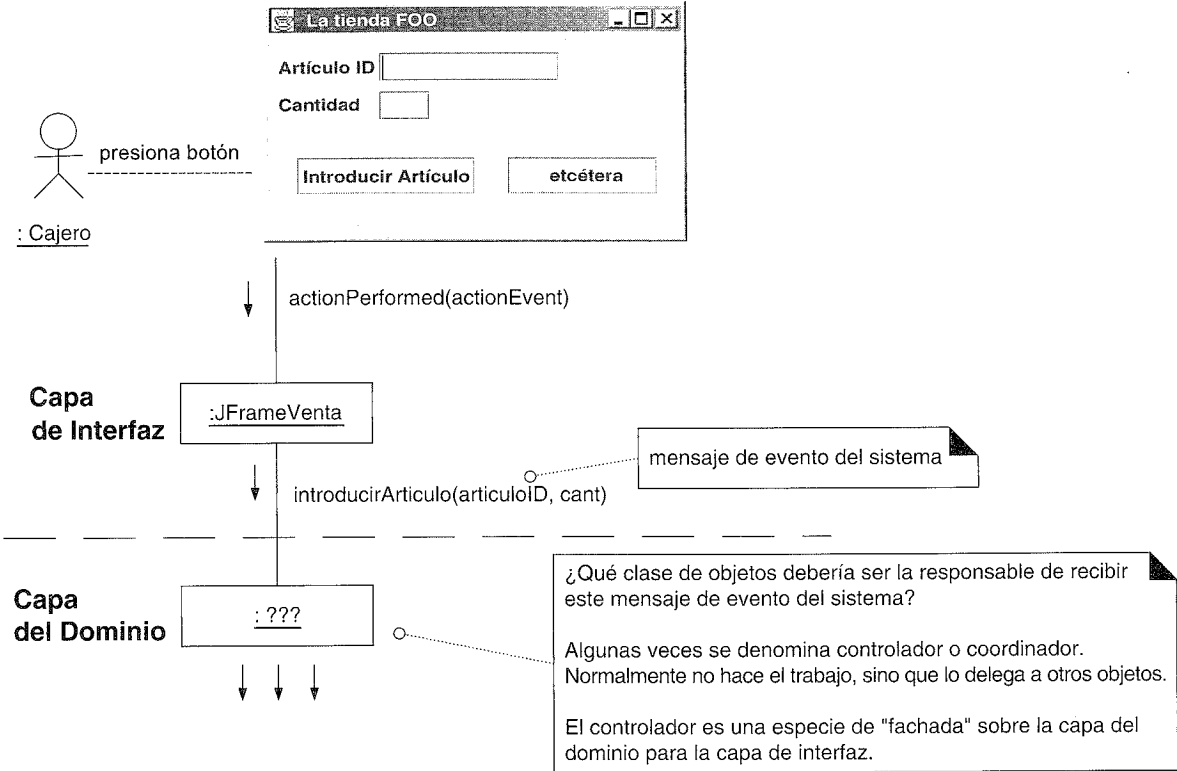


Figura 16.14. ¿Controlador para introducirArticulo?

En cuanto a los diagramas de interacción, significa que uno de los ejemplos de la Figura 16.15 podría ser útil.

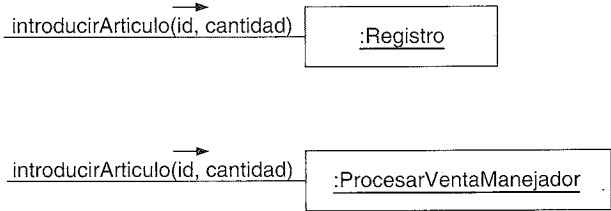


Figura 16.15. Opciones de Controlador.

En la elección de cuál de las clases es el controlador más apropiado influyen otros factores, que se estudiarán en la siguiente sección.

Durante el diseño, se asignan a una o más clases controlador las operaciones del sistema que se identificaron durante el análisis del comportamiento del sistema, como *Registro*, tal como se muestra en la Figura 16.16.

Los sistemas reciben eventos de entrada externos, normalmente a través de una GUI manejada por una persona. Otros medios de entrada pueden ser mensajes externos, como en un conmutador de telecomunicaciones para el procesamiento de llamadas, o señales desde sensores, como en sistemas de control de procesos.

Discusión

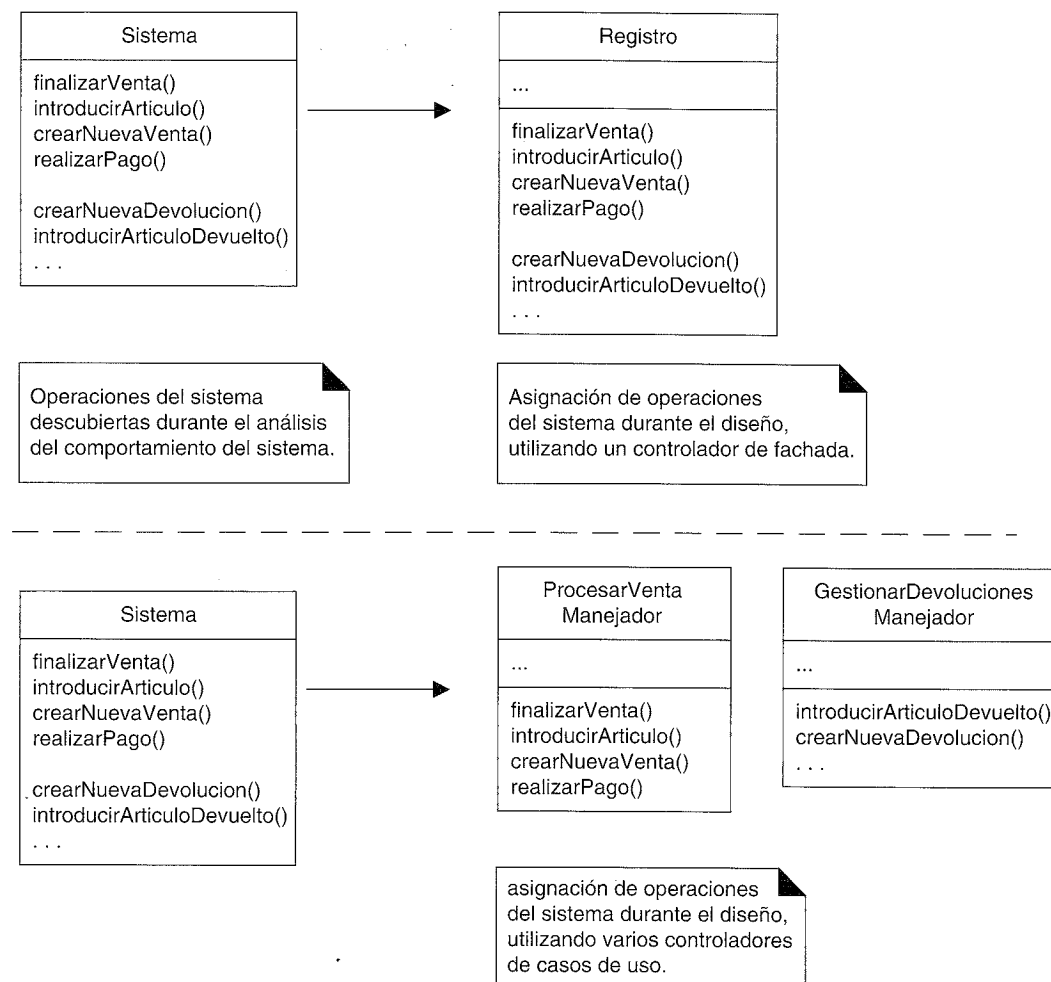


Figura 16.16. Asignación de las operaciones del sistema.

En todos los casos, si se utiliza un diseño de objetos, se debe escoger algún manejador para estos eventos. El patrón Controlador proporciona guías acerca de las opciones generalmente aceptadas y adecuadas. Como se ilustró en la Figura 16.14, el controlador es una especie de fachada en la capa del dominio para la capa de la interfaz.

A menudo, es conveniente utilizar la misma clase controlador para todos los eventos del sistema de un caso de uso de manera que es posible mantener la información acerca del estado del caso de uso en el controlador. Tal información es útil, por ejemplo, para identificar eventos del sistema que se apartan de la secuencia establecida (por ejemplo, una operación de *realizarPago* antes que la operación *finalizarVenta*). Podrían utilizarse diferentes controladores para casos de usos distintos.

Un error típico del diseño de los controladores es otorgarles demasiada responsabilidad.

Normalmente, un controlador debería *delegar* en otros objetos el trabajo que se necesita hacer; coordina o controla la actividad. No realiza mucho trabajo por sí mismo.

Por favor, diríjase a la sección “Cuestiones y Soluciones” que aparece más adelante para una discusión más elaborada.

La primera categoría de controlador es el controlador de fachada que representa al sistema global, dispositivo o subsistema. La idea es elegir algún nombre de clase que sugiera una cubierta, o fachada, sobre las otras capas de la aplicación, y que proporcione las llamadas a los servicios más importantes desde la capa de UI hacia las otras capas. Podría ser una abstracción de toda la unidad física, como un *Registro*⁶, *Conmutador-DeTelecomunicaciones*, *Telefono* o *Robot*; una clase que represente el sistema software completo, como un *SistemaPDV*, o cualquier otro concepto que el diseñador elija que represente al sistema o subsistema global, incluso, por ejemplo, *JuegoAjedrez* si fuera un software de juegos.

Los controladores de fachada son adecuados cuando no existen “demasiados” eventos del sistema, o no es posible que la interfaz de usuario (UI) redirija mensajes de los eventos del sistema a controladores alternativos, como un sistema de procesamiento de mensajes.

Si se elige un controlador de casos de uso, entonces hay un controlador diferente para cada caso de uso. Nótese que no es un objeto del dominio; es una construcción artificial para dar soporte al sistema (una *Fabricación Pura* en términos de los patrones GRASP). Por ejemplo, si la aplicación NuevaEra contiene casos de uso tales como *Procesar Venta* y *Gestionar Devoluciones*, entonces podría haber una clase *Procesar-VentaManejador* y así sucesivamente.

¿Cuándo se debería escoger un controlador de casos de uso? Es una alternativa a tener en cuenta cuando la asignación de las responsabilidades a un controlador de fachada nos conduce a diseños con baja cohesión o alto acoplamiento, generalmente cuando el controlador de fachada se está “inflando” con excesivas responsabilidades. Un controlador de casos de uso es una buena elección cuando hay muchos eventos del sistema repartidos en diferentes procesos; el controlador factoriza la gestión en clases separadas manejables, y también proporciona una base para conocer y razonar sobre el estado de los escenarios actuales en marcha.

En el UP y en el método más antiguo de Jacobson, Objectory [Jacobson92], existen los conceptos (opcionales) de clases frontera (*boundary*), control y entidad. Los **objetos frontera** son abstracciones de las interfaces, los **objetos entidad** son los objetos software del dominio independiente de la aplicación (y normalmente persistentes), y los **objetos control** son los manejadores de los casos de uso tal y como se describen en el patrón Controlador.

Un importante corolario del patrón Controlador es que los objetos interfaz (por ejemplo, los objetos ventana o elementos gráficos) y la capa de presentación no deberían ser responsables de llevar a cabo los eventos del sistema. En otras palabras, las operaciones del sistema se deberían manejar en la lógica de la aplicación o capas del dominio en lugar de en la capa de interfaz del sistema. Diríjase a la sección “Cuestiones y Soluciones” para ver un ejemplo.

⁶ Se utilizan varios términos para una unidad de PDV física, entre las que figuran: registro, terminal de punto de venta (TPDV), etcétera. A lo largo del tiempo, el “registro” ha llegado a encarnar la noción tanto de una unidad física, como la abstracción lógica de lo que registra ventas y pagos.

El objeto Controlador es normalmente un objeto del lado del cliente en el mismo proceso que la UI (por ejemplo, una aplicación con una GUI con las Swing de Java), entonces no es exactamente aplicable cuando el UI es un cliente Web en un navegador, y hay software del lado del servidor involucrado. En el último caso, existen varios patrones comunes para manejar los eventos del sistema que están fuertemente influenciados por el marco tecnológico escogido en el lado del servidor, como los servlets de Java. No obstante, es un estilo común crear controladores de casos de uso en el lado del servidor con o bien un servlet o un bean sesión EJB (*Enterprise JavaBeans*) para cada caso de uso. El objeto de sesión del lado del servidor representa una “sesión” de interacción con un actor externo.

Si la UI no es un cliente web (por ejemplo, es una GUI Swing o Windows), pero la aplicación invoca servicios remotos, es todavía común el uso del patrón Controlador. La UI reenvía la solicitud al Controlador local del lado del cliente, y el Controlador podría reenviar toda o parte de la gestión de la petición a los servicios remotos. Este diseño disminuye el acoplamiento entre la UI y los servicios remotos, y hace más fácil, por ejemplo, abastecer los servicios de manera local o remota, por medio de la indirección del Controlador del lado del cliente.

Resumiendo, el Controlador recibe la solicitud del servicio desde la capa de UI y coordina su realización, normalmente delegando a otros objetos.

Beneficios

- *Aumenta el potencial para reutilizar y las interfaces conectables (pluggable).* Asegura que la lógica de la aplicación *no* se maneja en la capa de interfaz. Técnicamente, las responsabilidades de un controlador podrían manejarse en un objeto interfaz, pero la implicación de tal diseño es que el código del programa y la lógica relacionada con la realización de la lógica de la aplicación estaría embebida en los objetos ventana o interfaz. Un diseño de una interfaz como controlador reduce la oportunidad de reutilizar la lógica en futuras aplicaciones, puesto que está ligada a una interfaz particular (por ejemplo, objetos ventana) que es raramente aplicable en otras aplicaciones. En cambio, delegando la responsabilidad de una operación del sistema a un controlador ayuda a la reutilización de la lógica en futuras aplicaciones. Y puesto que la lógica no está ligada a la capa de interfaz, puede sustituirse por una interfaz nueva.
- *Razonamiento sobre el estado de los casos de uso.* A veces es necesario asegurar que las operaciones del sistema tienen lugar en una secuencia válida, o ser capaces de razonar sobre el estado actual de la actividad y operaciones del caso de uso que está en marcha. Por ejemplo, podría ser necesario garantizar que la operación *realizarPago* no puede ocurrir hasta que haya tenido lugar la operación *finalizarVenta*. Si es así, es necesario capturar en algún sitio esta información de estado; el controlador es una opción razonable, especialmente si el mismo controlador se utiliza en todo el caso de uso (lo que se recomienda).

Cuestiones y Soluciones

Controladores saturados

Una clase controlador pobremente diseñada tendrá baja cohesión —no centrada en algo concreto y que gestiona demasiadas áreas de responsabilidad—; este controlador se denomina **controlador saturado**. Signos de que existe un controlador saturado son:

- Existe una *única* clase controlador que recibe *todos* los eventos del sistema en el sistema, y hay muchos. Esto ocurre a veces si se elige un controlador de fachada.
- El propio controlador realiza muchas de las tareas necesarias para llevar a cabo los eventos del sistema, sin delegar trabajo. Normalmente esto conlleva una violación de los patrones Experto en Información y Alta Cohesión.
- Un controlador tiene muchos atributos y mantiene información significativa sobre el sistema o el dominio, que debería haberse distribuido a otros objetos, o duplica información que se encuentra en otros sitios.

Hay varios remedios para un controlador saturado entre los que se encuentran:

1. Añadir más controladores: un sistema no tiene que tener sólo uno. En lugar de un controlador de fachada, utilice controladores de casos de uso. Por ejemplo, considere una aplicación con muchos eventos del sistema, como un sistema de reservas de vuelos.

Podría contener los siguientes controladores:

Controladores de casos de uso
RealizarReservaManejador
GestionarHorariosManejador
GestionarTarifasManejador

2. Diseñe el controlador de manera que, ante todo, delegue el cumplimiento de cada responsabilidad de una operación del sistema a otros objetos.

La capa de interfaz no maneja eventos del sistema

Reiterando: un corolario importante del patrón Controlador es que los objetos interfaz (por ejemplo, los objetos ventana) y la capa de interfaz, no deberían ser responsables de manejar los eventos del sistema. Como ejemplo, considere un diseño en Java que utiliza un *JFrame* para mostrar la información.

Asuma que la aplicación NuevaEra tiene una ventana que muestra la información de la venta y captura las operaciones del cajero. Utilizando el patrón Controlador, la Figura 16.17 ilustra una relación aceptable entre el *JFrame*, el Controlador y otros objetos en una parte del sistema de PDV (simplificado).

Nótese que la clase *JFrameVenta* —perteneciente a la capa de interfaz— pasa el mensaje *introducirArticulo* al objeto *Registro*. No se involucró en el procesamiento de la operación o en decidir cómo manejarla; la ventana sólo la delega a otra capa.

Asignando la responsabilidad de las operaciones del sistema a objetos en la capa de aplicación o del dominio —utilizando el patrón Controlador— en lugar de en la capa de interfaz incrementa el potencial para reutilizar. Si un objeto de la capa de interfaz (como el *JFrameVenta*) maneja una operación del sistema —que representa parte de un proceso de negocio— entonces la lógica del proceso del negocio estaría contenida en un

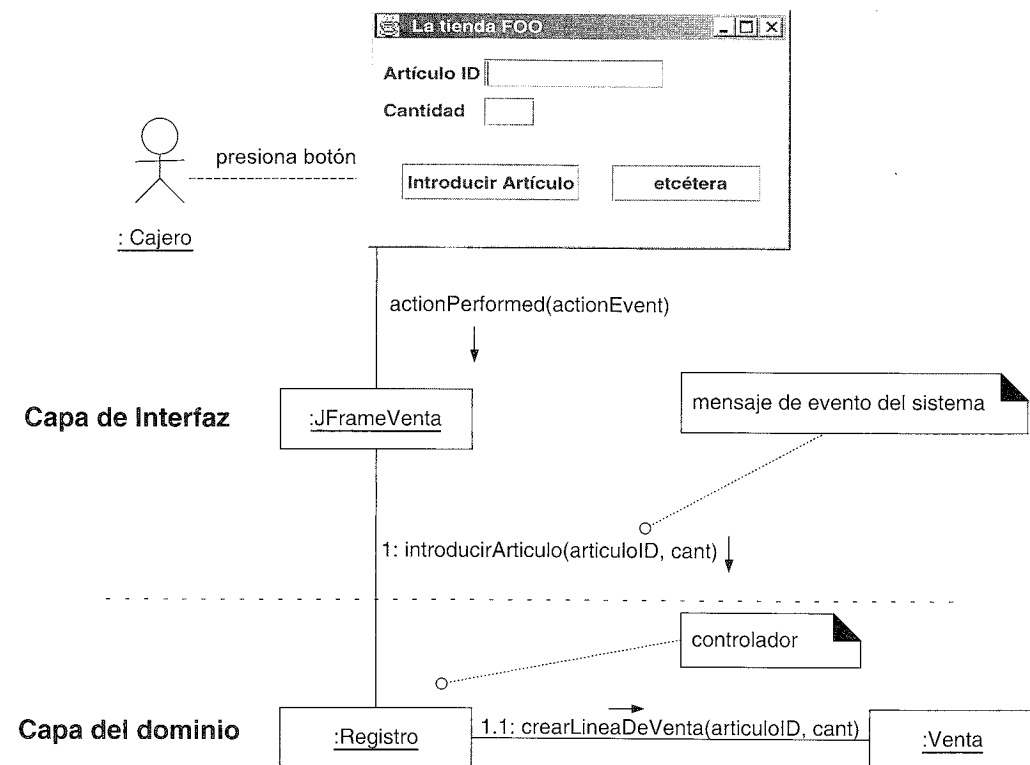


Figura 16.17. Acoplamiento deseable entre la capa de interfaz y la del dominio.

objeto interfaz (por ejemplo, del tipo ventana), por lo que la oportunidad para reutilizar es baja debido a su acoplamiento con una interfaz particular y con la aplicación.

En consecuencia, no es conveniente el diseño de la Figura 16.18.

La localización de la responsabilidad de las operaciones del sistema en un controlador que es un objeto del dominio facilita la reutilización de la lógica del programa lo que permite soportar el proceso de negocio asociado en futuras aplicaciones. También facilita la desconexión de la capa de interfaz y la utilización de un framework o tecnología de interfaz diferente o ejecutar el sistema en un modo “por lotes” sin conexión.

Sistemas de manejo de mensajes y el patrón *Command*

Algunas aplicaciones son sistemas de manejo de mensajes o servidores que reciben peticiones de otros procesos. Un conmutador de telecomunicaciones es un ejemplo típico. En tales sistemas, el diseño de la interfaz y el controlador es algo diferente. Los detalles se estudiarán en un capítulo posterior, pero en esencia, una solución típica es utilizar el patrón *Command* [GHJV95] y el patrón *Command Processor* [BMRSS96], que se presentarán en el Capítulo 34.

- **Command:** En un sistema de manejo de mensajes, cada mensaje podría representarse y manejarse mediante un objeto *Command* separado [GHJV95].
- **Fachada:** Un controlador de fachada es un tipo de Fachada (*Facade*) [GHJV95].

Patrones
Relacionados

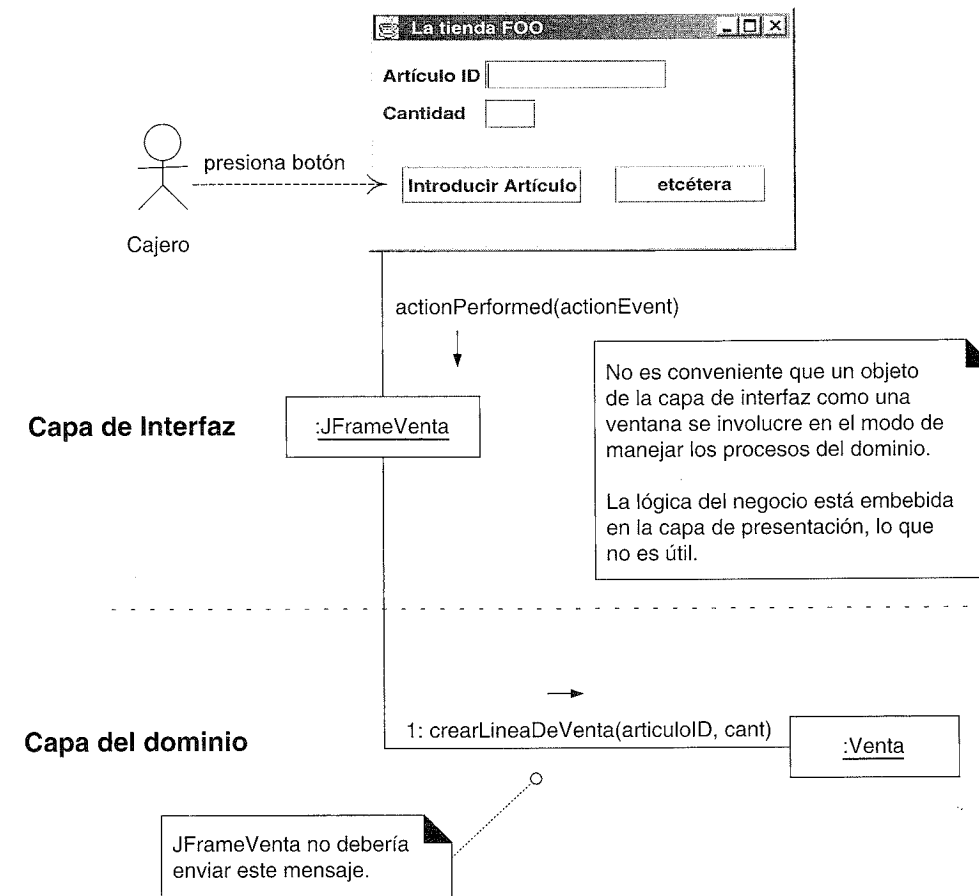


Figura 16.18. Acoplamiento menos conveniente entre la capa de interfaz y la del dominio.

- **Capas:** Es un patrón POSA [BMRSS96]. La ubicación de la lógica del dominio en la capa del dominio en lugar de en la capa de presentación forma parte del patrón de Capas (Layers).
- **Fabricación Pura:** Es otro patrón GRASP. Una Fabricación Pura es una creación arbitraria del diseñador, no una clase software cuyo nombre se inspira en el Modelo del Dominio. Un controlador de caso de uso es un tipo de Fabricación Pura.

16.11. Diseño de objetos y tarjetas CRC

Aunque formalmente no forma parte de UML, otro mecanismo que se utiliza algunas veces para ayudar a asignar responsabilidades e indicar las colaboraciones con otros objetos son las **tarjetas CRC** (tarjetas Clase-Responsabilidad-Colaborador) [BC89]. Kent Beck y Ward Cunningham fueron quienes promovieron el uso de estas tarjetas y son los principales responsables de estimular a los diseñadores de software a pensar de manera más abstracta en términos de asignación de responsabilidades y colaboraciones, y también del uso de los patrones.

Las tarjetas CRC son fichas, una por cada clase, en las que se escriben brevemente las responsabilidades de la clase, y una lista de los objetos con los que colabora para llevar a cabo esas responsabilidades. Se desarrollan normalmente en una sesión de trabajo en grupo pequeño. Los patrones GRASP se podrían aplicar cuando se tiene en cuenta el diseño mientras se utilizan las tarjetas CRC.

Las tarjetas CRC son una técnica para registrar los resultados de la asignación de responsabilidades y asignaciones. La información recopilada se puede enriquecer utilizando diagramas de clases y de interacción. Lo importante no son las tarjetas o los diagramas, sino tener presente la asignación de responsabilidades.

16.12. Lecturas adicionales

La metáfora de los objetos que colaboran con responsabilidades, o el **Diseño Dirigido por Responsabilidades**, surgió especialmente a partir del trabajo sobre objetos en Small-talk en Tektronix, Portland, de Kent Beck, Ward Cunningham, Rebecca Wirfs-Brock, y otros, que ha tenido gran influencia. El libro de texto que sirve de referencia es *Designing Object-Oriented Software* [WWW90], tan relevante hoy como cuando fue escrito.

Otros dos textos recomendados que destacan los principios fundamentales del diseño de objetos son *Object-Oriented Design Heuristics* de Riel y *Object Models* de Coad.

MODELO DE DISEÑO:
REALIZACIÓN DE CASOS DE USO
CON LOS PATRONES GRASP

Para inventar, necesitas una buena imaginación y un montón de trastos viejos.

Thomas Edison

Objetivos

- Diseñar las realizaciones de los casos de uso.
- Aplicar los patrones GRASP para asignar responsabilidades a las clases.
- Utilizar la notación de los diagramas de interacción UML para ilustrar el diseño de objetos.

Introducción

Este capítulo presenta cómo crear un diseño de objetos que colaboran con responsabilidades. Se presta especial atención a la aplicación de los patrones GRASP para desarrollar una solución bien diseñada. Por favor, obsérvese que los patrones GRASP como tales o por el nombre no son lo importante; son simplemente un apoyo al aprendizaje para ayudar a discutir y realizar de manera metódica el diseño de objetos fundamental.

Este capítulo expone los principios, utilizando el ejemplo del PDV NuevaEra, según los cuales un diseñador orientado a objetos asigna las responsabilidades y establece las interacciones entre los objetos —una técnica fundamental en el desarrollo orientado a objetos—.

Nota:

La asignación de responsabilidades y el diseño de colaboraciones son etapas muy importantes y creativas durante el diseño, mientras se elaboran los diagramas o mientras se programa.

El material es intencionalmente detallado; pretende ilustrar de manera exhaustiva que no existen decisiones “mágicas” o injustificadas en el diseño de objetos —la asignación de responsabilidades y la elección de las interacciones de objetos pueden explicarse y aprenderse de una forma racional—.

17.1. Realizaciones de casos de uso

Citando textualmente, “Una realización de caso de uso describe cómo se realiza un caso de uso particular en el modelo de diseño, en función de los objetos que colaboran” [RUP]. De manera más precisa, un diseñador puede describir el diseño de uno o más *escenarios* de un caso de uso; cada uno de estos se denomina una realización del caso de uso. La realización del caso de uso es un término o concepto del UP que se utiliza para recordarnos la conexión entre los requisitos expresados como casos de uso, y el diseño de objetos que satisface los requisitos.

Los diagramas de interacción UML son un lenguaje común para ilustrar las realizaciones de los casos de uso. Y como se estudió en el capítulo anterior, existen principios o patrones del diseño de objetos, como el Experto en Información y Bajo Acoplamiento, que se pueden aplicar durante este trabajo de diseño.

Como repaso, la Figura 17.20 (casi al final de este capítulo) expone las relaciones entre algunos artefactos del UP:

- El caso de uso sugiere los eventos del sistema que se muestran explícitamente en los diagramas de secuencia del sistema.
- Opcionalmente, podrían describirse los detalles de los efectos de los eventos del sistema en términos de los cambios de los objetos del dominio en los contratos de las operaciones del sistema.
- Los eventos del sistema representan los mensajes que inician los diagramas de interacción, los cuales representan el modo en el que los objetos interactúan para llevar a cabo las tareas requeridas —la realización del caso de uso—.
- Los diagramas de interacción comprenden la interacción de mensajes entre objetos software cuyos nombres se inspiran algunas veces en los nombres de las clases conceptuales del Modelo del Dominio, además de otras clases de objetos.

17.2. Comentarios sobre los artefactos

Diagramas de interacción y realizaciones de casos de uso

En la iteración actual estamos teniendo en cuenta varios escenarios y eventos del sistema tales como:

- *Procesar Venta*: *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta*, *realizarPago*.

Si se utilizan los diagramas de interacción para representar las realizaciones de los casos de uso, se necesitará un diagrama de colaboración diferente para mostrar el manejo de cada mensaje de evento del sistema. Por ejemplo (Figura 17.1):

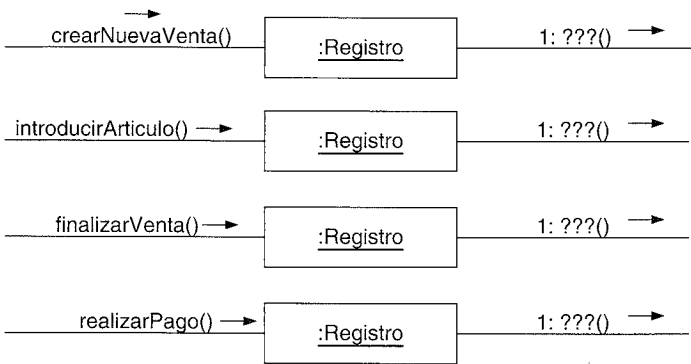


Figura 17.1. Diagramas de colaboración y manejo de los mensajes de eventos del sistema.

Por otro lado, si se utilizan los diagramas de secuencia, *podría* ser posible encajar todos los mensajes de eventos del sistema en el mismo diagrama, como en la Figura 17.2.

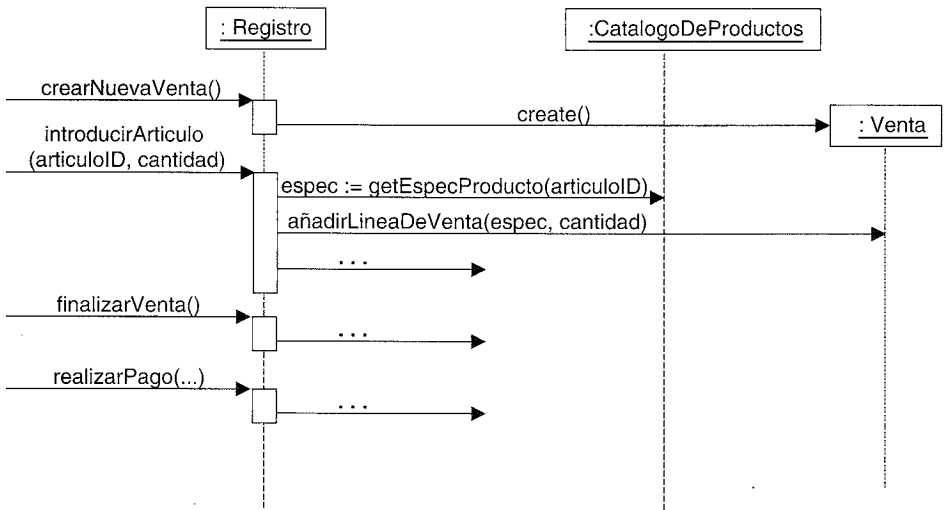


Figura 17.2. Un diagrama de secuencia y el manejo de los mensajes de eventos del sistema.

Sin embargo, ocurre a menudo que el diagrama de secuencia es entonces demasiado complejo o largo. Es legal, como con los diagramas de interacción, utilizar un diagrama de secuencia para cada mensaje de evento del sistema, como en la Figura 17.3.

Contratos y las realizaciones de casos de uso

Reiterando, podría ser posible diseñar las realizaciones de los casos de uso directamente a partir del texto de los casos de uso. Además, para algunas operaciones del sistema, se podrían haber escrito los contratos que añaden más detalles o son más específicos. Por ejemplo:

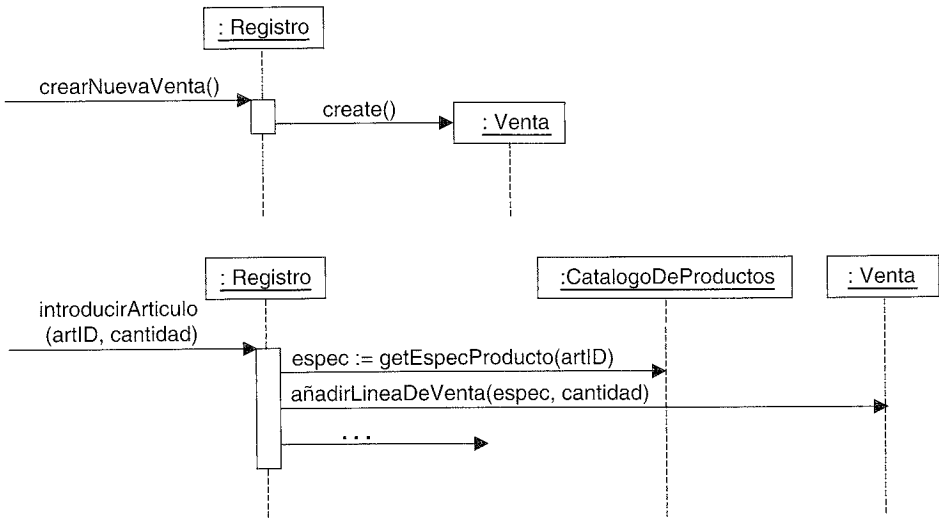


Figura 17.3. Múltiples diagramas de secuencia y manejo de los mensajes de eventos del sistema.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ...

Al mismo tiempo que tenemos en cuenta el texto de los casos de uso, para cada contrato, trabajamos cuidadosamente en expresar en el cambio de estado en las postcondiciones y diseñamos las interacciones para satisfacer los requisitos. Por ejemplo, dada esta operación parcial del sistema *introducirArticulo*, la Figura 17.4 muestra un diagrama de interacción parcial que satisface el cambio de estado de la creación de la instancia de *LineaDeVenta*.

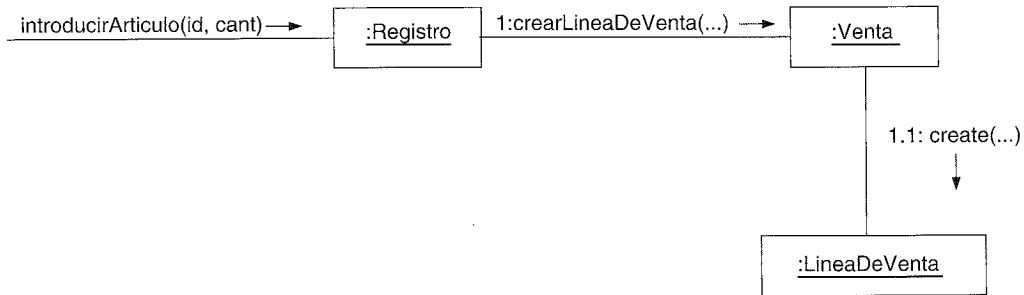


Figura 17.4. Diagrama de interacción parcial.

Advertencia: Los requisitos no son perfectos

Es útil tener presente que los casos de uso que se han escrito previamente y los contratos dan sólo una idea aproximada de lo que se tiene que conseguir. La historia del desarrollo del software nos va descubriendo invariablemente que los requisitos no son perfectos, o han cambiado. Esto no es una excusa para no tratar de hacer un buen trabajo de requisitos, sino el reconocimiento de que se necesita involucrar continuamente a los clientes y a los expertos en la materia en estudio para revisar y proporcionar retroalimentación sobre el comportamiento del sistema que se está desarrollando.

Una ventaja del desarrollo iterativo es que favorece de manera natural el descubrimiento de nuevos resultados del análisis y diseño durante el trabajo de diseño e implementación. El espíritu del desarrollo iterativo es capturar un grado “razonable” de información durante el análisis de requisitos, completando los detalles durante el diseño y la implementación.

El Modelo del Dominio y las realizaciones de los casos de uso

Algunos de los objetos software que interactúan mediante el paso de mensajes en los diagramas de interacción se inspiran en el Modelo del Dominio, como la clase conceptual *Venta* y la clase del diseño *Venta*. La elección de la asignación adecuada de la responsabilidad utilizando los patrones GRASP depende, en parte, de la información del Modelo del Dominio. Como ya se ha dicho, el Modelo del Dominio existente es probable que no sea perfecto; se espera que haya errores y omisiones. Se descubrirán nuevos conceptos que se olvidaron previamente, se ignorarán conceptos que se identificaron anteriormente, y lo mismo ocurrirá con las asociaciones y los atributos.

Clases conceptuales vs. clases del diseño

Recordemos que el Modelo del Dominio del UP no representa clases software, pero podría utilizarse para inspirar la presencia y los nombres de algunas clases software en el Modelo de Diseño. Durante la elaboración de los diagramas de interacción o la programación, los desarrolladores podrían mirar en el Modelo del Dominio para asignar los nombres a algunas clases del diseño; por tanto, se crea un diseño con un salto en la representación más bajo entre el diseño del software y nuestra percepción del dominio del mundo real con el que el software está relacionado (ver Figura 17.5).

¿Se deben limitar las clases del Modelo de Diseño a clases con nombres inspirados a partir del Modelo del Dominio? En absoluto, durante este trabajo de diseño es conveniente descubrir nuevas clases conceptuales que se obviaron durante el análisis de dominio inicial, y también crear clases software cuyos nombres y objetivos no estén relacionados en absoluto con el Modelo del Dominio.

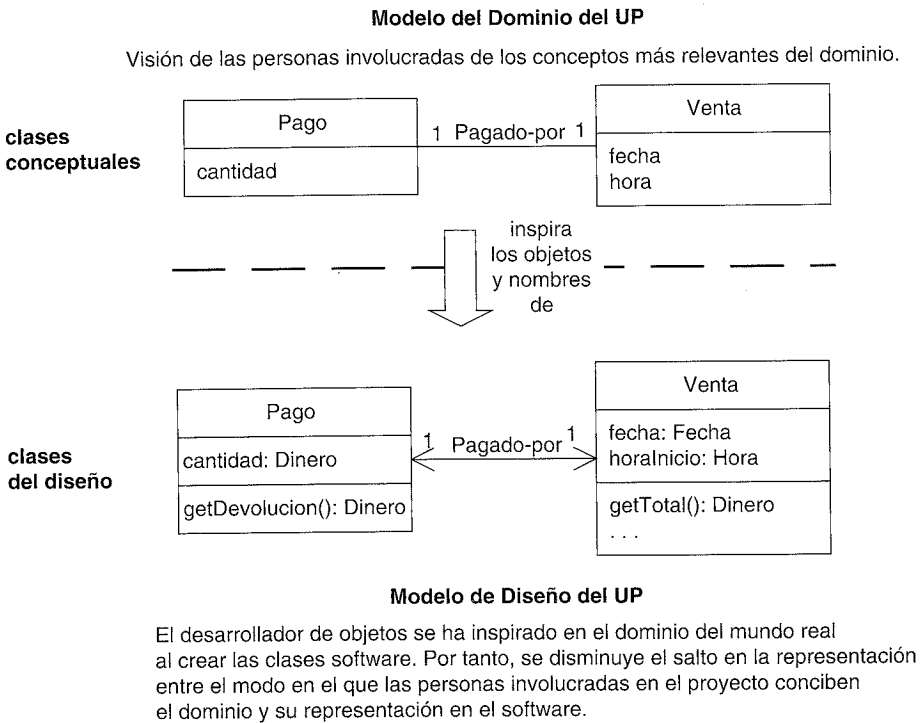


Figura 17.5. Disminución del salto en la representación nombrando las clases del diseño a partir de las clases conceptuales.

17.3. Realizaciones de casos de uso para la iteración de NuevaEra.

Las siguientes secciones exploran las elecciones y decisiones tomadas durante el diseño de una realización de caso de uso con objetos basado en los patrones GRASP. Las explicaciones son intencionalmente detalladas, en un intento de ilustrar que la creación de los diagramas de interacción bien diseñados no se trata de un “rollo” sin fundamento, sino que su construcción se basa en principios justificables.

En cuanto a la notación, el diseño de objetos para cada mensaje de eventos del sistema se representará en un diagrama independiente, para centrarse en las cuestiones de diseño de cada uno. Sin embargo, podrían haberse agrupado juntos en un único diagrama de secuencia.

17.4. Diseño de objetos: crearNuevaVenta

La operación del sistema *crearNuevaVenta* tiene lugar cuando un cajero solicita comenzar una nueva venta, después de que haya llegado un cliente con cosas que comprar. El caso de uso podría haber sido suficiente para decidir lo que era necesario, pero para este caso de estudio escribimos los contratos para todos los eventos del sistema, para que la explicación sea lo más completa posible.

Contrato CO1: crearNuevaVenta

Operación:	crearNuevaVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	ninguna
Postcondiciones:	- Se creó una instancia de Venta v (creación de instancias). - v se asoció con el Registro (formación de asociaciones). - Se inicializaron los atributos de v.

Elección de la clase controlador

Nuestra primera elección de diseño comprende la elección del controlador para el mensaje de operación del sistema *crearNuevaVenta*. De acuerdo con el patrón Controlador, algunas de las opciones podrían ser:

representa el “sistema” global, dispositivo o subsistema	<i>Registro, SistemaPDV</i>
representa un receptor o manejador de todos los eventos del sistema de un escenario de caso de uso	<i>ProcesarVentaManejador, ProcesarVentaSesion</i>

Es aceptable elegir un controlador de fachada como el *Registro* si sólo hay unas pocas operaciones del sistema y el controlador no está asumiendo demasiadas responsabilidades (en otras palabras, si va a perder la cohesión). Es adecuado elegir un controlador de caso de uso cuando hay muchas operaciones del sistema y deseamos distribuir las responsabilidades con el fin de mantener cada clase controlador ligera y centrada (en otras palabras, cohesiva). En este caso, *Registro* será suficiente, puesto que sólo hay unas pocas operaciones del sistema.

Este *Registro* es un objeto software del Modelo de Diseño. No es un registro físico real sino una abstracción del software cuyo nombre se eligió para disminuir el salto en la representación entre nuestra concepción del dominio y el software.

Por tanto, el diagrama de interacción que se muestra en la Figura 17.6, comienza enviando el mensaje *crearNuevaVenta* al objeto software *Registro*.

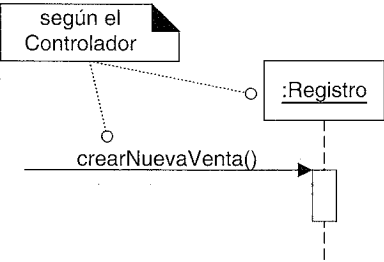


Figura 17.6. Aplicación del patrón GRASP Controlador.

Creación de una nueva Venta

Se debe crear un nuevo objeto software *Venta*, y el patrón GRASP Creador sugiere la asignación de la responsabilidad de creación a la clase que agrega, contiene o registra el objeto que se va a crear.

El análisis del Modelo del Dominio revela que se puede considerar que el *Registro* registra una *Venta*; de hecho, la palabra “registro” durante muchos años ha significado la cosa que graba (o registra) las transacciones contables, como las ventas.

Por tanto, el *Registro* es un candidato razonable para crear una *Venta*. Y consintiendo que el *Registro* cree la *Venta*, se puede asociar fácilmente a ella a lo largo del tiempo, de manera que durante futuras operaciones en la sesión, el *Registro* tendrá una referencia a la instancia de *Venta* actual.

Además de lo anterior, cuando se crea la *Venta*, se debe crear una colección vacía (contenedor, como una *List* de Java) para guardar todas las futuras instancias de *LineaDeVenta* que se añadirán. La instancia de *Venta* contendrá y mantendrá esta colección, lo que implica, según el Creador, que la *Venta* es una buena candidata para crearla.

Por tanto, el *Registro* crea la *Venta*, y la *Venta* crea una colección vacía, representada mediante un multiobjeto en el diagrama de interacción.

Por lo tanto, el diagrama de interacción de la Figura 17.7 ilustra el diseño.

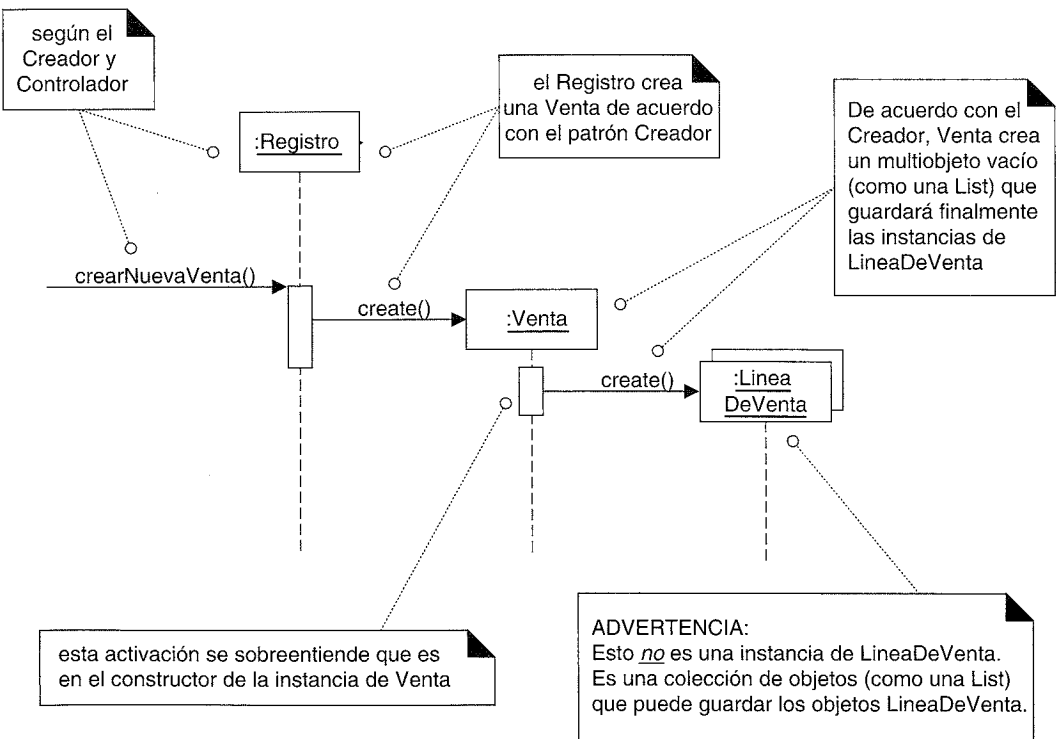


Figura 17.7. Creación de Venta y multiobjeto.

Conclusión

El diseño no fue difícil, pero lo importante de haberlo explicado de manera cuidadosa en términos del Controlador y el Creador era ilustrar que se pueden decidir y explicar, de manera racional y sistemática, los detalles de un diseño en base a principios y patrones, como los GRASP.

17.5. Diseño de objetos: introducirArticulo

La operación del sistema *introducirArticulo* tiene lugar cuando un cajero introduce el *articuloID* y (opcionalmente) la cantidad de ese artículo que se va a comprar. A continuación presentamos el contrato completo:

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).

Se construirá un diagrama de interacción que satisfaga la postcondición de *introducirArticulo*, utilizando los patrones GRASP para ayudar a tomar las decisiones de diseño.

Elección de la clase controlador

Nuestra primera elección tiene que ver con el manejo de la responsabilidad para el mensaje de operación del sistema *introducirArticulo*. Basado en el patrón Controlador, como para *crearNuevaVenta*, continuaremos utilizando el *Registro* como controlador.

¿Mostrar por pantalla la descripción y precio del artículo?

Debido a un principio de diseño denominado **Separación Modelo-Vista**, los objetos que no pertenecen a la GUI (como un *Registro* o *Venta*) no son responsables de involucrarse en las tareas de salida. Por tanto, aunque el caso de uso establezca que se muestran por pantalla la descripción y el precio después de la operación, el diseño lo ignorará en este momento.

Todo lo que se requiere con respecto a las responsabilidades de mostrar la información es que la información se conozca, lo que sucede en este caso.

Creación de una nueva LineaDeVenta

La postcondición del contrato *introducirArticulo* indica la creación, inicialización y asociación de una *LineaDeVenta*. El análisis del Modelo del Dominio revela que una *Venta* contiene objetos *LineaDeVenta*. Inspirándonos en el dominio, una *Venta* software podría contener igualmente objetos software *LineaDeVenta*. De aquí que, según el Creador, una *Venta* software sea una candidata adecuada para crear una *LineaDeVenta*.

La *Venta* puede asociarse con la *LineaDeVenta* recién creada almacenando la nueva instancia en su colección de líneas de venta. La postcondición indica que la nueva *LineaDeVenta*, cuando se crea, necesita una cantidad; por tanto, el *Registro* debe pasar esta cantidad a la *Venta*, quien debe pasarla como parámetro en el mensaje *create* (en Java, eso se implementaría como una llamada al constructor con un parámetro).

Por tanto, de acuerdo con el Creador, se envía a la *Venta* el mensaje *crearLineaDeVenta* para que cree una *LineaDeVenta*. La *Venta* crea una *LineaDeVenta*, y después almacena la nueva instancia en su colección permanente.

Los parámetros del mensaje *crearLineaDeVenta* incluyen la *cantidad*, de manera que la *LineaDeVenta* pueda registrarla, y del mismo modo la *EspecificacionDelProducto* que se corresponde con el *articuloID*.

Localización de una EspecificacionDelProducto

La *LineaDeVenta* necesita asociarse con la *EspecificacionDelProducto* que se corresponde con el *articuloID* de entrada. Esto implica que es necesario recuperar una *EspecificacionDelProducto*, en base a la coincidencia de un *articuloID*.

Antes de considerar *cómo* realizar la búsqueda, es conveniente considerar *quién* debe ser el responsable de ella. Por tanto, el primer paso es:

Comience asignando responsabilidades estableciendo claramente la responsabilidad.

Volviendo a plantear el problema:

¿Quién debe ser el responsable de conocer una *EspecificacionDelProducto*, basada en la coincidencia de un *articuloID*?

Esto no es ni un problema de creación ni uno sobre la elección de un controlador para un evento del sistema. Vamos a ver la primera aplicación del Experto en Información en el diseño.

En muchos casos, el patrón Experto es el principal patrón que se aplica. El Experto en Información sugiere que el objeto que tiene la información que se requiere para abordar la responsabilidad debería llevarla a cabo. ¿Quién conoce todos los objetos *EspecificacionDelProducto*?

El análisis del Modelo del Dominio revela que el *CatalogoDeProductos* lógicamente contiene todas las instancias *EspecificacionDelProducto*. Una vez más, inspirándonos en el dominio, diseñamos las clases software con una organización parecida: un *CatalogoDeProductos* software contendrá objetos software *EspecificacionDelProducto*.

Con eso decidido, entonces según el Experto en Información el *CatalogoDeProductos* es un buen candidato para esta responsabilidad de búsqueda puesto que conoce todos los objetos *EspecificacionDelProducto*.

Esto podría implementarse, por ejemplo, con un método denominado *getEspecificacion*¹.

Visibilidad del CatalogoDeProductos

¿Quién debería enviar el mensaje *getEspecificacion* al *CatalogoDeProductos* para solicitar una *EspecificacionDelProducto*?

Es razonable asumir que se crearon un *Registro* y un *CatalogoDeProductos* durante el caso de uso inicial *Poner en Marcha*, y que hay una conexión permanente desde el objeto *Registro* al objeto *CatalogoDeProductos*. Con este supuesto (que podríamos anotar en una lista de tareas de cosas que debemos asegurar en el diseño cuando vayamos a diseñar la inicialización), entonces es posible que el *Registro* envíe el mensaje *getEspecificacion* al *CatalogoDeProductos*.

Esto implica otro concepto en el diseño de objetos: la visibilidad. La **visibilidad** es la capacidad de un objeto de “ver” o tener una referencia a otro objeto.

Para que un objeto envíe un mensaje a otro objeto éste tiene que ser visible a aquél.

Puesto que asumiremos que el *Registro* tiene una conexión permanente —o referencia— al *CatalogoDeProductos*, éste es visible al *Registro* y, por tanto, puede enviarle un mensaje como el de *getEspecificacion*.

El siguiente capítulo abordará el tema de la visibilidad con más detenimiento.

Recuperación de objetos EspecificacionDelProducto de una base de datos

En la versión final de la aplicación del PDV NuevaEra, es improbable que todas las instancias de *EspecificacionDelProducto* se encuentren realmente en memoria. Será más probable que se almacenen en una base de datos relacional u objetual y se recuperarán bajo demanda; algunas podrían almacenarse en el proceso del cliente por motivos de rendimiento y tolerancia a fallos. Sin embargo, las cuestiones relacionadas con la recuperación de una base de datos se pospondrán por ahora en aras de la simplicidad. Se asumirá que todos los objetos *EspecificacionDelProducto* se encuentran en memoria.

El Capítulo 34 presentará el tema del acceso a base de datos de los objetos persistentes, que es un tema amplio en el que influye normalmente la elección de la tecnología, como J2EE, .NET, etcétera.

¹ La asignación de los nombres a los métodos de acceso es, por supuesto, una cuestión de estilo de cada lenguaje. Java siempre utiliza la forma *objeto.getFoo()*, C++ tiende a utilizar *objeto.foo()*, y C# utiliza *objeto.Foo*, lo que oculta (como Eiffel y Ada) si es una llamada a un método o un acceso directo a un atributo público. En los ejemplos se utiliza el estilo de Java.

El diseño de objetos de introducirArticulo

Dada la discusión anterior, el diagrama de interacción de la Figura 17.8 refleja las decisiones en cuanto a la asignación de responsabilidades y el modo en el que deberían interactuar los objetos. Obsérvese la considerable reflexión que se hizo para llegar a este diseño, basada en los patrones GRASP; el diseño de las interacciones entre objetos y la asignación de responsabilidades requiere alguna deliberación.

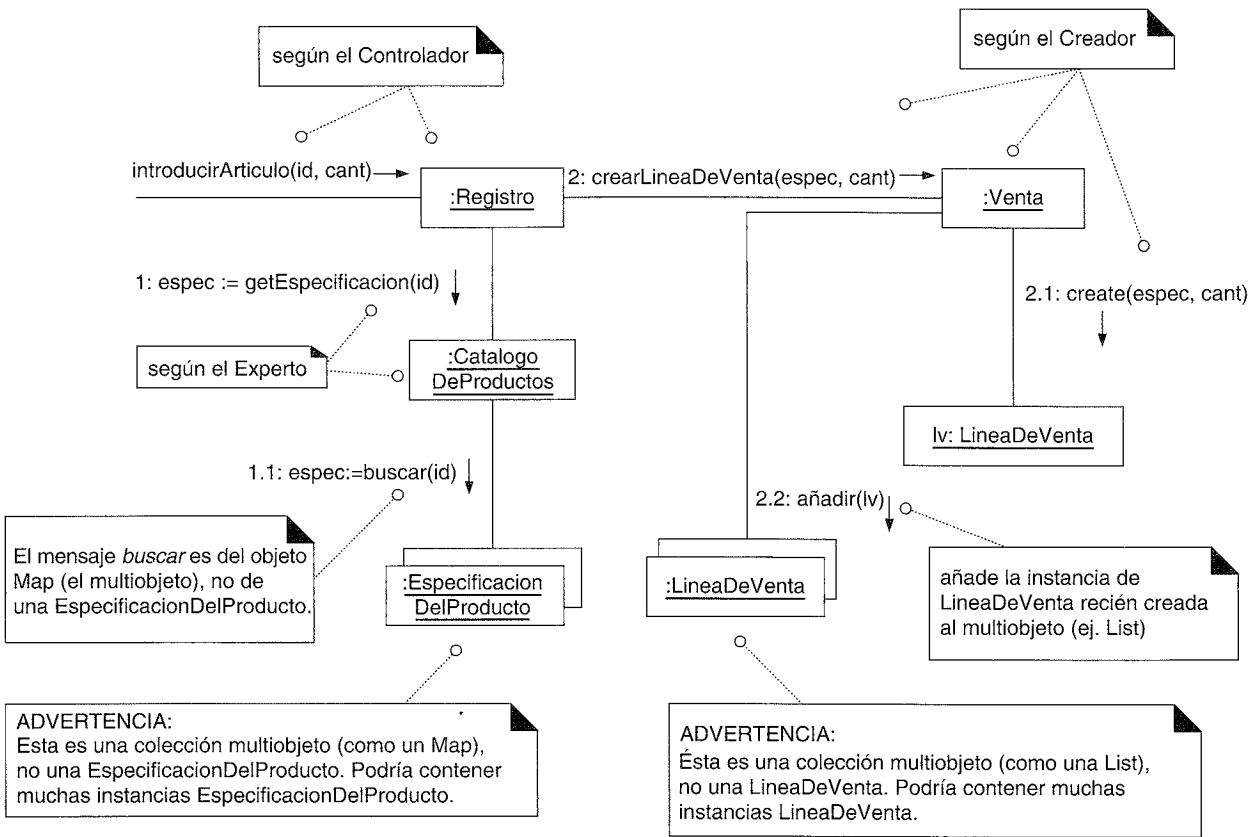


Figura 17.8. El diagrama de interacción para introducirArticulo.

Mensajes a los multiobjetos

Nótese que la interpretación en UML del envío de un mensaje a un multiobjeto es que se trata de un mensaje a la propia colección de objetos, en vez de una transmisión implícita a los miembros de la misma. Esto es especialmente obvio para las operaciones de colección genéricas como buscar y añadir.

- Por ejemplo, en el diagrama de interacción de introducirArticulo:
- El mensaje buscar enviado al multiobjeto EspecificacionDelProducto es un mensaje que se envía una vez a la estructura de datos de la colección representada por el multiobjeto (como un Map de Java).
 - El mensaje genérico e independiente del lenguaje buscar se traducirá, durante la programación, para un lenguaje específico y librería. Quizás será finalmente

Map.get en Java. Podría haberse utilizado el mensaje get en los diagramas; se utilizó buscar para hacer entender que los diagramas de diseño podrían requerir alguna traducción a diferentes lenguajes y librerías.

- El mensaje añadir enviado al multiobjeto LineaDeVenta es para añadir un elemento a la estructura de datos de la colección representada por un multiobjeto (como una List de Java).

17.6. Diseño de objetos: finalizarVenta

La operación del sistema finalizarVenta tiene lugar cuando un cajero presiona un botón indicando el final de la venta. A continuación presentamos el contrato:

Contrato CO3: finalizarVenta

Operación:	finalizarVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Venta.esCompleta pasó a ser verdad (modificación de atributos).

Elección de la clase controlador

Nuestra primera elección tiene que ver con el manejo de la responsabilidad para el mensaje de operación del sistema finalizarVenta. Basado en el patrón GRASP Controlador, como para introducirArticulo, continuaremos utilizando el Registro como controlador.

Valor del atributo Venta.esCompleta

- La postcondición del contrato establece que:
- Venta.esCompleta pasó a ser verdad (modificación de atributos).
- Como siempre, el primer patrón a tener en cuenta debería ser el Experto, a menos que sea un problema de creación o del controlador (que no lo es).
- ¿Quién debería ser el responsable de poner el valor del atributo esCompleto de la Venta a verdad?

Según el Experto, debería ser la propia Venta, puesto que conoce y mantiene el atributo esCompleto. Por tanto, el Registro enviará un mensaje seHaCompletado a la Venta para asignarle el valor true.

Notación UML para mostrar las restricciones, notas y algoritmos

La Figura 17.9 muestra el mensaje seHaCompletado, pero no pone de manifiesto lo que ocurre en el método seHaCompletado (aunque en este caso se reconoce que es trivial). Algunas veces en UML deseamos utilizar texto para describir el algoritmo de un método, o especificar alguna restricción.

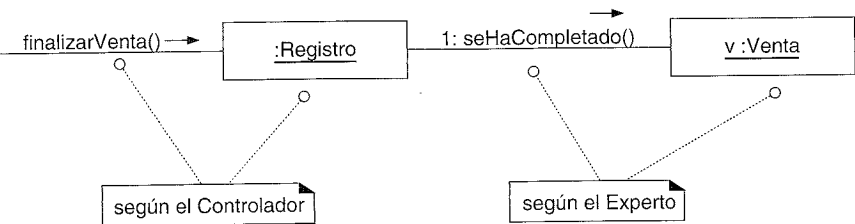


Figura 17.9. Finalización de la entrada de un artículo.

Para estas necesidades, UML proporciona tanto **restricciones** como **notas**. Una restricción UML es una información semánticamente significativa que se anexa a un elemento del modelo. Las restricciones en UML son texto encerrado entre llaves { }; por ejemplo, { x > 20 }. Se puede utilizar cualquier lenguaje formal o informal para las restricciones, y UML incluye especialmente **OCL** (lenguaje de restricciones de objetos) [WK99] si uno desea utilizarlo.

Una nota de UML es un comentario que no tiene impacto semántico, como la fecha de la creación o el autor.

Una nota siempre se muestra en un **cuadro de nota** (un cuadro de texto con la esquina doblada).

Una restricción podría mostrarse como un simple texto entre llaves, que es adecuado para declaraciones cortas. Sin embargo, las restricciones largas también podrían colocarse en un “cuadro de nota”, en cuyo caso el presunto cuadro de nota realmente contiene una restricción en lugar de una nota. El texto del cuadro va entre llaves para indicar que es una restricción.

En la Figura 17.10 se utilizan ambos estilos. Nótese que el estilo de restricción simple (entre llaves pero no en un cuadro) solamente muestra una sentencia que debe ser verdad (el significado clásico de una restricción en lógica). Por otro lado, la “restricción” en el cuadro de nota muestra la implementación del método Java de la restricción. Ambos estilos son legales para representar una restricción en UML.

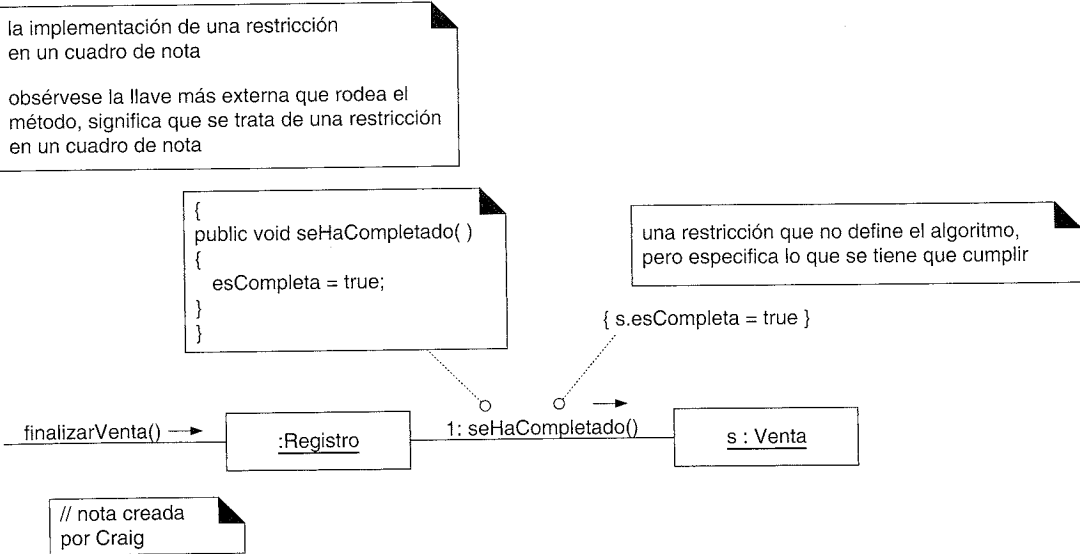


Figura 17.10. Restricciones y notas.

Cálculo del total de la Venta

Considere el siguiente fragmento del caso de uso *Procesar Venta*:

Escenario principal de éxito (o Flujo Básico):

- 1. El Cliente llega...
- 2. El Cajero le dice al Sistema que cree una nueva venta
- 3. El Cajero introduce el identificador del artículo
- 4. El Sistema registra la línea de la venta y ...
El Cajero repite los pasos 3-4 hasta que se indique
- 5. El Sistema presenta el total con los impuestos calculados.

En el paso 5, se presenta (o se muestra por pantalla) un total. Debido al principio de Separación Modelo-Vista, no deberíamos preocuparnos por el diseño de cómo se mostrará el total de la venta, pero es necesario asegurar que se conoce el total. Nótese que ninguna clase de diseño actualmente conoce el total de la venta, de manera que necesitamos crear un diseño de interacciones de objetos que satisfaga este requisito.

Como siempre, el Experto en Información debería ser un patrón a tener en cuenta a no ser que se trate de un problema de controlador o de creación (que no lo es).

Probablemente es obvio que la propia *Venta* deba ser la responsable de conocer su total, pero considere el siguiente análisis sólo con el fin de hacer el proceso de razonamiento para encontrar un Experto transparente como el cristal —con un ejemplo sencillo—.

- 1. Establezca la responsabilidad:
 - ¿Quién debe ser el responsable de conocer el total de la venta?
- 2. Reúna la información necesaria:
 - El total de la venta es la suma de los subtotales de todas las líneas de venta.
 - El subtotal de la línea de venta := cantidad de la línea de venta * precio de la descripción del producto.
- 3. Liste la información requerida para abordar esta responsabilidad y las clases que conocen esta información.

Información requerida para el total de la Venta	Experto en Información
EspecificacionDelProducto.precio	EspecificacionDelProducto
LineaDeVenta.cantidad	LineaDeVenta
Todas las LineasDeVenta de la Venta actual	Venta

A continuación presentamos un análisis detallado:

- ¿Quién deber ser responsable del cálculo del total de la *Venta*? Según el Experto, debería ser la propia *Venta*, puesto que conoce todas las instancias de *LineaDeVenta* cuyos subtotales se deben sumar para calcular el total de la venta. Por tanto, la *Venta* tendrá la responsabilidad de conocer su total, implementada como un método *getTotal*.

- Para que una *Venta* calcule su total, necesita el subtotal de cada *LineaDeVenta*. ¿Quién debe ser responsable de crear el subtotal de la *LineaDeVenta*? Según el Experto, debería ser la propia *LineaDeVenta*, puesto que conoce la cantidad y la *EspecificacionDelProducto* con la que está asociada. Por tanto, la *LineaDeVenta* tendrá la responsabilidad de conocer su subtotal, implementada como un método *getSubtotal*.
- Para que una *LineaDeVenta* calcule su subtotal, necesita el precio de la *EspecificacionDelProducto*. ¿Quién debería ser responsable de proporcionar el precio de la *EspecificacionDelProducto*? Según el Experto, debería ser la propia *EspecificacionDelProducto*, puesto que encapsula el precio como un atributo. Por tanto, la *EspecificacionDelProducto* tendrá la responsabilidad de conocer su precio, implementada como una operación *getPrecio*.

Aunque el análisis anterior es trivial en este caso, y el atormentador grado de elaboración presentado está fuera de lugar para el diseño que nos ocupa, la misma estrategia de razonamiento para encontrar un Experto puede y debe aplicarse en situaciones más difíciles. Descubrirá que una vez que aprenda estos principios puede hacer rápidamente esta clase de razonamiento mentalmente.

El diseño de *Venta--getTotal*

Dada la anterior discusión, es conveniente ahora construir un diagrama de interacción que ilustre lo que ocurre cuando se envía el mensaje *getTotal* a la *Venta*. El primer mensaje de este diagrama es *getTotal*, pero observe que el mensaje *getTotal* no es un evento del sistema.

Eso nos lleva a la siguiente observación:

No todos los diagramas de interacción comienzan con un mensaje de evento del sistema; pueden comenzar con cualquier mensaje para el que el diseñador desee mostrar las interacciones.

El diagrama de interacción se muestra en la Figura 17.11. Primero se envía el mensaje *getTotal* a la instancia de *Venta*. La *Venta* entonces enviará un mensaje *getSubtotal* a cada instancia de *LineaDeVenta* relacionada. La *LineaDeVenta* enviará sucesivamente un mensaje *getPrecio* a las instancias *EspecificacionDelProducto* asociadas.

Puesto que la aritmética (usualmente) no se ilustra mediante mensajes, los detalles de los cálculos se pueden ilustrar adjuntando al diagrama algoritmos o restricciones que definan los cálculos.

¿Quién enviará el mensaje *getTotal* a la *Venta*? Lo más probable es que sea un objeto de la capa de UI, como un *JFrame* de Java.

Obsérvese en la Figura 17.12 el uso de las notas de algoritmos y restricciones, para exponer los detalles de *getTotal* y *getSubtotal*.

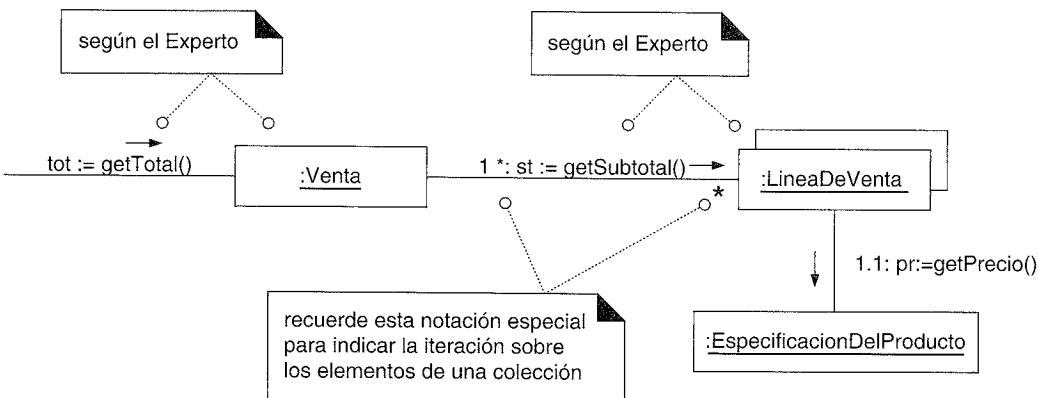


Figura 17.11. Diagrama de interacción *Venta--getTotal*.

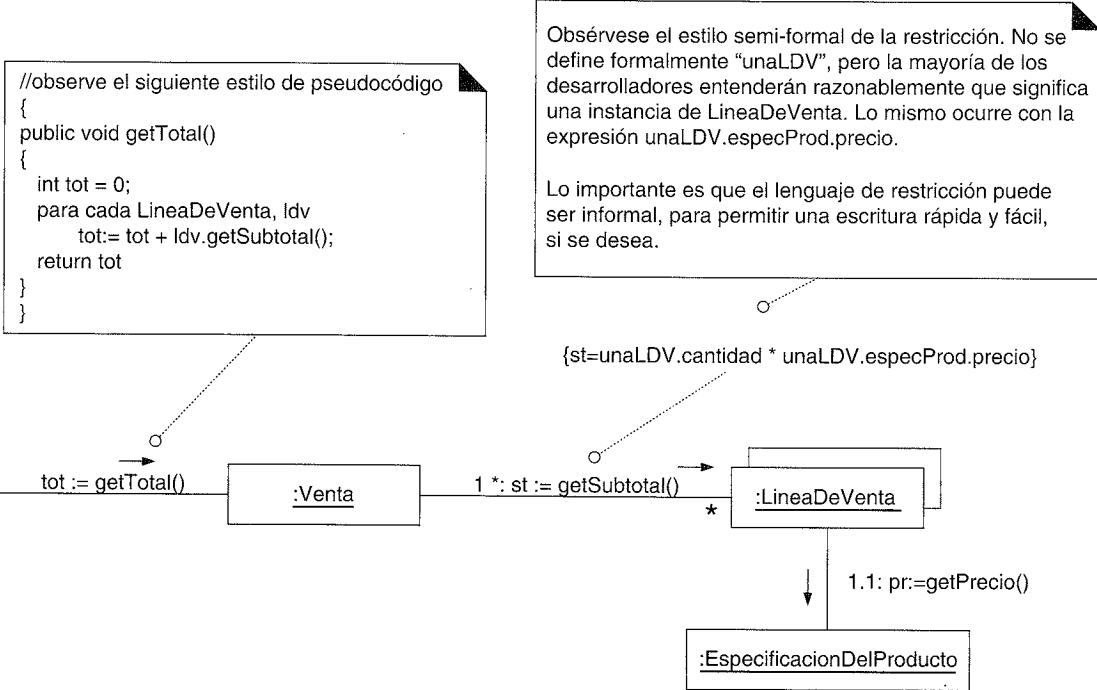


Figura 17.12. Notas de algoritmos y restricciones.

17.7. Diseño de objetos: *realizarPago*

La operación del sistema *realizarPago* tiene lugar cuando un cajero introduce la cantidad de dinero entregada para el pago. A continuación presentamos el contrato completo:

Contrato CO4: *realizarPago*

Operación: realizarPago(cantidad: Dinero)
Referencias cruzadas: Caso de Uso: Procesar Venta

Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de Pago p (creación de instancias). - p.cantidadEntregada pasó a ser cantidad (modificación de atributos). - p se asoció con la Venta actual (formación de asociaciones). - La Venta actual se asoció con la Tienda (formación de asociaciones); (para añadirlo al registro histórico de las ventas completadas).

Se construirá un diseño que satisfaga la postcondición de *realizarPago*.

Creación del Pago

Una de las postcondiciones del contrato establece:

- Se creó una instancia de *Pago p* (creación de instancias).

Ésta es una responsabilidad de creación, de manera que se debería aplicar el patrón GRASP Creador.

¿Quién registra, agrega, utiliza más estrechamente, o contiene un *Pago*? Existe un cierto atractivo en establecer que un *Registro* lógicamente registra un *Pago*, porque en el dominio real un “registro” recopila la información contable, luego es un candidato según el objetivo de reducir el salto en la representación en el diseño del software. Adicionalmente, es razonable esperar que una *Venta* software utilizará estrechamente un *Pago*; por tanto, podría ser una candidata.

Otra forma de encontrar un creador es utilizar el patrón Experto en función de quién es el Experto en Información con respecto a los datos de inicialización —la cantidad entregada en este caso—. El *Registro* es el controlador que recibe el mensaje de la operación del sistema *realizarPago*, luego, inicialmente tendrá la cantidad entregada. En consecuencia, el *Registro* es de nuevo un candidato.

Resumiendo, hay dos candidatos:

- *Registro*.
- *Venta*.

Ahora, esto nos lleva a una idea de diseño clave:

Cuando hay elecciones de diseño alternativas, mire detenidamente las implicaciones en cuanto a la cohesión y el acoplamiento de las alternativas, y posiblemente también considere las futuras presiones de evolución de las alternativas. Elija una alternativa con buena cohesión, acoplamiento y estabilidad ante posibles cambios futuros.

Considere algunas de las implicaciones de estas elecciones en función de los patrones GRASP Alta Cohesión y Bajo Acoplamiento. Si se elige la *Venta* para crear el *Pago*, es más ligero el trabajo (o responsabilidades) del *Registro* —dando lugar a una defini-

ción de *Registro* más simple—. También, el *Registro* no necesita conocer la existencia de una instancia de *Pago* porque se puede registrar indirectamente por medio de la *Venta* —lo que produce una disminución del acoplamiento en el *Registro*—. Esto nos lleva al diseño que se muestra en la Figura 17.13.

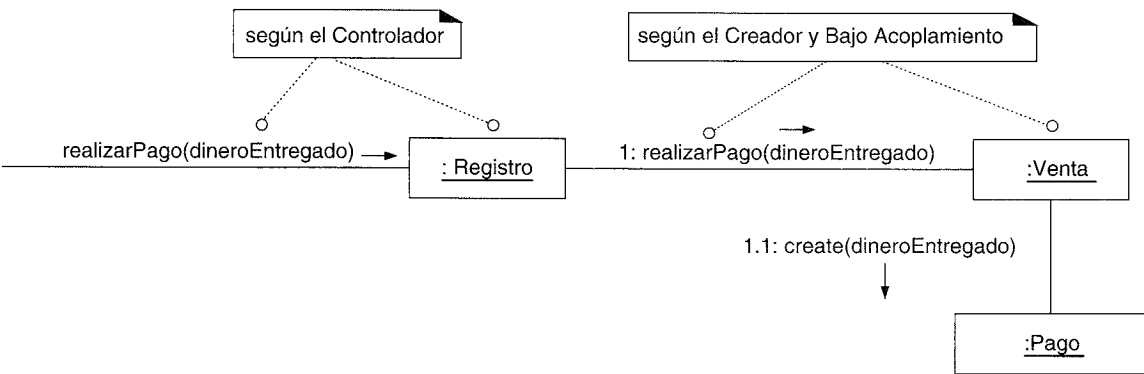


Figura 17.13. Diagrama de interacción Registro--realizarPago.

Este diagrama de interacción satisface la postcondición del contrato: se ha creado el *Pago*, asociado con la *Venta*, y se asigna el valor a su *cantidadEntregada*.

Registro de una Venta

Una vez completada, los requisitos establecen que la venta se debería colocar en un registro de históricos. Como siempre, el Experto en Información debería ser uno de los primeros patrones a tener en cuenta a menos que sea un problema de controlador o de creación (que no lo es), y se debería establecer la responsabilidad:

¿Quién es el responsable de conocer todas las ventas registradas y hacer el apunte en el registro?

Según el objetivo de salto en la representación bajo en el diseño del software (en relación con nuestros conceptos del dominio) es razonable que una *Tienda* conozca todas las ventas registradas, puesto que están fuertemente relacionadas con sus asuntos financieros. Otras alternativas comprenden conceptos clásicos de contabilidad, como *LibroMayorDeVentas*. Tiene sentido utilizar un objeto *LibroMayorDeVentas* cuando el diseño crece y la tienda pierde la cohesión (ver Figura 17.14).

Nótese también que la postcondición del contrato indica que se relacione la *Venta* con la *Tienda*. Éste es un ejemplo donde las postcondiciones podría no ser lo que realmente queremos conseguir en el diseño. Quizás no pensamos en el *LibroMayorDeVentas* antes, pero ahora que lo tenemos, elegimos usarlo en lugar de una *Tienda*. Si éste fuera el caso, idealmente se añadiría también el *LibroMayorDeVentas* al Modelo del Dominio, ya que es un nombre de un concepto en el dominio del mundo real. Son de esperar este tipo de descubrimientos y cambios durante el trabajo de diseño.

En este caso, nos mantendremos fieles al plan original de utilizar la *Tienda* (ver Figura 17.15).

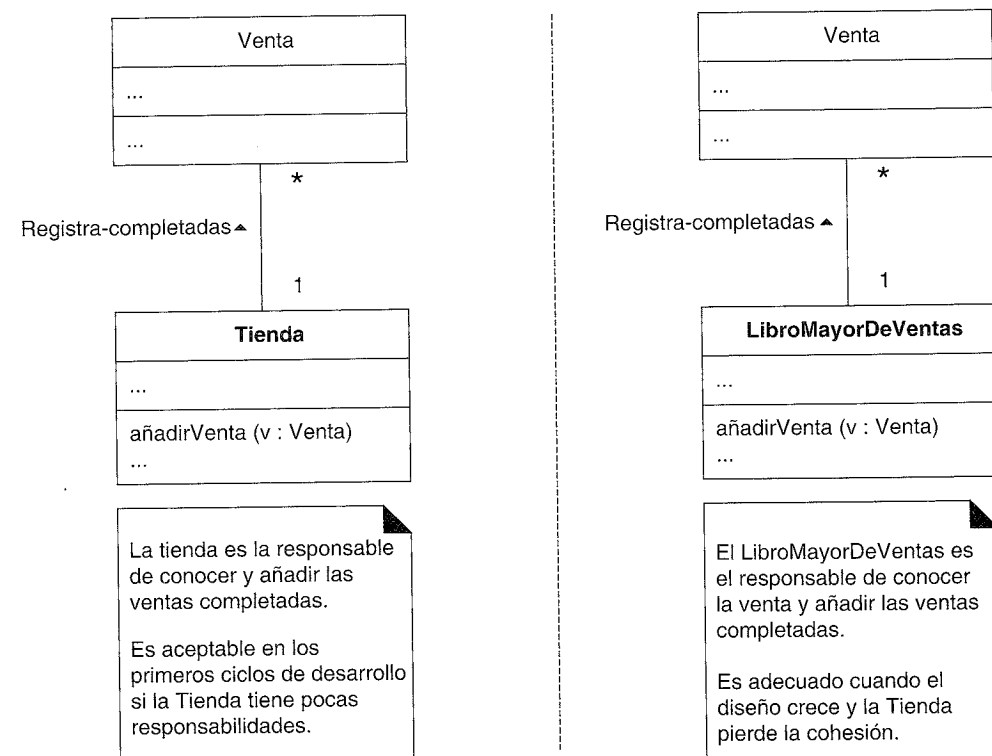


Figura 17.14. ¿Quién debería ser responsable de conocer la venta completada?

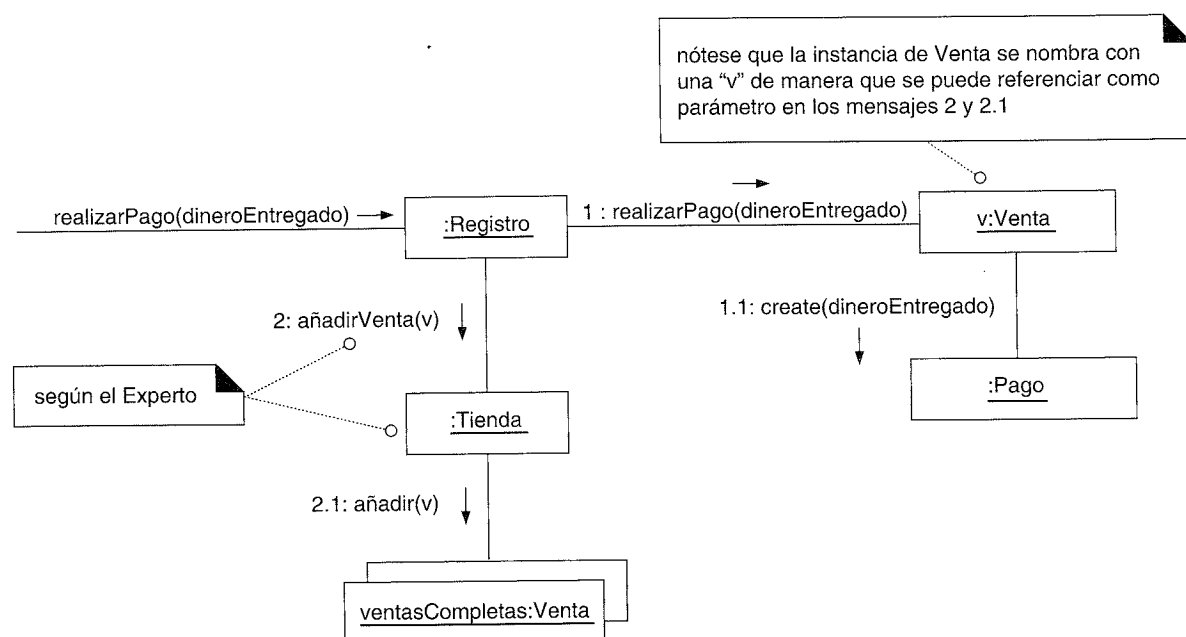


Figura 17.15. Registro de una venta completada.

Cálculo de la devolución

El caso de uso *Procesar Venta* implica que se imprima el saldo deudor a partir de un pago en un recibo y se muestre por pantalla de algún modo.

Debido al principio de Separación Modelo-Vista, no deberíamos preocuparnos por el modo en el que se visualizará o imprimirá el dinero que hay que devolver, pero es necesario asegurar que se conoce. Nótese que ninguna clase conoce actualmente la devolución, de manera que necesitamos crear un diseño de interacciones de objetos que satisfaga estos requisitos.

Como siempre, se debería tener en cuenta el Experto en Información a menos que sea un problema de controlador o creación (que no lo es), y se debería establecer la responsabilidad:

¿Quién es el responsable de conocer la devolución?

Para calcular la devolución, se requiere el total de la venta y el dinero en efectivo entregado. Por tanto, la *Venta* y el *Pago* son Expertos parciales en la solución del problema.

Si el *Pago* es ante todo responsable de conocer la devolución, necesitará tener visibilidad de la *Venta*, para pedirle su total. Puesto que actualmente no sabe nada de la *Venta*, este enfoque incrementa el acoplamiento global en el diseño —no soportaría el patrón de Bajo Acoplamiento—.

En cambio, si la *Venta* es principalmente responsable de conocer la devolución, necesita tener visibilidad del *Pago*, para solicitarle el dinero en efectivo entregado. Puesto que la *Venta* ya tiene visibilidad del *Pago* —como su creador— este enfoque no incrementa el acoplamiento global y, por tanto, es preferible este diseño.

En consecuencia, el diagrama de interacción de la Figura 17.16 proporciona una solución para conocer la cantidad a devolver.

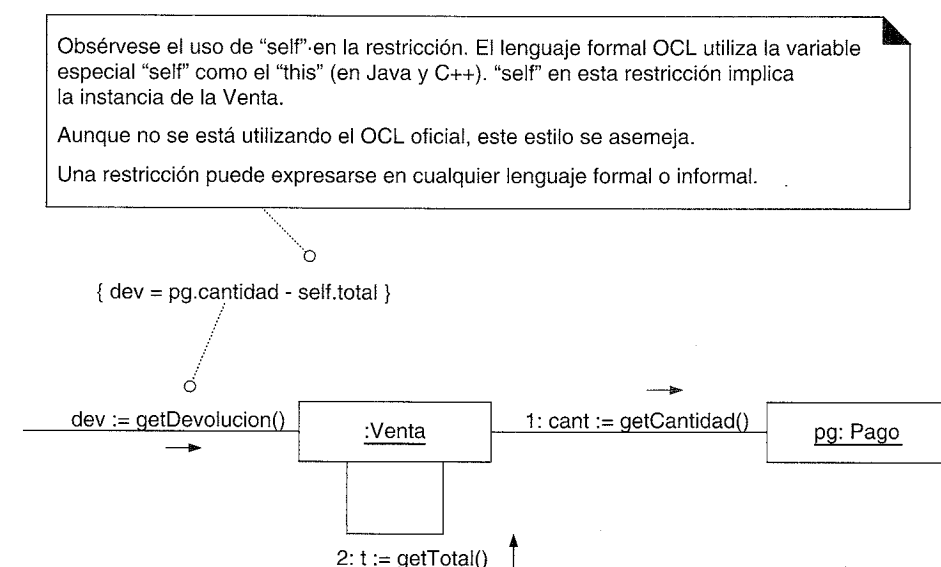


Figura 17.16. Diagrama de interacción Venta--getDevolucion.

17.8. Diseño de objetos: ponerEnMarcha

¿Cuándo crear el diseño de ponerEnMarcha?

La mayoría, si no todos, los sistemas tienen el caso de uso *ponerEnMarcha*, y alguna operación del sistema inicial relacionada con el comienzo de la aplicación. Aunque esta operación del sistema *ponerEnMarcha* es la primera que se va a ejecutar, posponga el desarrollo de un diagrama de interacción para ella hasta que se hayan tenido en cuenta todas las otras operaciones del sistema. Esto asegura que se haya descubierto la información relativa a las actividades de inicialización que se requieren para soportar los diagramas de interacción de operaciones del sistema posteriores.

Haga el diseño de la inicialización en último lugar.

Cómo comienzan las aplicaciones

La operación *ponerEnMarcha* representa de manera abstracta la fase de inicialización de la ejecución cuando se lanza una aplicación. Para entender cómo diseñar un diagrama de interacción para esta operación, es útil entender el contexto en el que puede ocurrir la inicialización. El modo en el que una aplicación comienza y se inicializa depende del lenguaje de programación y del sistema operativo.

En todos los casos, un estilo de diseño común es crear en último término un **objeto del dominio inicial**, que es el primer objeto software del “dominio” que se crea.

Una nota sobre la terminología: como se verá, las aplicaciones se organizan en capas lógicas que separan los aspectos más importantes de la aplicación. Esto comprende la capa de UI (para las cuestiones de UI) y una capa del “dominio” (para cuestiones de la lógica del dominio). La capa del dominio del Modelo de Diseño está formada por las clases software cuyos nombres están inspirados en el vocabulario del dominio, y que contienen la lógica de la aplicación. Prácticamente todos los objetos del diseño que hemos considerado, como *Venta* y *Registro*, son objetos del dominio en la capa del dominio del Modelo de Diseño.

El objeto de dominio inicial, una vez creado, es el responsable de la creación de los objetos del dominio que son sus “hijos”. Por ejemplo, si se elige una *Tienda* como el objeto del dominio inicial, podría ser el responsable de la creación de un objeto *Registro*.

El lugar donde se crea este objeto del dominio inicial depende de la tecnología de objetos escogida. Por ejemplo, en una aplicación Java, podría crearlo el método *main*, o delegar el trabajo al objeto *factoría* que lo crea.

```
public class Main
{
    public static void main(String [] args)
    {
        //La Tienda es el objeto del dominio inicial.
        //La Tienda crea algún otro objeto del dominio.
```

```
Tienda tienda = new Tienda();
Registro registro = tienda.getRegistro();
JFrameProcesarVenta frame = new JFrameProcesarVenta(registro);
...
}
```

Interpretación de la operación del sistema ponerEnMarcha

La discusión anterior ilustra que la operación del sistema *ponerEnMarcha* es una abstracción independiente del lenguaje. Durante el diseño, existen variaciones en cuanto al lugar de creación del objeto inicial, y si controla o no el proceso. El objeto del dominio inicial no suele tomar el control si se trata de una GUI; en otro caso, lo hace con frecuencia.

Los diagramas de operación para la operación *ponerEnMarcha* representan lo que ocurre cuando se crea el objeto inicial del dominio del problema, y opcionalmente lo que sucede si toma el control. No incluyen ninguna actividad anterior o siguiente de los objetos en la capa de GUI, si existe alguno.

Por tanto, la operación *ponerEnMarcha* puede reinterpretarse como:

1. En un diagrama de interacción, envíe un mensaje *create()* para crear el objeto de dominio inicial.
2. (opcional) Si el objeto inicial toma el control del proceso, en un segundo diagrama de interacción, envíe el mensaje *ejecutar* (o algo equivalente) al objeto inicial.

La operación PonerEnMarcha de la aplicación del PDV

La operación del sistema *ponerEnMarcha* tiene lugar cuando un responsable de la tienda enciende el sistema de PDV y se carga el software. Asuma que el objeto del dominio inicial *no* es responsable de controlar el proceso; el control permanecerá en la capa de UI (como un *JFrame* de Java) después de que se cree el objeto del dominio inicial. Por tanto, el diagrama de interacción para la operación *ponerEnMarcha* podría reinterpretarse únicamente como el envío del mensaje *create()* para crear el objeto inicial.

Elección del objeto del dominio inicial

¿Cuál debería ser la clase del objeto del dominio inicial?

Elija como objeto del dominio inicial la clase de la raíz de la jerarquía de agregación o contención, o cercana a ella. Esto podría ser un controlador de fachada, como un *Registro*, o algún otro objeto que se considera que contiene todos o la mayoría de los objetos, como una *Tienda*.

Las consideraciones de Alta Cohesión y Bajo Acoplamiento podrían influir en la elección entre las alternativas. En esta aplicación, se elige la *Tienda* como el objeto inicial.

Objetos persistentes: EspecificacionDelProducto

Las instancias de *EspecificacionDelProducto* residirán en un medio de almacenamiento persistente, como una base de datos relacional u objetual. Durante la operación *ponerEnMarcha*, si sólo hay unos pocos de estos objetos, se podrían cargar todos en la memoria principal del ordenador. Sin embargo, si hay muchos, cargarlos todos consumiría demasiada memoria o tiempo. Alternativamente —y más probable— se cargarán en memoria bajo demanda las instancias individuales cuando se requieran.

El diseño de la manera de cargar dinámicamente bajo demanda los objetos desde una base de datos a memoria es sencilla si se utiliza una base de datos objetual, pero difícil para una base de datos relacional. Este problema se pospone por ahora y se asume que todas las instancias de *EspecificacionDelProducto* pueden ser creadas en memoria “mágicamente” por el objeto *CatalogoDeProductos*.

El Capítulo 34 estudia el problema de los objetos persistentes y el modo de cargarlos en memoria.

Diseño de Tienda--create()

Las tareas de creación e inicialización se derivan a partir de las necesidades del trabajo de diseño anterior, como el diseño de la gestión de *introducirArticulo*, etcétera. Reflexionando sobre los diseños de interacciones previos, se puede identificar el siguiente trabajo de inicialización:

- Se necesita crear una *Tienda*, *Registro*, *CatalogoDeProductos* y objetos *EspecificacionDelProducto*.
- Se necesitan asociar los objetos *EspecificacionDelProducto* con el *CatalogoDeProductos*.
- Se necesita asociar la *Tienda* con el *CatalogoDeProductos*.
- Se necesita asociar la *Tienda* con el *Registro*.
- Se necesita asociar el *Registro* con el *CatalogoDeProductos*.

La Figura 17.17 muestra un diseño. Se escogió la *Tienda* para crear el *CatalogoDeProductos* y el *Registro* según el patrón Creador. Del mismo modo, se eligió el *CatalogoDeProductos* para crear los objetos *EspecificacionDelProducto*. Recuerde que este enfoque de creación de las especificaciones es temporal. En el diseño final, se materializará desde una base de datos, cuando sea necesario.

Notación UML: Obsérvese que la creación de todas las instancias de *EspecificacionDelProducto* y su inclusión en un contenedor tienen lugar en una sección de repetición, que se indica con el “*” a continuación de los números de secuencia.

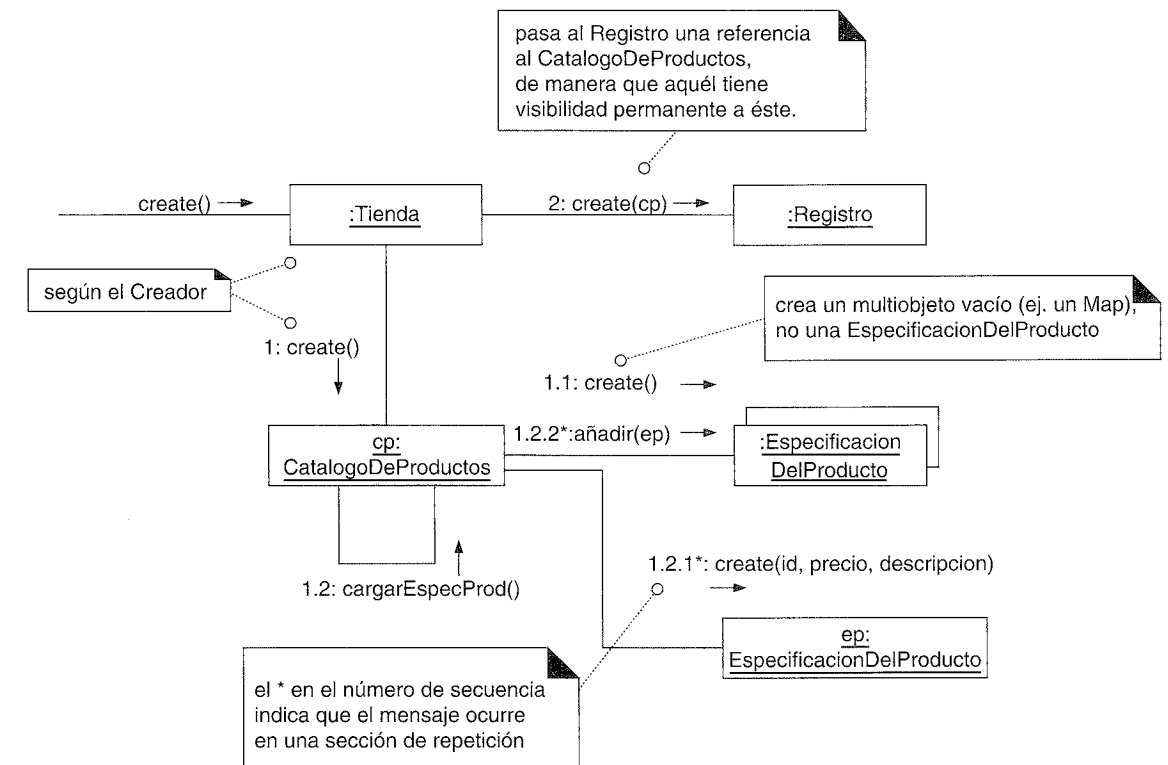


Figura 17.17. Creación del objeto del dominio inicial y los objetos siguientes.

Una desviación interesante entre el modelado del mundo real y el diseño se ilustra en el hecho de que el objeto software *Tienda* sólo crea un objeto *Registro*. Una tienda real podría albergar *muchos* registros o terminales de PDV reales. Sin embargo, estamos considerando un diseño software, no la vida real. En nuestros requisitos actuales, nuestra *Tienda* software sólo necesita crear una única instancia de un *Registro* software.

La multiplicidad entre las clases de objetos del Modelo del Dominio y el Modelo del Diseño podría no ser la misma.

17.9. Conexión de la capa de UI con la capa del dominio

Como se ha discutido brevemente, las aplicaciones se organizan en capas lógicas que separan los aspectos más importantes de la aplicación, como la capa de UI (para las cuestiones de la UI) y una capa de “dominio” (para las cuestiones de la lógica del dominio).

Entre los diseños típicos según los cuales los objetos de la capa del dominio son visibles a los objetos de la capa de la UI encontramos los siguientes:

- Una rutina de inicialización (por ejemplo, un método *main* en Java) crea tanto un objeto de la UI como un objeto del dominio, y pasa el objeto del dominio a la UI.

- Un objeto de la UI recupera el objeto del dominio de una fuente bien conocida, como un objeto factoría que es responsable de la creación de los objetos del dominio.

La muestra de código que se presentó anteriormente es un ejemplo del primer enfoque:

```
public class Main
{
    public static void main (String[] args)
    {
        Tienda tienda = new Tienda();
        Registro registro = tienda.getRegistro();
        JFrameProcesarVenta frame = new JFrameProcesarVenta(registro);
        ...
    }
}
```

Una vez que el objeto de la UI está conectado a la instancia del *Registro* (el controlador de fachada en este diseño), puede reenviarle mensajes de eventos del sistema, como los mensajes *introducirArticulo* y *finalizarVenta* (ver Figura 17.18).

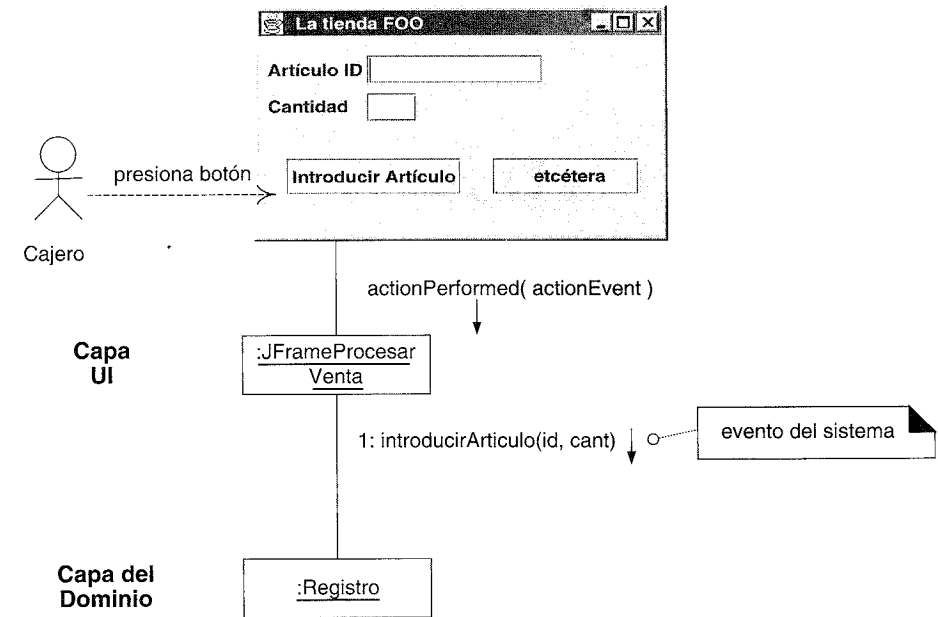


Figura 17.18. Conexión de la capas de UI y del dominio.

En el caso del mensaje *introducirArticulo*, la ventana necesita mostrar la suma parcial después de cada entrada. Hay varias soluciones de diseño:

- Añadir el método *getTotal* al *Registro*. La UI envía el mensaje *getTotal* al *Registro*, quien lo reenvía a la *Venta*. Esto tiene la posible ventaja de que mantiene bajo acoplamiento entre la UI y el modelo del dominio —la UI sólo conoce al objeto *Re-*

gistro—. Pero comienza a expandir el interfaz del objeto *Registro*, haciéndolo menos cohesivo.

- Una UI solicita una referencia al objeto *Venta* actual, y entonces cuando necesita el total (o cualquier otra información relacionada con la venta), envía el mensaje directamente a la *Venta*. Este diseño incrementa el acoplamiento entre la UI y el modelo del dominio. Sin embargo, como se estudió en la discusión del patrón GRASP Bajo Acoplamiento, un mayor acoplamiento por sí mismo no es un problema; más bien, en especial constituye un problema el acoplamiento entre cosas inestables. Asuma que decidimos que la *Venta* es un objeto estable que será una parte integral del diseño —lo que es muy razonable—. Entonces, el acoplamiento con la *Venta* no es un problema.

Como se ilustra en la Figura 17.19, este diseño sigue el segundo enfoque.

Nótese que en estos diagramas la ventana Java (*JFrameProcesarVenta*), que forma parte de la capa de UI, no es responsable de manejar la lógica de la aplicación. La ventana remite las solicitudes de trabajo (las operaciones del sistema) a la capa del dominio, por medio del *Registro*. Esto nos lleva al siguiente principio de diseño:

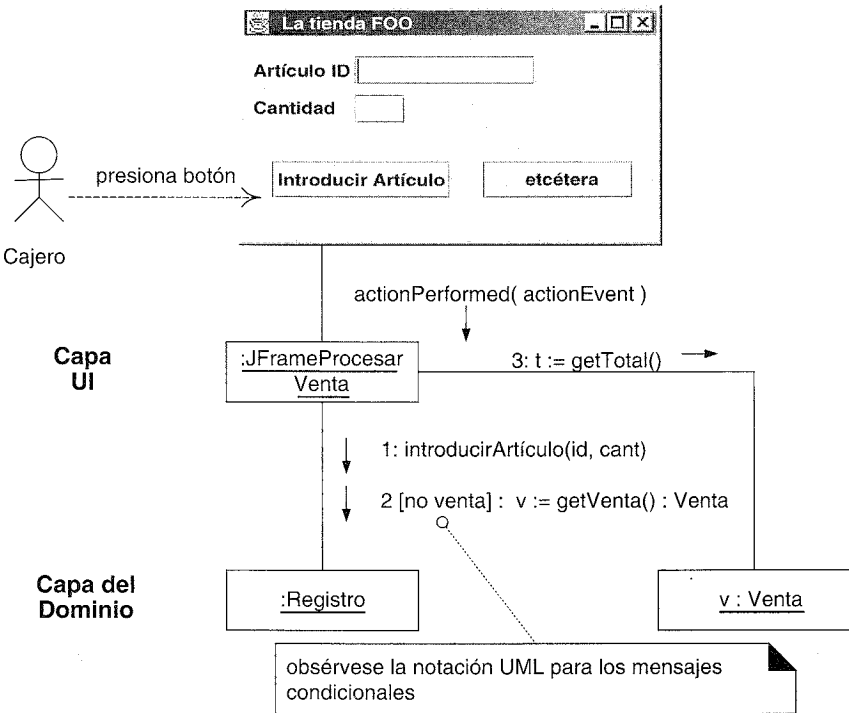


Figura 17.19. Conexión de las capas de la UI y del dominio.

Responsabilidades de la capa del dominio y de interfaz

La capa de UI no debería tener ninguna responsabilidad de la lógica del dominio. Sólo debería ser responsable de las tareas de la interfaz de usuario, como actualizar los elementos gráficos. La capa de UI debería remitir las solicitudes de las tareas orientadas al dominio a la capa del dominio, que es la responsable de manejarlas.

17.10. Realizaciones de casos de uso en el UP

Las realizaciones de los casos de uso forman parte del Modelo de Diseño del UP. Este capítulo ha resaltado la elaboración de los diagramas de interacción, pero es común y se recomienda que se elaboren los diagramas de clases en paralelo. Los diagramas de clases se estudiarán en el Capítulo 19.

Tabla 17.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Fases

Inicio: El Modelo de Diseño y las realizaciones de los casos de uso normalmente no comenzarán hasta la elaboración porque comprende decisiones de diseño detalladas que son prematuras durante la fase de inicio.

Elaboración: Durante esta fase, podrían crearse las realizaciones de los casos de uso para los escenarios más significativos desde el punto de vista de la arquitectura o de más riesgo del diseño. Sin embargo, no se harán los diagramas de UML para cada escenario, y no necesariamente con todo detalle o de grano fino. La idea es realizar los diagramas de interacción para las realizaciones de los casos de uso claves que se benefician de algún estudio anticipado y exploración de alternativas, centrándose en las decisiones de diseño más importantes.

Construcción: Se crean las realizaciones de los casos de uso para el resto de problemas de diseño.

En el UP, el trabajo de la realización de los casos de uso es una actividad de diseño. La Figura 17.21 ofrece sugerencias sobre el momento y el lugar para llevar a cabo este trabajo.

17.11. Resumen

Lo esencial de un diseño de objetos lo constituyen el diseño de las interacciones de objetos y la asignación de responsabilidades. Las decisiones que se tomen pueden influir

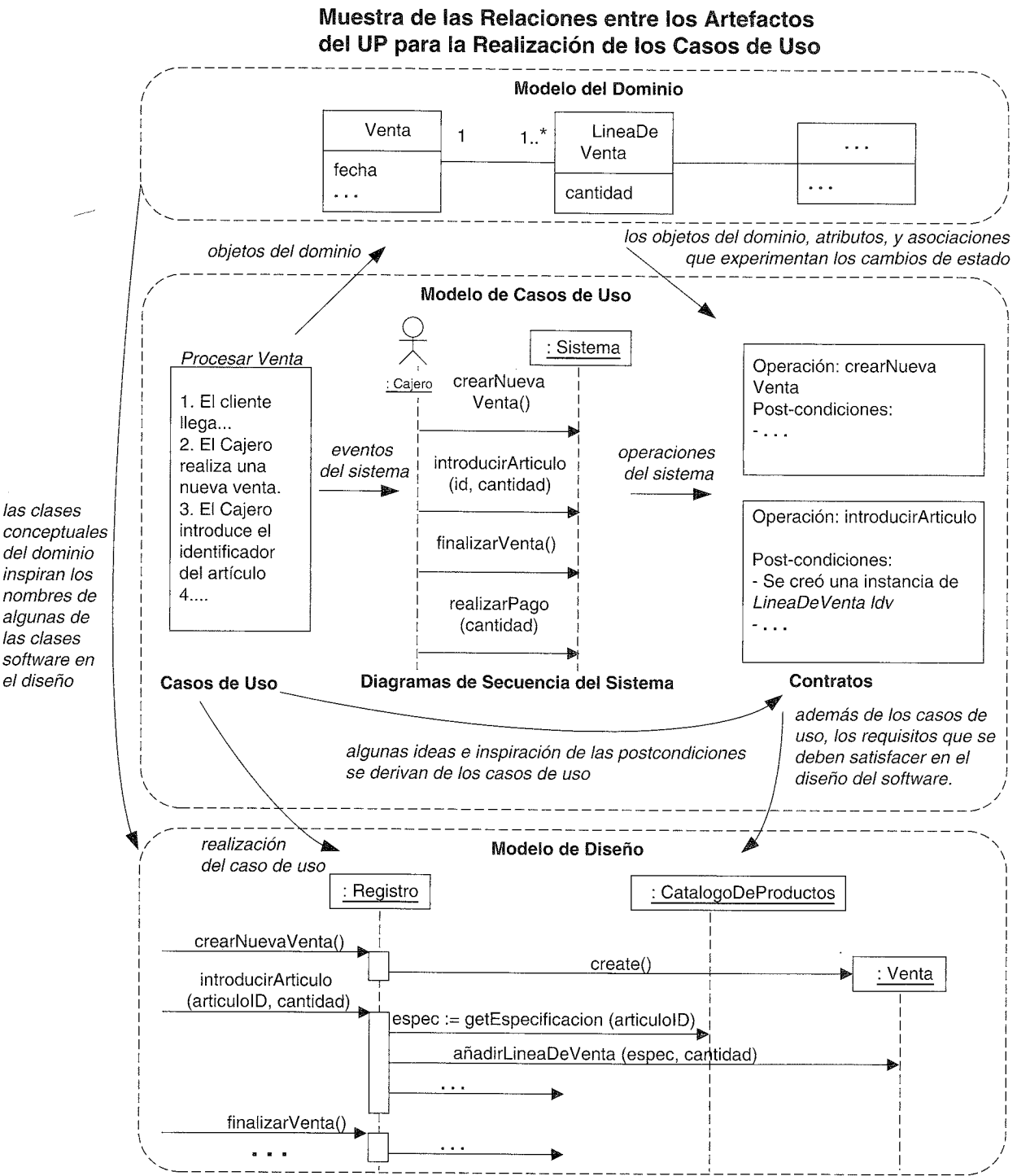


Figura 17.20. Muestra de la influencia entre los artefactos UP.

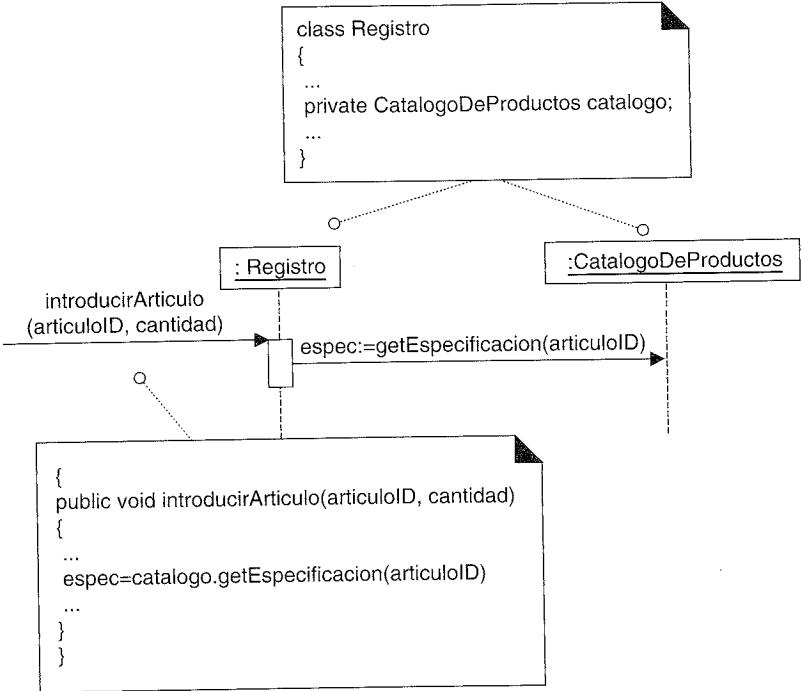


Figura 18.1. Se requiere visibilidad desde el Registro al CatalogoDeProductos¹.

UML tiene una notación especial para representar la visibilidad; este capítulo presenta varios tipos de visibilidad y su descripción.

18.2. Visibilidad

En su uso habitual, por **visibilidad** se entiende la capacidad de un objeto de “ver” o tener una referencia a otro objeto. De manera más general, está relacionado con el tema del alcance: ¿se encuentra un recurso (tal como una instancia) al alcance de otro? Hay cuatro formas comunes de alcanzar la visibilidad desde un objeto A a un objeto B:

- **Visibilidad de atributo:** B es un atributo de A.
- **Visibilidad de parámetro:** B es un parámetro de un método de A.
- **Visibilidad local:** B es un objeto local (no un parámetro) en un método de A.
- **Visibilidad global:** B es de algún modo visible globalmente.

El motivo para tener en cuenta la visibilidad es la siguiente:

Para que un objeto A envíe un mensaje a un objeto B, B debe ser visible a A.

Por ejemplo, para crear un diagrama de interacción en el que se envía un mensaje desde una instancia de *Registro* a una instancia de *CatalogoDeProductos*, el *Registro*

¹ En éste y en los siguientes ejemplos de código, podrían hacerse simplificaciones del lenguaje en aras de la brevedad y claridad.

debe tener visibilidad del *CatalogoDeProductos*. Una solución típica para la visibilidad es mantener una referencia a una instancia del *CatalogoDeProductos* como atributo del *Registro*.

Visibilidad de atributo

La **visibilidad de atributo** desde A a B existe cuando B es un atributo de A. Es una visibilidad relativamente permanente porque persiste mientras existan A y B. Ésta es una forma de visibilidad muy común en los sistemas orientados a objetos.

Para ilustrarlo, sea una definición de clase en Java para el *Registro*, una instancia de *Registro* tendría visibilidad de atributo a un *CatalogoDeProducto*, puesto que es un atributo (variable de instancia Java) del *Registro*.

```
public class Registro
{
  ...
  private CatalogoDeProductos catalogo;
  ...
}
```

Se requiere esta visibilidad porque en el diagrama de *introducirArticulo* que se muestra en la Figura 18.2, un *Registro* necesita enviar el mensaje *getEspecificacion* a un *CatalogoDeProductos*:

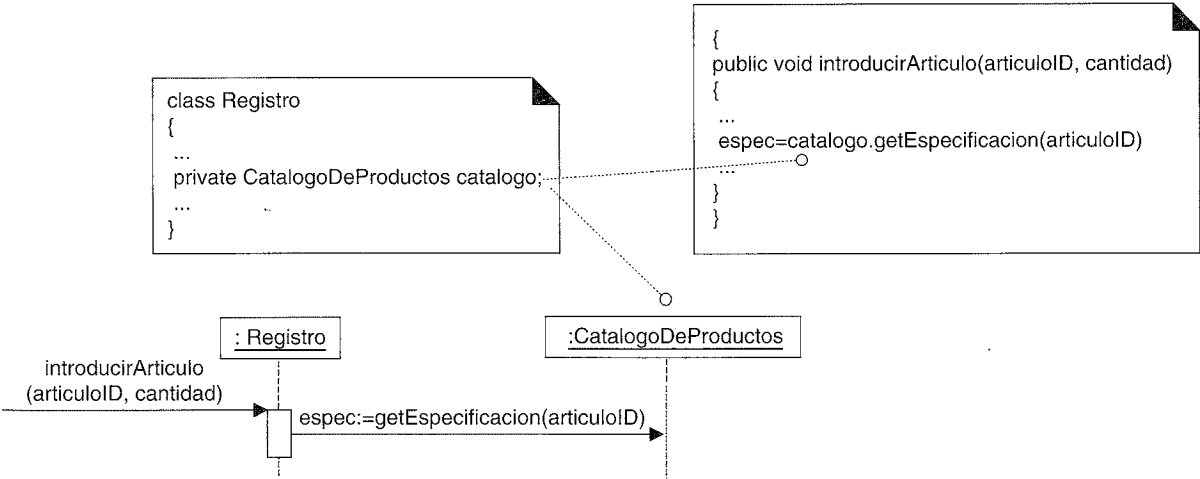


Figura 18.2. Visibilidad de atributo.

Visibilidad de parámetro

La **visibilidad de parámetro** desde A a B existe cuando B se pasa como parámetro a un método de A. Es una visibilidad relativamente temporal porque persiste sólo en el alcance del método. Después de la visibilidad de atributo, es la segunda forma más común de visibilidad en los sistemas orientados a objetos.

Para ilustrarlo, cuando se envía el mensaje *crearLineaDeVenta* a la instancia de *Venta*, se pasa como parámetro una instancia de *EspecificacionDelProducto*. En el alcance del método *crearLineaDeVenta*, la *Venta* tiene visibilidad de parámetro de una *EspecificacionDelProducto* (ver Figura 18.3).

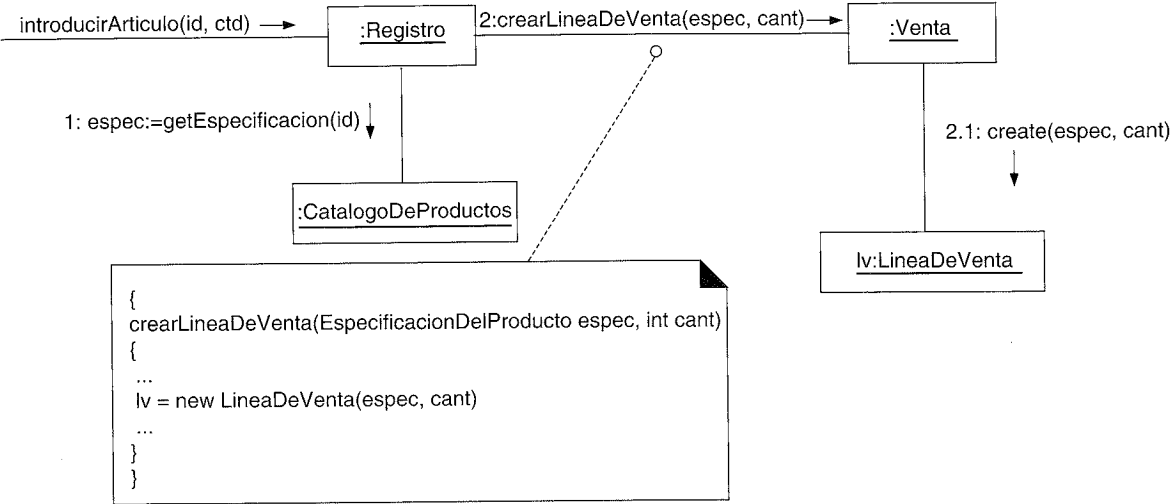


Figura 18.3. Visibilidad de parámetro.

Es habitual transformar la visibilidad de parámetro en visibilidad de atributo. Por ejemplo, cuando la *Venta* crea una nueva *LineaDeVenta*, le pasa en el método de inicialización una *EspecificacionDelProducto* (sería su **constructor** en Java y C++). En el método de inicialización, se asigna el parámetro a un atributo; por tanto, se establece visibilidad de atributo (ver Figura 18.4).

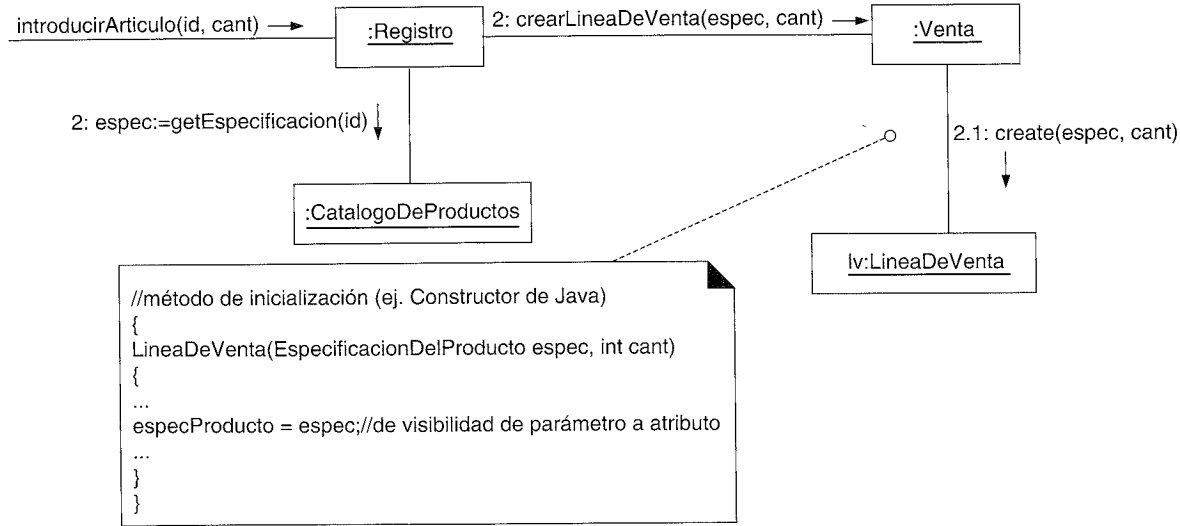


Figura 18.4. De visibilidad de parámetro a visibilidad de atributo.

Visibilidad local

La **visibilidad local** desde A a B existe cuando B se declara como un objeto local en un método de A. Es una visibilidad relativamente temporal porque sólo persiste en el alcance del método. Después de la visibilidad de parámetro, es la tercera forma de visibilidad más común en los sistemas orientados a objetos.

Dos medios comunes de alcanzar la visibilidad local son:

- Crear una nueva instancia local y asignarla a una variable local.
- Asignar a una variable local el objeto de retorno de la invocación a un método.

Como con la visibilidad de parámetro, es habitual transformar la visibilidad declarada localmente en visibilidad de atributo.

Un ejemplo de la segunda variación (asignación a una variable local del objeto de retorno) se puede encontrar en el método *introducirArticulo* de la clase *Registro* (Figura 18.5).

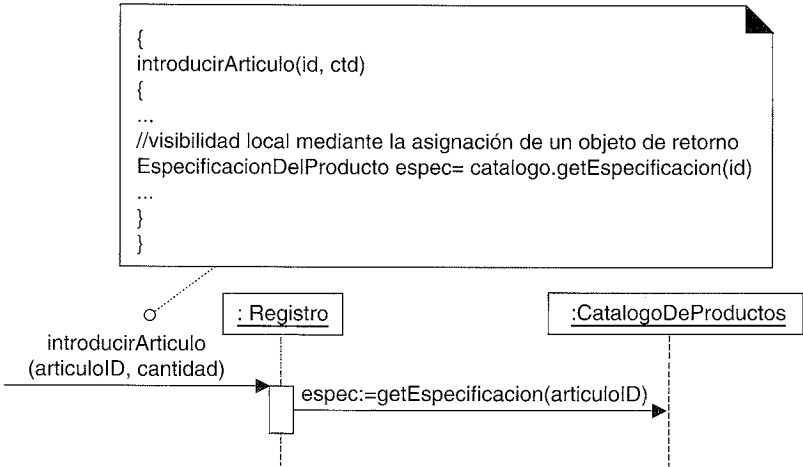


Figura 18.5. Visibilidad local.

Una versión sutil de la segunda variación es cuando el método no declara explícitamente una variable, sino que implícitamente existe una como resultado de un objeto de retorno de una invocación a un método. Por ejemplo:

```
//hay una visibilidad local implícita al objeto foo
//resultado de la llamada a getFoo
unObjeto.getFoo().hacerBar();
```

Visibilidad global

La **visibilidad global** de A a B existe cuando B es global a A. Es una visibilidad relativamente permanente porque persiste mientras existan A y B. Es la forma menos común de visibilidad en los sistemas orientados a objetos.

Un medio de conseguir visibilidad global es asignar una instancia a una variable global, lo que es posible en algunos lenguajes, como C++, pero no en otros, como Java.

El método que se prefiere para conseguir la visibilidad global es utilizar el patrón Singleton [GHJV95], que se presentará en un capítulo posterior.

18.3. Representación de la visibilidad en UML

UML incluye notación para representar el tipo de visibilidad en un diagrama de colaboración (ver Figura 18.6). Estos adornos son opcionales y normalmente no se exigen; son útiles cuando se necesita alguna aclaración.

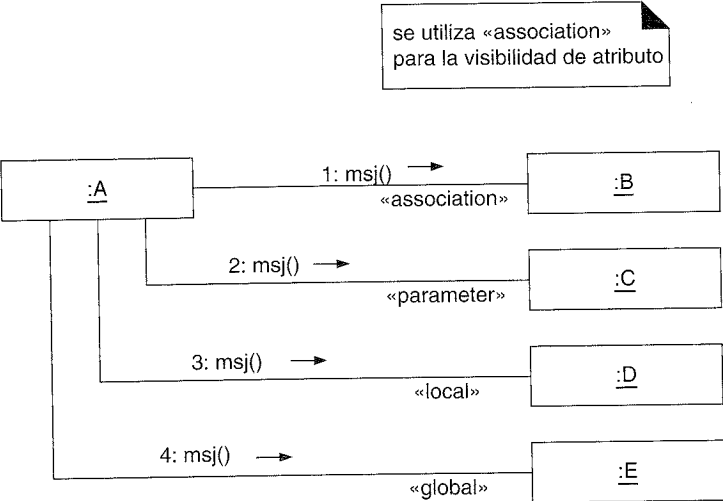


Figura 18.6. Implementación de los estereotipos para la visibilidad.

MODELO DE DISEÑO: CREACIÓN DE LOS DIAGRAMAS DE CLASES DE DISEÑO

Iterar es humano, ser recursico, divino.

Anónimo

Objetivos

- Crear los diagramas de clases de diseño (DCD).
- Identificar las clases, métodos y asociaciones que se van a mostrar en un DCD.

Introducción

Una vez terminados los diagramas de interacción para las realizaciones de los casos de uso de la iteración actual de la aplicación de PDV NuevaEra, es posible identificar la especificación de las clases software (e interfaces) que participan en la solución software, y añadirles detalles de diseño, como los métodos.

UML proporciona la notación para representar los detalles de diseño en los diagramas de clase; en este capítulo, vamos a estudiarla y a crear los DCD.

19.1. Cuándo crear los DCD

Aunque esta presentación de los DCD viene después de la creación de los diagramas de interacción, en la práctica normalmente se crean en paralelo. Al comienzo del diseño se podrían esbozar muchas clases, nombres de métodos y relaciones aplicando los patrones para asignar responsabilidades, antes de la elaboración de los diagramas de interacción. Es posible y deseable elaborar algo de los diagramas de interacción, actualizar entonces los DCD, después extender los diagramas de interacción algo más, y así sucesivamente.

Estos diagramas de clases podrían utilizarse como una notación más gráfica alternativa a las tarjetas CRC para recoger las responsabilidades y colaboraciones.

19.2. Ejemplo de DCD

El DCD de la Figura 19.1 ilustra una definición software parcial de las clases *Registro* y *Venta*.

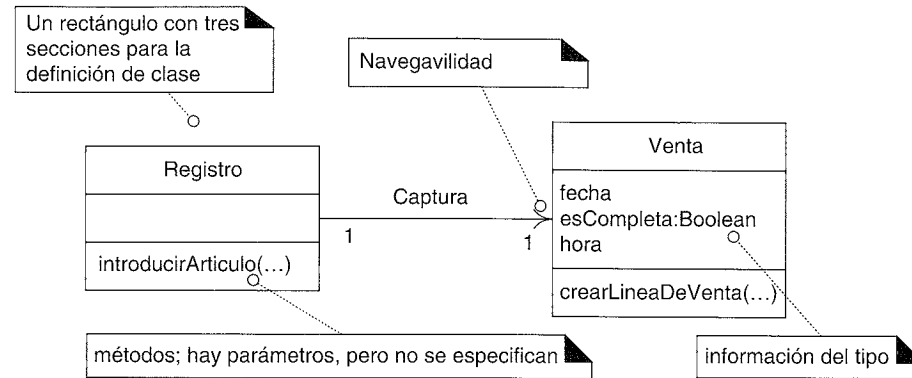


Figura 19.1. Ejemplo de diagrama de clases de diseño.

Además de las asociaciones y atributos básicos, el diagrama se amplía para representar, por ejemplo, los métodos de cada clase, información del tipo de los atributos, visibilidad de los atributos y navegación entre los objetos.

19.3. Terminología del DCD y el UP

Un **diagrama de clases de diseño** (DCD) representa las especificaciones de las clases e interfaces software (por ejemplo, las interfaces de Java) en una aplicación. Entre la información general encontramos:

- Clases, asociaciones y atributos.
- Interfaces, con sus operaciones y constantes.
- Métodos.
- Información acerca del tipo de los atributos.
- Navegabilidad.
- Dependencias.

A diferencia de las clases conceptuales del Modelo del Dominio, las clases de diseño de los DCD muestran las definiciones de las clases software en lugar de los conceptos del mundo real.

El UP no define de manera específica ningún artefacto denominado “diagrama de clases de diseño”. El UP define el Modelo de Diseño, que contiene varios tipos de diagramas, que incluye los diagramas de interacción, de paquetes, y los de clases. Los diagramas de clases del Modelo de Diseño del UP contienen “clases de diseño” en términos del UP. De ahí que sea común hablar de “diagramas de clases de diseño”, que es más corto que, e implica, “diagramas de clases en el Modelo de Diseño”.

19.4. Clases del Modelo de Dominio vs. clases del Modelo de Diseño

Seamos reiterativos, en el Modelo de Dominio del UP, una *Venta* no representa una definición software, sino que se trata de una abstracción de un concepto del mundo real acerca del cual estamos interesados en realizar una declaración. En cambio, los DCD expresan —para la aplicación software— la definición de las clases como componentes software. En estos diagramas, una *Venta* representa una clase software (ver Figura 19.2).

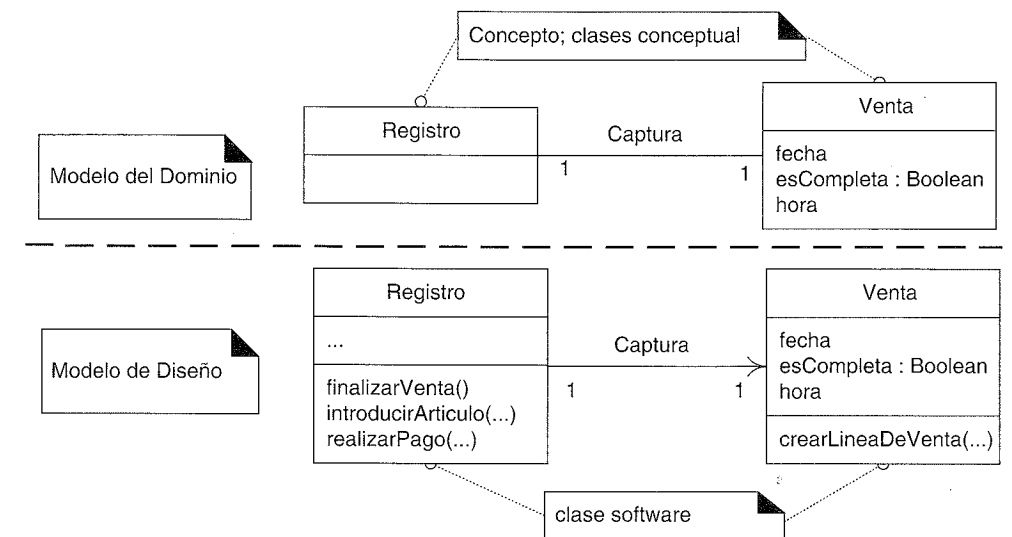


Figura 19.2. Clases del Modelo del Dominio vs. clases del Modelo de Diseño.

19.5. Creación de un DCD del PDV NuevaEra

Identificación y representación de las clases software

El primer paso en la creación de los DCD como parte del modelo de la solución es identificar aquellas clases que participan en la solución software. Se pueden encontrar examinando todos los diagramas de interacción y listando las clases que se mencionan.

Para la aplicación del PDV son:

Registro	Venta
CatalogoDeProductos	EspecificacionDelProducto
Tienda	LineaDeVenta
Pago	

El siguiente paso es dibujar un diagrama de clases para estas clases e incluir los atributos que se identificaron previamente en el Modelo del Dominio que también se utilizan en el diseño (ver Figura 19.3).

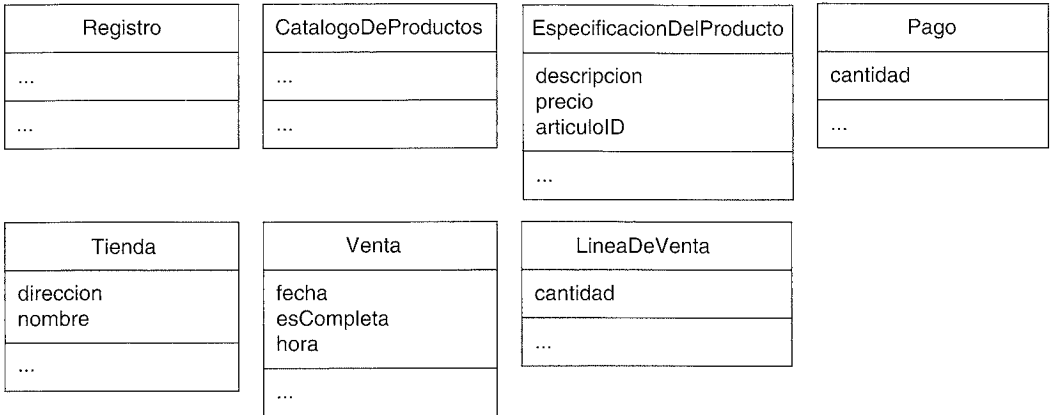


Figura 19.3. Clases software de la aplicación.

Nótese que algunos de los conceptos del Modelo del Dominio, como *Cajero*, no están presentes en el diseño. No es necesario —para la iteración actual— representarlos en el software. Sin embargo, en iteraciones posteriores, cuando se aborden nuevos requisitos y casos de uso, podrían formar parte del diseño. Por ejemplo, cuando se implementen los requisitos de seguridad y de inicio de sesión, es probable que sea relevante una clase software denominada *Cajero*.

Añadir los nombres de los métodos

Se pueden identificar los nombres de los métodos analizando los diagramas de interacción. Por ejemplo, si se envía el mensaje *crearLineaDeVenta* a una instancia de la clase *Venta*, entonces la clase *Venta* debe definir un método *crearLineaDeVenta* (ver Figura 19.4).

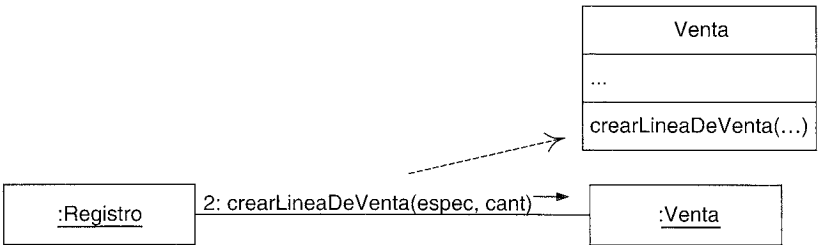


Figura 19.4. Nombres de los métodos a partir de los diagramas de interacción.

En general, el conjunto de todos los mensajes enviados a una clase X a lo largo de todos los diagramas de interacción indican la mayoría de los métodos que debe definir la clase X.

La inspección de todos los diagramas de interacción de la aplicación del PDV da lugar a la asignación de métodos que se muestra en la Figura 19.5.

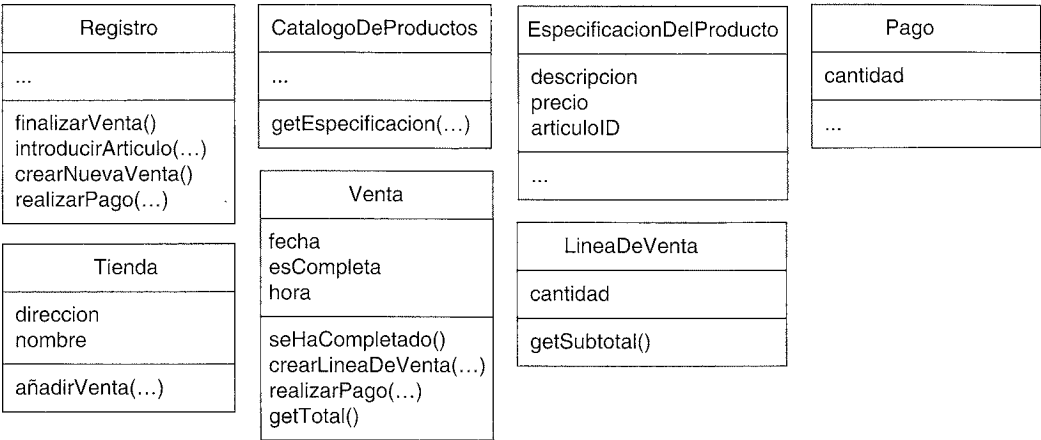


Figura 19.5. Métodos de la aplicación.

Cuestiones acerca de los nombres de los métodos

Se deben tener en cuenta las siguientes cuestiones especiales en relación con los nombres de los métodos:

- Interpretación del mensaje *create*.
- Descripción de los métodos de acceso.
- Interpretación de los mensajes a los multiobjetos.
- Sintaxis dependiente del lenguaje.

Nombres de los métodos—create

El mensaje *create* es una forma independiente del lenguaje posible en UML para indicar instanciación e inicialización. Al traducir el diseño a un lenguaje de programación orientado a objetos, se debe expresar en función de sus estilos para la instanciación e inicialización. No existe un método *create* ni en C++, ni en Java, ni en Smalltalk. Por ejemplo, en C++, implica la asignación automática, o asignación de almacenamiento libre con el operador *new*, seguido de una llamada al constructor. En Java, implica la invocación del operador *new*, seguido de una llamada al constructor.

Debido a las múltiples interpretaciones, y también debido a que la inicialización es una actividad muy común, es habitual omitir en el DCD los métodos relacionados con la creación y los constructores.

Nombres de los métodos—métodos de acceso

Los **métodos de acceso** recuperan (método de obtención, *accessor*) o establecen (método de cambio, *mutador*) el valor de los atributos. En algunos lenguajes (como Java) es un estilo común tener un *accessor* y un *mutador* para cada atributo, y declarar todos los

atributos como privados (para imponer la encapsulación de datos). Normalmente, se excluye la descripción de estos métodos en el diagrama de clases debido a que generan mucho ruido; para n atributos, hay $2n$ métodos sin interés. Por ejemplo, no se muestra, aunque está presente, el método *getPrecio* (o *precio*) de la *EspecificacionDelProducto*, porque se trata simplemente de un método de acceso.

Nombres de los métodos—multiobjetos

Un mensaje a un multiobjeto se interpreta como un mensaje al propio objeto contenedor/colección. Por ejemplo, el siguiente mensaje al multiobjeto *buscar* se debe interpretar como un mensaje al objeto contenedor/colección, como a un *Map* en Java, un *map* en C++, o un *Dictionary* en Smalltalk (ver Figura 19.6).

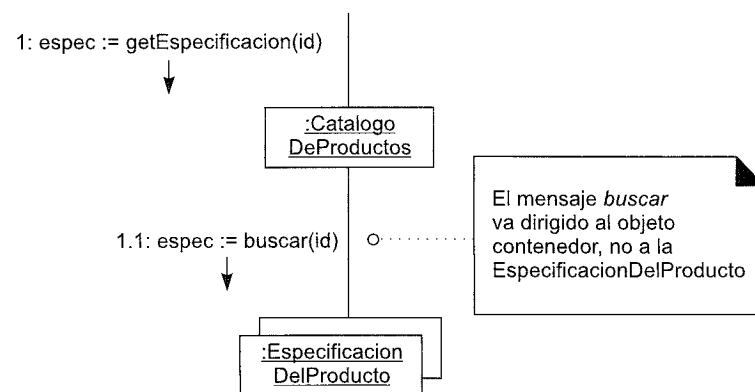


Figura 19.6. Mensaje a un multiobjeto.

Por tanto, el método *buscar* no forma parte de la clase *EspecificacionDelProducto* sino de la interfaz del multiobjeto. En consecuencia, no es correcto añadir el método *buscar* a la clase *EspecificacionDelProducto*.

Normalmente, estas interfaces o clases de contenedores/colecciones (como la interfaz *java.util.Map*) son elementos de las librerías predefinidas, y no es útil mostrar explícitamente estas clases en el DCD, porque añaden ruido, pero poca información nueva.

Nombres de los métodos—sintaxis dependiente del lenguaje

Algunos lenguajes, como Smalltalk, tienen una sintaxis que es muy diferente del formato UML básico de *nombreMetodo(listaParametros)*. Se recomienda que se utilice el formato UML básico, incluso si el lenguaje de implementación que se planea utilizar tiene una sintaxis diferente. Idealmente, la traducción debería tener lugar en el momento de la generación de código, en lugar de durante la creación de los diagramas de clase. Sin embargo, UML permite otra sintaxis para la especificación de los métodos.

Añadir más información sobre los tipos

Opcionalmente, todos los tipos de los atributos, parámetros de los métodos y los valores de retorno se podrían mostrar. La cuestión sobre si se muestra o no esta información se debe considerar en el siguiente contexto:

Se debería crear un DCD teniendo en cuenta los destinatarios.

- Si se está creando en una herramienta CASE con generación automática de código, son necesarios todos los detalles y de modo exhaustivo.
- Si se está creando para que lo lean los desarrolladores de software, los detalles exhaustivos de bajo nivel podrían afectar negativamente por el nivel de ruido.

Por ejemplo, ¿es necesario mostrar todos los parámetros y la información de sus tipos? Depende de lo obvia que sea la información para la audiencia a la que está destinada.

El diagrama de clases de diseño de la Figura 19.7 muestra más información sobre los tipos.

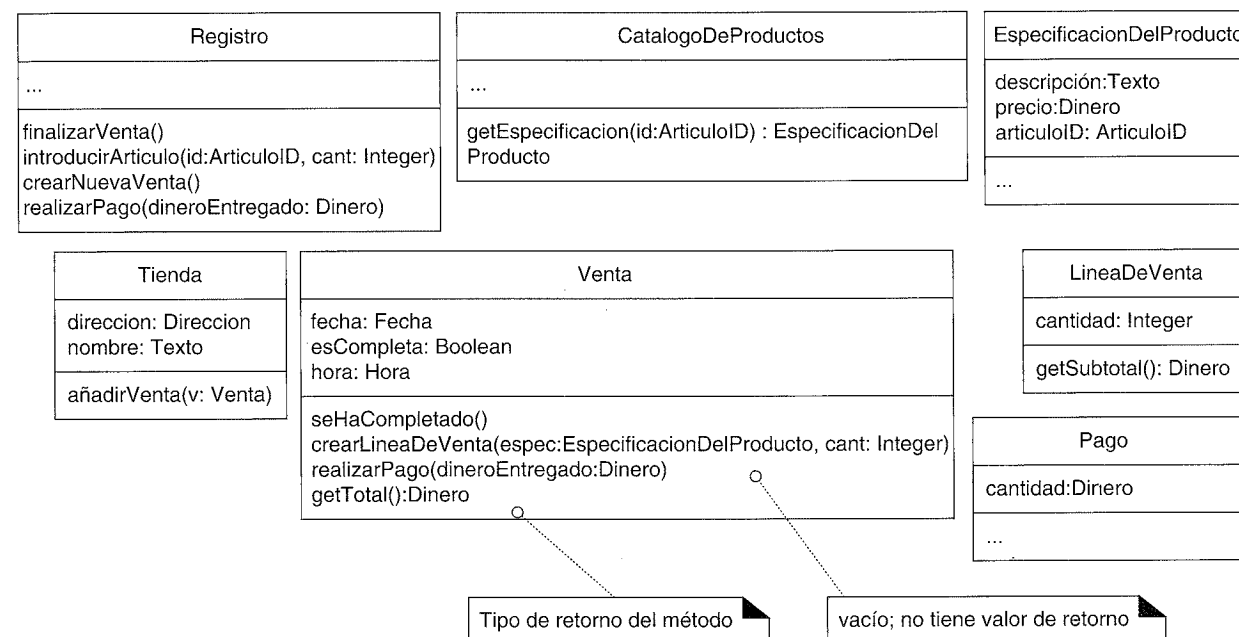


Figura 19.7. Añadir información acerca de los tipos.

Añadir asociaciones y navegabilidad

Cada extremo de asociación se denomina rol, y en los DCD el rol podría decorarse con una flecha de navegabilidad. La **navegabilidad** es una propiedad del rol que indica que es posible navegar unidireccionalmente a través de la asociación desde los objetos de la clase origen a la clase destino. La navegabilidad implica visibilidad —normalmente visibilidad de atributo (ver Figura 19.8)—.

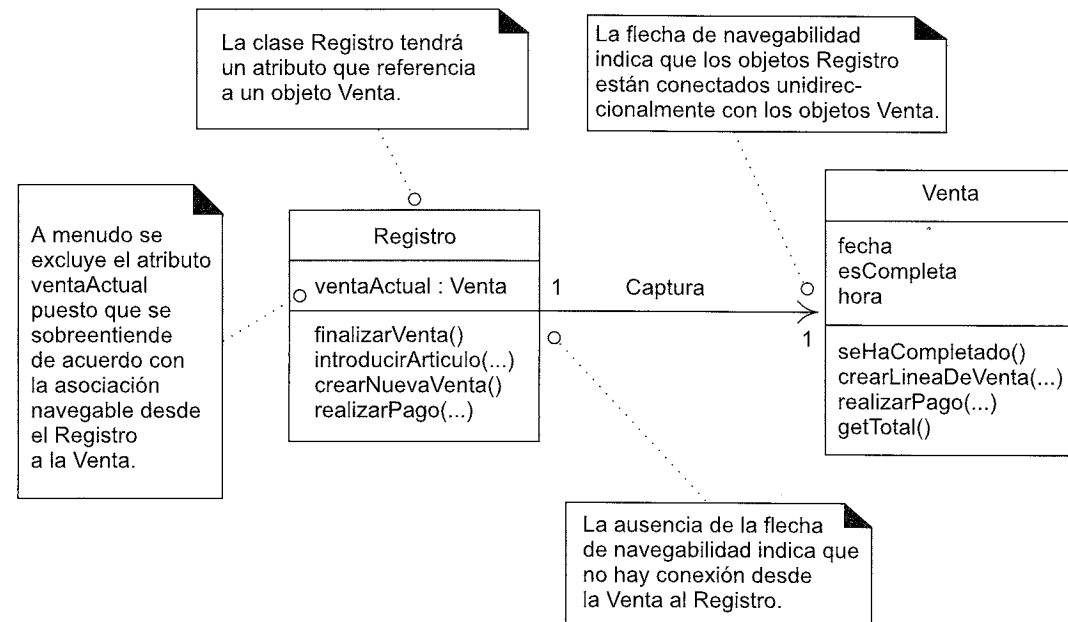


Figura 19.8. Representación de la navegabilidad o visibilidad de atributo.

La interpretación habitual de una asociación con una flecha de visibilidad es la visibilidad de atributo desde la clase origen hasta la clase destino. Durante la implementación en un lenguaje orientado a objetos por lo general se transforma en un atributo en la clase origen que hace referencia a una instancia de la clase destino. Por ejemplo, la clase *Registro* definirá un atributo que referencia a una instancia de *Venta*.

La mayoría, por no decir todas, las asociaciones en los DCD deberían adornarse con las flechas de navegabilidad necesarias.

En un DCD, se eligen las asociaciones de acuerdo a un criterio necesito-conocer orientado al software estricto —¿qué asociaciones se requieren para satisfacer la visibilidad y las necesidades de memoria actuales que se indican en los diagramas de interacción?—. Esto contrasta con las asociaciones en el Modelo del Dominio, que se podrían justificar por la intención de mejorar la comprensión del dominio del problema. Una vez más; vemos que existe una diferencia entre los objetivos del Modelo del Diseño y del Modelo del Dominio: uno es analítico, el otro una descripción de los componentes software.

La visibilidad y las asociaciones requeridas entre las clases se dan a conocer mediante los diagramas de interacción. A continuación, presentamos algunas situaciones comunes que sugieren la necesidad de definir una asociación con un adorno de visibilidad de A a B:

- A envía un mensaje a B.
- A crea una instancia de B.
- A necesita mantener una conexión a B.

Por ejemplo, a partir del diagrama de interacción de la Figura 19.9 que comienza con el mensaje *create* a la *Tienda*, y a partir del contexto más amplio de los otros diagramas

de interacción, se puede distinguir que probablemente la *Tienda* debería tener una conexión permanente con las instancias de *Registro* y el *CatalogoDeProductos* que crea. También es razonable que el *CatalogoDeProductos* necesite una conexión permanente con la colección de objetos *EspecificacionDelProducto* que crea. De hecho, muy a menudo el creador de otro objeto requiere una conexión permanente con él. Por tanto, las conexiones implícitas se presentarán como asociaciones en el diagrama de clases.

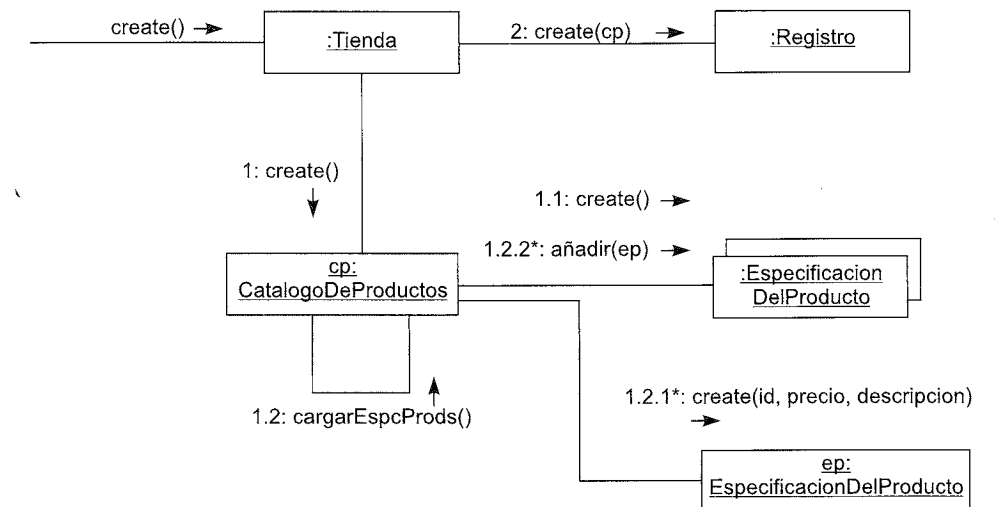


Figura 19.9. La navegabilidad se identifica a partir de los diagramas de interacción.

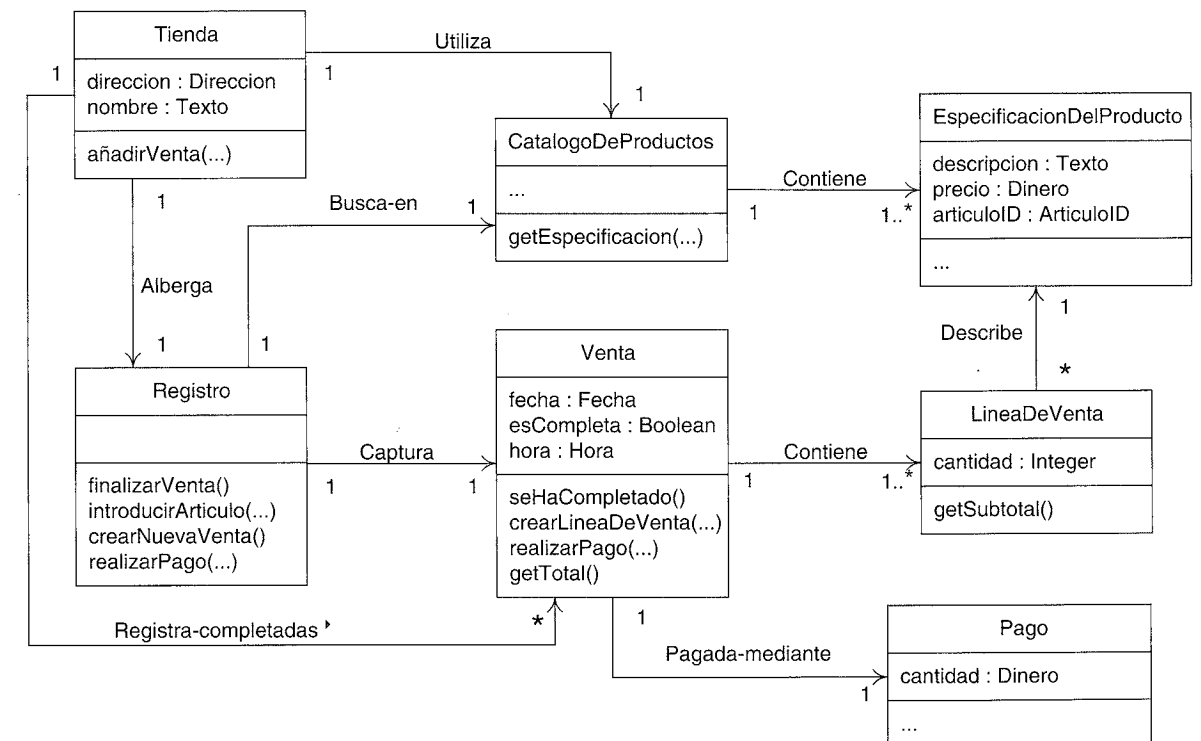


Figura 19.10. Asociaciones con adornos de navegabilidad.

Basado en el criterio anterior para las asociaciones y la navegabilidad, el análisis de todos los diagramas de interacción generados para la aplicación del PDV NuevaEra dará lugar a un diagrama de clases (ver Figura 19.10) con las siguientes asociaciones (se oculta la información exhaustiva sobre los tipos por claridad).

Nótese que éste no es exactamente el mismo conjunto de asociaciones que se generó para el diagrama de clases del Modelo del Dominio. Por ejemplo, en el modelo del dominio no existía la asociación *Busca-en* entre el *Registro* y el *CatalogoDeProductos* —no se pensó que fuera una relación importante y permanente en ese momento—. Pero durante la creación de los diagramas de interacción, se decidió que el objeto software *Registro* debería tener una conexión permanente con el *CatalogoDeProductos* software para buscar los objetos *EspecificacionDelProducto*.

Añadir las relaciones de dependencia

UML incluye una **relación de dependencia** general, que indica que un elemento (de cualquier tipo, como clases, casos de uso, etc.) tiene conocimiento de otro elemento. Se representa mediante una línea de flecha punteada. En los diagramas de clases la relación de dependencia es útil para describir la visibilidad entre clases que no es de atributo; en otras palabras, declaración de visibilidad de parámetro, local o global. En cambio, la vi-

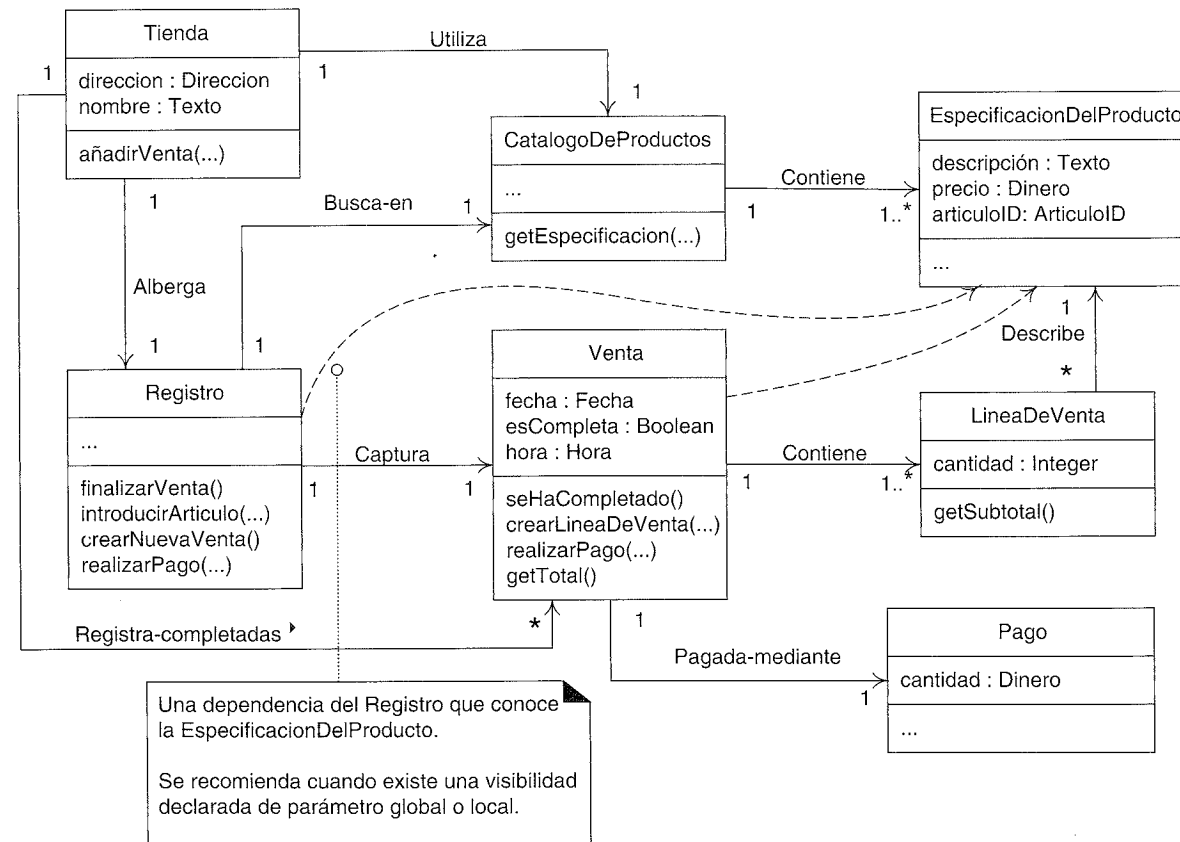


Figura 19.11. Relaciones de dependencia que indican una visibilidad que no es de atributo.

sibilidad de atributo simple se muestra mediante una línea de asociación ordinaria y una flecha de navegabilidad. Por ejemplo, el objeto software *Registro* recibe un objeto de retorno de tipo *EspecificacionDelProducto* a partir del mensaje de especificación que envía al *CatalogoDeProductos*. Por tanto, el *Registro* tiene una visibilidad a corto plazo, declarada localmente, de la *EspecificacionDelProducto*. Y una *Venta* recibe una *EspecificacionDelProducto* como parámetro en el mensaje *crearLineaDeVenta*; tiene visibilidad de parámetro de una especificación.

Estas visibilidades que no son de atributo podrían representarse con la línea de flecha punteada que indica una relación de dependencia (ver Figura 19.11). La curvatura de las líneas de dependencias no tiene ningún significado; se acomoda a la representación gráfica.

19.6. Notación para los detalles de los miembros

UML proporciona una notación variada para describir las características de los miembros de las clases e interfaces, como la visibilidad, valores iniciales, etcétera. En la Figura 19.12 se muestra un ejemplo.

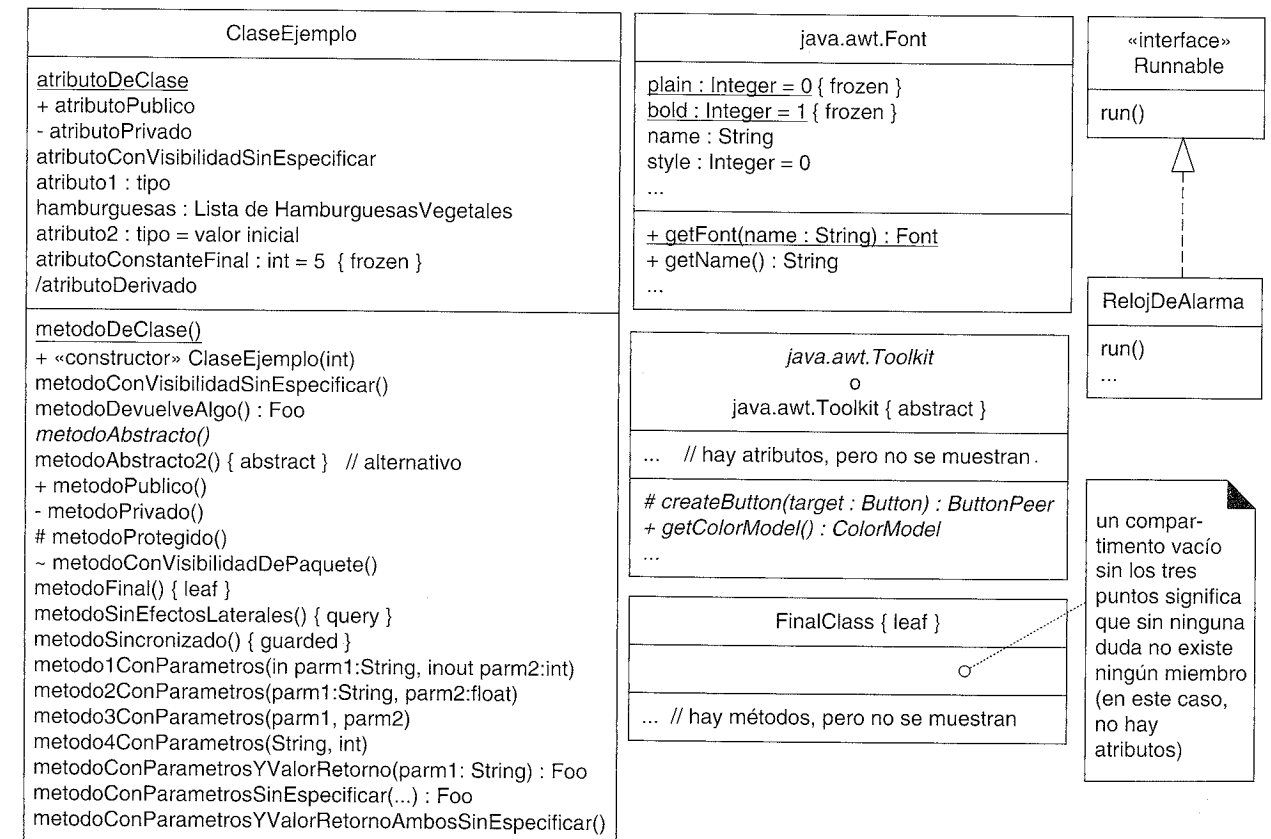


Figura 19.12. Detalles de la notación UML para algunos miembros de los diagramas de clases.

Visibilidad por defecto en UML

Si no se muestra explícitamente ningún marcador de visibilidad para un atributo o método, ¿cuál es el valor por defecto? Respuesta: no hay un valor por defecto. Si no se muestra nada, en UML significa “sin especificar”. Sin embargo, la convención común es asumir que los atributos son privados y los métodos públicos, a menos que se indique otra cosa.

La iteración actual del diagrama de clases de diseño del PDV NuevaEra (ver Figura 19.13) no tiene muchos detalles interesantes de los miembros; todos los atributos son privados y todos los métodos públicos.

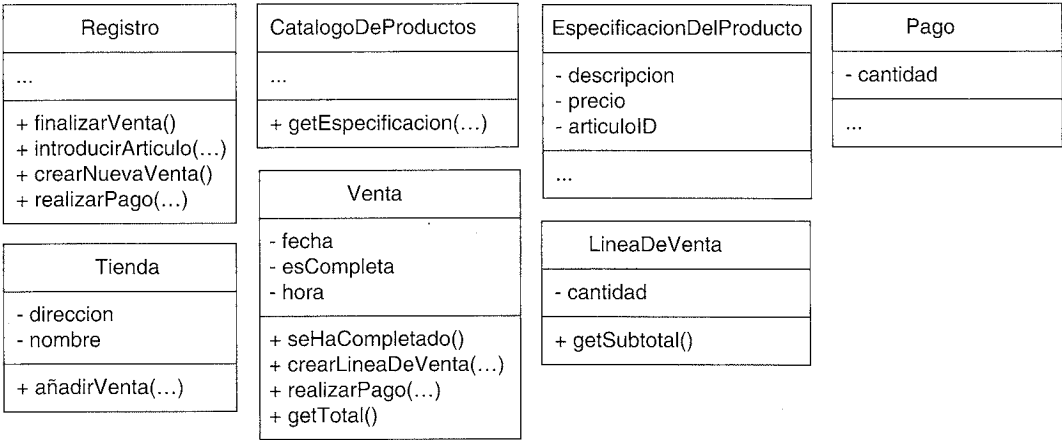


Figura 19.13. Detalles de los miembros del diagrama de clases del PDV.

Notación para el cuerpo de los métodos en los DCD (y los diagramas de interacción)

El cuerpo de un método se puede representar como se ilustra en la Figura 19.14 tanto en el DCD como en el diagrama de interacción.

19.7. DCD, dibujo y herramientas CASE

Las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. En el Capítulo 35 sobre el dibujo y las herramientas CASE, se presentará una breve discusión sobre el contexto del proceso y la práctica de dibujar los DCD.

19.8. DCD en el UP

Los DCD forman parte de la realización de los casos de uso y, por tanto, miembros del Modelo de Diseño del UP.

Fases

Inicio: El Modelo de Diseño y los DCD normalmente no comenzarán hasta la elaboración porque comprenden decisiones de diseño, que son prematuras durante la fase de inicio.

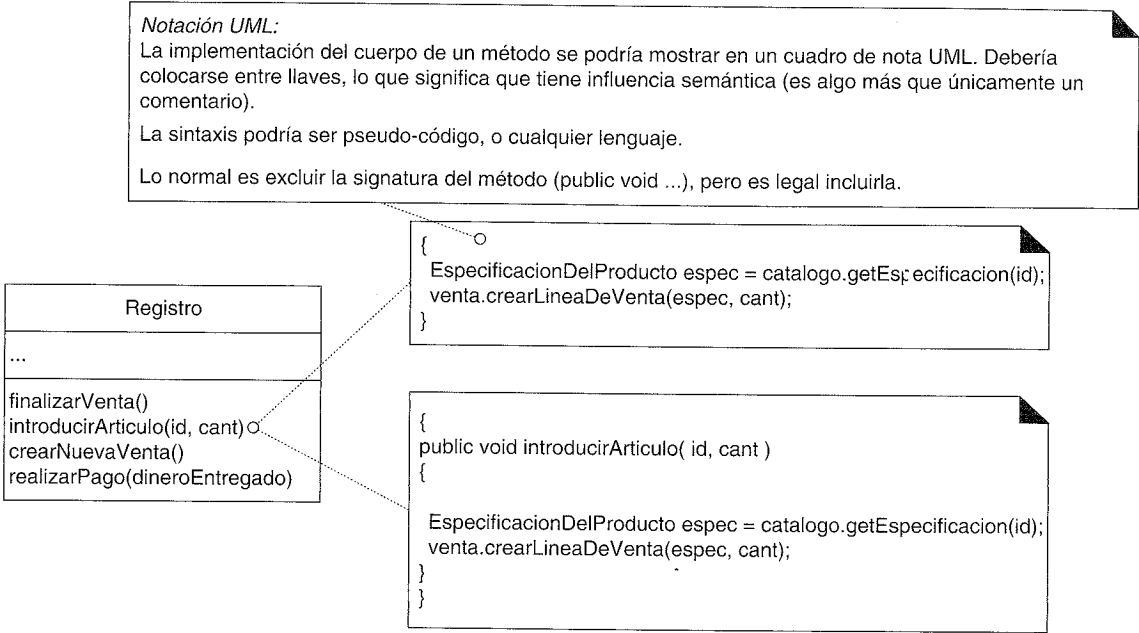


Figura 19.14. Notación para el cuerpo de los métodos.

Tabla 19.1. Muestra de los artefactos UP y evolución temporal. c-comenzar; r-refinar

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Elaboración: Durante esta fase, los DCD acompañarán a los diagramas de interacción de las realizaciones de los casos de uso; podrían crearse para las clases del diseño más significativas desde el punto de vista de la arquitectura.

Nótese que las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. Es recomendable generar los DCD de manera regular a partir del código fuente, para visualizar la estructura estática del sistema.

Construcción: Los DCD se continuarán generando a partir del código fuente como apoyo a la visualización de la estructura estática del sistema.

19.9. Artefactos del UP

La Figura 19.15 muestra la influencia entre los artefactos destacando los DCD.

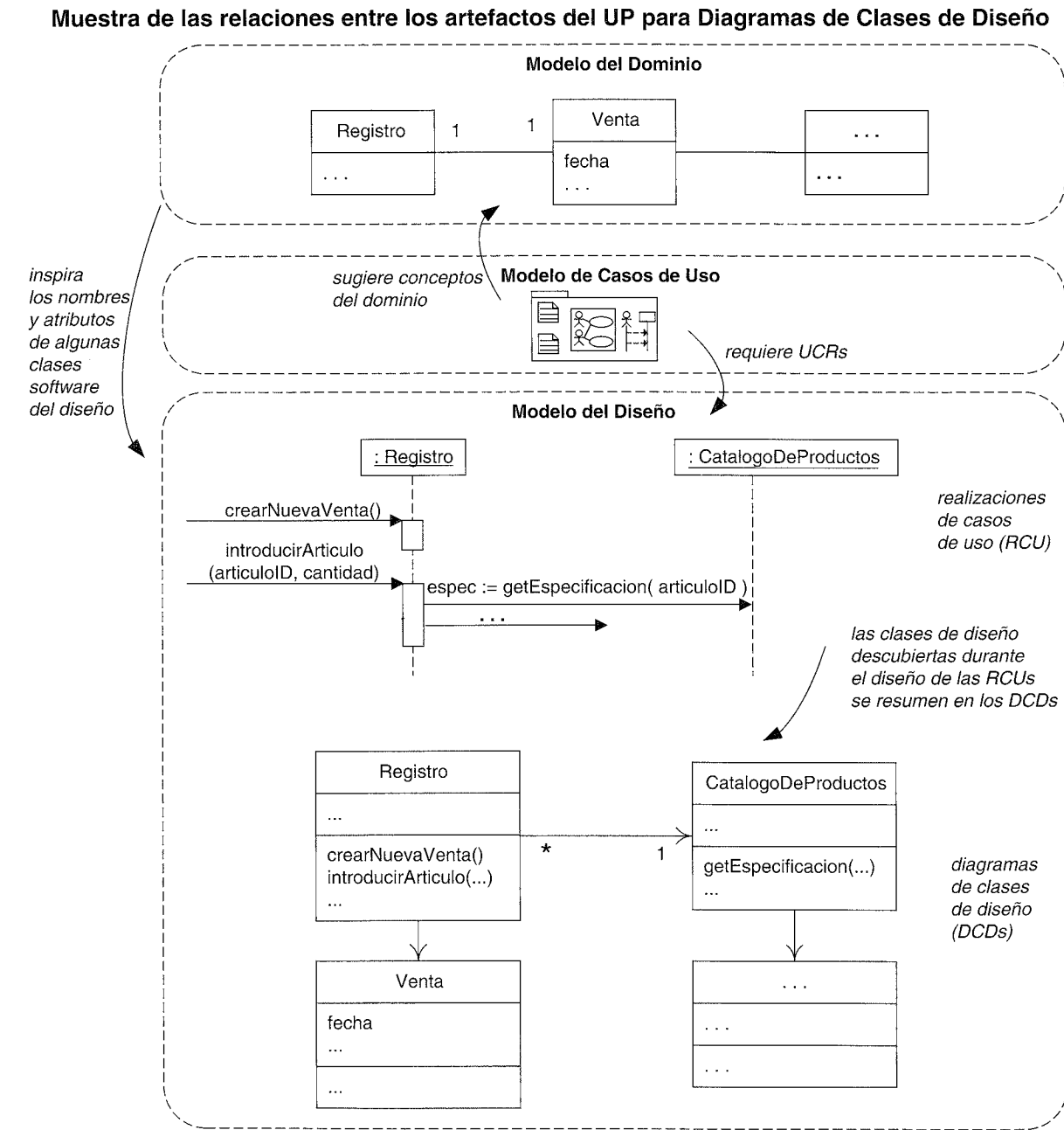


Figura 19.15. Muestra de la influencia entre los artefactos del UP.

MODELO DE IMPLEMENTACIÓN:
TRANSFORMACIÓN
DE LOS DISEÑOS EN CÓDIGO

Esté atento a los errores del programa anterior; sólo he demostrado que es correcto, pero nunca lo he usado.

Donald Knuth

Objetivos

- Transformar los artefactos del diseño en código en un lenguaje orientado a objetos.

Introducción

Después de completar los diagramas de interacción y los DCDs para la iteración actual de la aplicación NuevaEra, disponemos de suficientes detalles para generar el código de la capa del dominio de los objetos.

Los artefactos UML creados durante el trabajo de diseño —los diagramas de interacción y los DCDs— se utilizarán como entradas en el proceso de generación de código.

El UP define el Modelo de Implementación. Éste contiene los artefactos de implementación como el código fuente, las definiciones de bases de datos, las páginas JSP/XML/HTML, etcétera. Por tanto, el código que se va a crear en este capítulo forma parte del Modelo de Implementación.

Elección del lenguaje

Se utiliza Java para los ejemplos debido a su uso extendido y familiaridad. Sin embargo, la intención no es dar a entender que se recomienda Java en especial; C#, Visual Basic, C++ , Smalltalk, Python y muchos otros lenguajes son susceptibles de aplicar los prin-

cipios de diseño de objetos y la transformación a código que se presenta en este caso de estudio.

20.1. Programación y el proceso de desarrollo

El trabajo de diseño anterior no debería implicar que no exista prototipado o diseño durante la programación; las herramientas de desarrollo modernas proporcionan un entorno excelente para explorar rápidamente enfoques alternativos, y normalmente merece la pena elaborar algo (o incluso mucho) de diseño mientras se programa.

Sin embargo, algunos desarrolladores encuentran útil anticipar un poco mediante el modelado visual antes de programar, especialmente aquellos que se encuentran cómodos con el razonamiento visual o los lenguajes basados en diagramas.

Sugerencia

Para una iteración de dos semanas, considere dedicar por lo menos medio día próximo al comienzo de la iteración para llevar a cabo algo de trabajo de diseño mediante el modelado visual, antes de pasar a la programación. Utilice "herramientas" sencillas que soporten la realización rápida y creativa de diagramas, como una pizarra y una cámara digital. Si encuentra una herramienta para ingeniería del software asistida por ordenador (CASE, *computer-aid software engineering*) para UML que sea igualmente rápida, sencilla, y adecuada, excelente.

La creación de código en un lenguaje de programación orientado a objetos —como Java o C#— no forma parte del A/DOO; es un objetivo final. Los artefactos creados en el Modelo de Diseño del UP proporcionan parte de la información necesaria para generar el código.

Una ventaja del A/DOO y de la programación OO —cuando se utiliza con el UP— es que proporcionan una guía de principio a fin desde los requisitos hasta el código. Los distintos artefactos suministran información a los artefactos posteriores de manera útil y siguiendo una traza, culminando finalmente en una aplicación en ejecución. Esto no sugiere que el camino será fácil, o que se pueda seguir de manera mecánica —existen demasiadas variables—. Pero contar con una guía proporciona un punto de partida para experimentar y discutir.

Creatividad y cambio durante la implementación

Durante el trabajo de diseño se tomaron algunas decisiones y se llevó a cabo un trabajo creativo. Veremos a lo largo de la siguiente presentación que la generación de código —en este ejemplo— es un proceso de traducción relativamente mecánico.

Sin embargo, en general, el trabajo de programación no es una etapa de generación de código trivial, más bien lo contrario. En realidad, los resultados generados durante el diseño son un primer paso incompleto; durante la programación y pruebas, se realizarán innumerables cambios y se descubrirán y resolverán problemas complicados.

Si se hace bien, los artefactos del diseño proporcionarán un núcleo elástico que podrá extenderse con elegancia y robustez para satisfacer los nuevos problemas encon-

trados durante la programación. En consecuencia, espere y planifique para cambios y desviaciones del diseño durante la programación.

Cambios del código y el proceso iterativo

Una ventaja del proceso de desarrollo iterativo e incremental es que los resultados de la iteración anterior pueden proporcionar los datos para el comienzo de la siguiente iteración (ver Figura 20.1). Por tanto, los resultados del análisis y diseño actual se están refinando y enriqueciendo continuamente a partir del trabajo de implementación anterior. Por ejemplo, cuando el código de la iteración N se desvía del diseño de la iteración N (que lo hará inevitablemente), el diseño final que es la base de la implementación puede ser la entrada para los modelos de análisis y diseño de la iteración N+1.

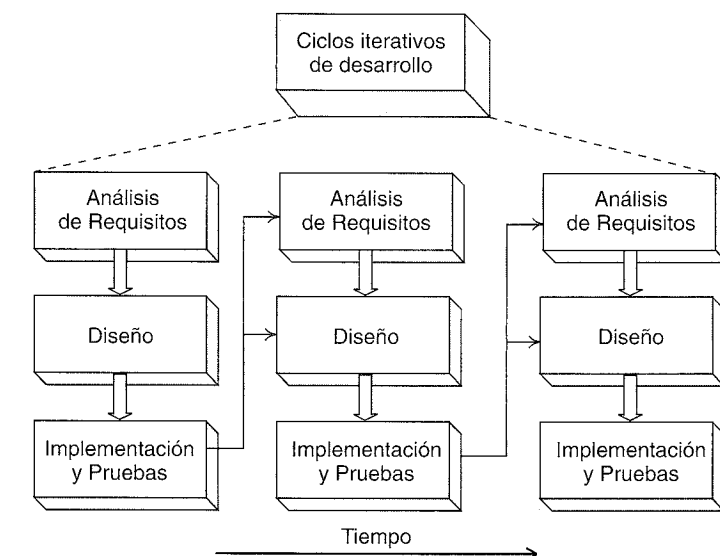


Figura 20.1. La implementación en una iteración influye en el diseño posterior.

Una de las primeras actividades en una iteración es sincronizar los diagramas de diseño; los primeros diagramas de la iteración N no corresponderán con el código final de esa misma iteración, y necesitan sincronizarse antes de que se extiendan con nuevos resultados del diseño.

Cambios en el código, herramientas CASE, e ingeniería inversa

Es deseable que los diagramas generados durante el diseño se actualicen de manera semi-automática para reflejar los cambios en el trabajo de codificación siguiente. Generalmente esto debería hacerse con una herramienta CASE que puede leer el código fuente y generar automáticamente, por ejemplo, diagramas de paquetes, clases y secuencia. Este es un aspecto de la **ingeniería inversa** —la actividad de generar diagramas a partir del código fuente (o a veces, ejecutable)—.

20.2. Transformación de los diseños en código

La implementación en un lenguaje orientado a objetos requiere la escritura de código fuente para:

- Las definiciones de las clases e interfaces.
- Las definiciones de los métodos.

Las secciones siguientes presentarán la generación en Java (como un caso típico).

20.3. Creación de las definiciones de las clases a partir de los DCDs

Como mínimo, los DCDs describen los nombres de las clases o interfaces, las superclases, signature de los métodos y los atributos simples de una clase. Esto es suficiente para crear una definición de clase básica en un lenguaje de programación orientado a objetos. Una discusión posterior explorará la incorporación de la información acerca de las interfaces y ámbito de nombres (o paquetes), entre otros detalles.

Definición de una clase con métodos y atributos simples

A partir del DCD, la transformación de las definiciones de los atributos básicas (campos de instancia simples de Java) y las signatures de los métodos de la *LineaDeVenta* a la definición Java es directa, como se muestra en la Figura 20.2.

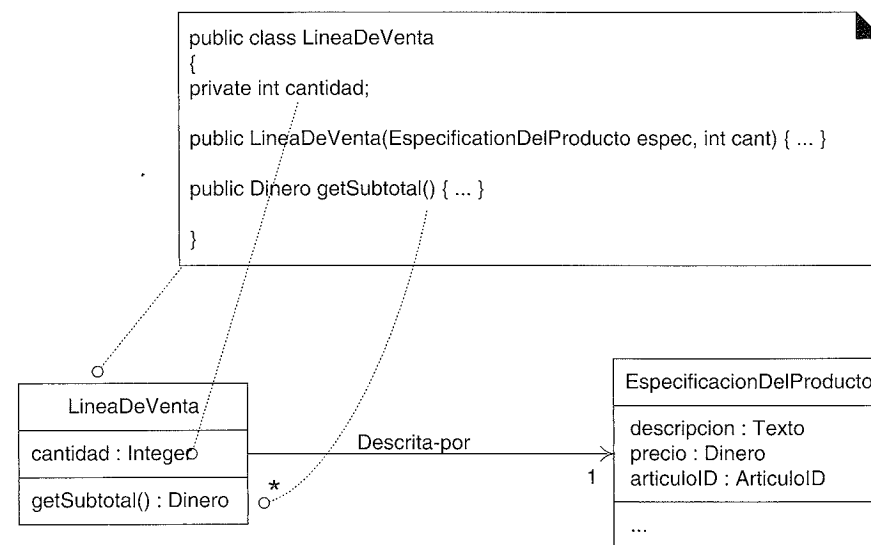


Figura 20.2. LineaDeVenta en Java.

Obsérvese que se ha incorporado en el código fuente el constructor Java *LineaDeVenta(...)*. Se deriva del envío del mensaje *create(espec, ctd)* a la *LineaDeVenta* en el diagrama de interacción *introducirArticulo*. Esto indica, en Java, que se requiere un constructor que soporte estos parámetros. A menudo se excluye de los diagramas de clases el método *create* puesto que es común que aparezca y tiene múltiples interpretaciones, dependiendo del lenguaje que se vaya a utilizar.

Añadir atributos de referencia

Un **atributo de referencia** es un atributo que referencia a otro objeto complejo, no a un tipo primitivo como una cadena de texto, un número, etcétera.

Los atributos de referencia de una clase se deducen de las asociaciones y la navegabilidad de un diagrama de clases.

Por ejemplo, una *LineaDeVenta* tiene una asociación con una *EspecificacionDelProducto*, con navegabilidad hacia ella. Es habitual que esto se interprete como un atributo de referencia en la clase *LineaDeVenta* que hace referencia a la instancia de *EspecificacionDelProducto* (ver Figura 20.3).

En Java, esto significa que se recomienda un campo de instancia que referencia a una instancia de *EspecificacionDelProducto*.

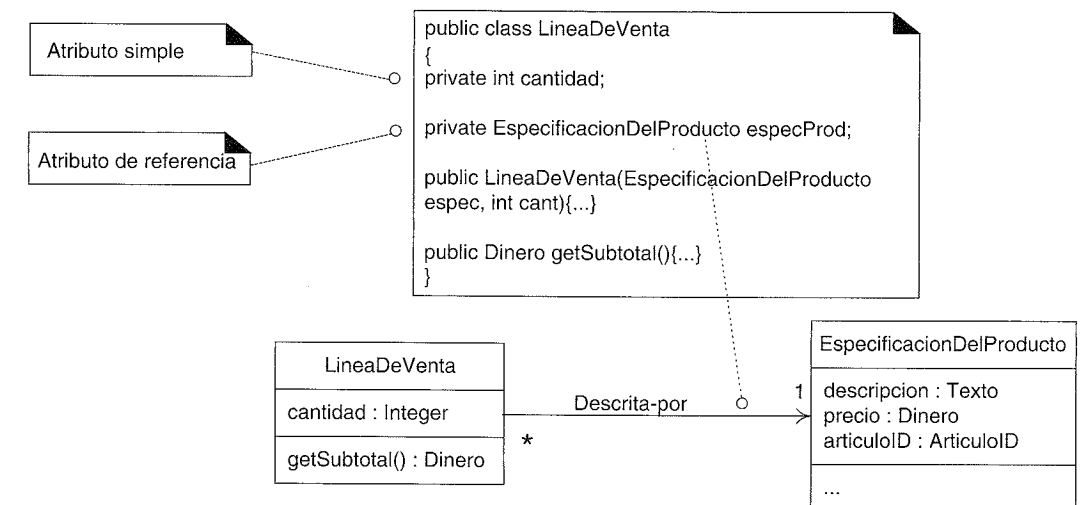


Figura 20.3. Inclusión de atributos de referencia.

Obsérvese que los atributos de referencia de una clase a menudo están implícitos, en lugar de explícitos, en un DCD.

Por ejemplo, aunque hemos añadido un campo de instancia en la definición Java de *LineaDeVenta* para que apunte a una *EspecificacionDelProducto*, no se declara explícitamente como un atributo en la sección de atributos del rectángulo de la clase. Existe una visibilidad de atributo *recomendada* —indicada por la asociación y navegabilidad— que se define explícitamente mediante un atributo durante la fase de generación de código.

Atributos de referencia y los nombres de los roles

La siguiente iteración estudiará el concepto de los nombres de los roles en los diagramas de estructura estáticos. Cada extremo de asociación se denomina rol. Brevemente, un **nombre de rol** es un nombre que identifica al rol y, a menudo, proporciona algo del contexto semántico acerca de la naturaleza del rol.

Si el nombre de un rol está presente en un diagrama de clases, utilícelo como base para el nombre del atributo de referencia durante la generación de código, como se muestra en la Figura 20.4.

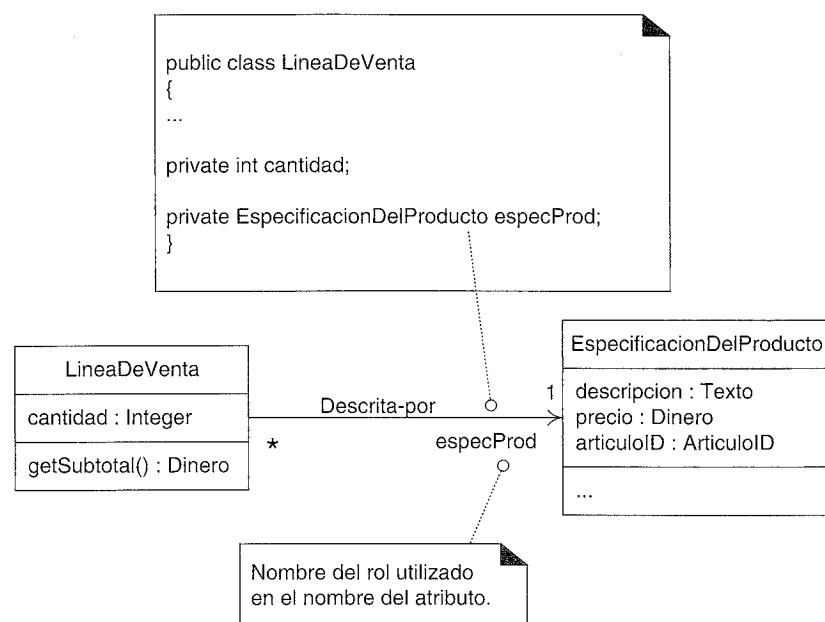


Figura 20.4. Los nombres de los roles podrían utilizarse para generar los nombres de las variables de instancia.

Transformación de los atributos

La clase *Venta* ilustra que en algunos casos uno debe considerar la transformación de los atributos desde el diseño al código en lenguajes diferentes. La Figura 20.5 muestra el problema y su solución.

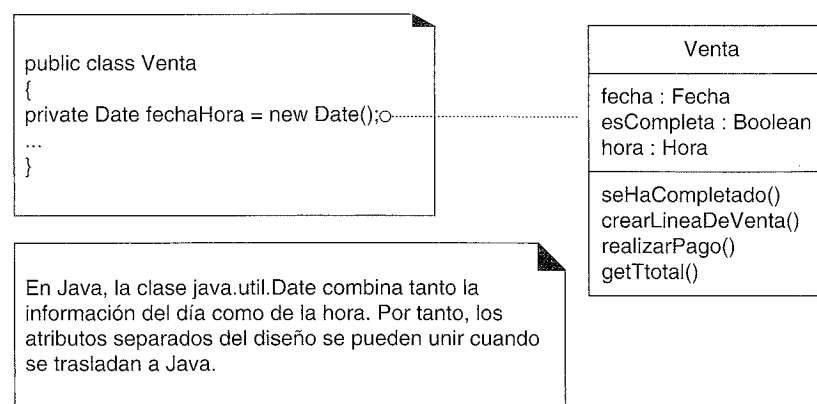


Figura 20.5. Transformación de la fecha y la hora en Java.

20.4. Creación de métodos a partir de los diagramas de interacción

Un diagrama de interacción muestra los mensajes que se envían como respuesta a la invocación de un método. La secuencia de estos mensajes se traduce en una serie de sentencias en la definición del método. El diagrama de interacción *introducirArticulo* de la Figura 20.6 ilustra la definición Java del método *introducirArticulo*.

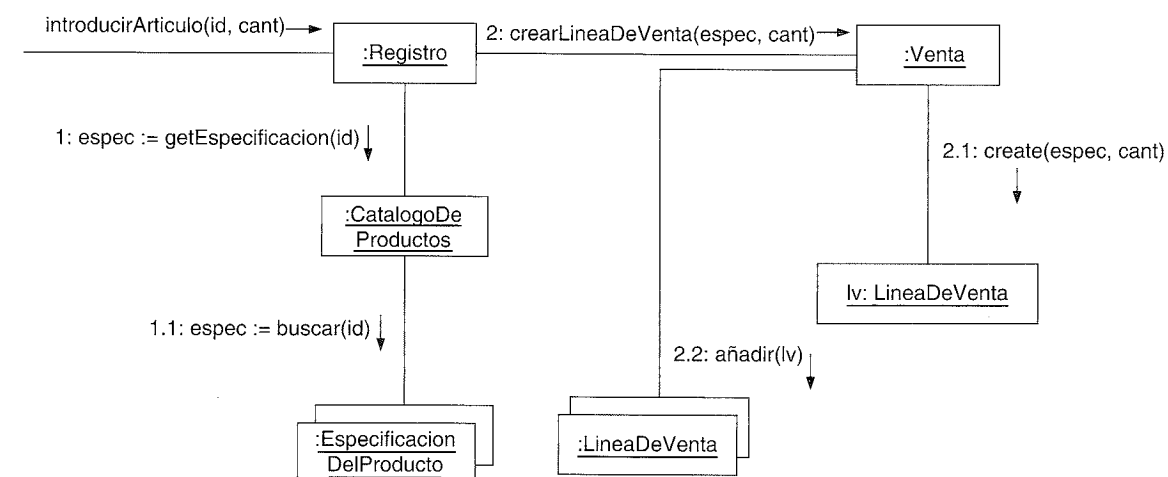


Figura 20.6. El diagrama de interacción de *introducirArticulo*.

En este ejemplo, se utilizará la clase *Registro*. La Figura 20.7 muestra una definición en Java.

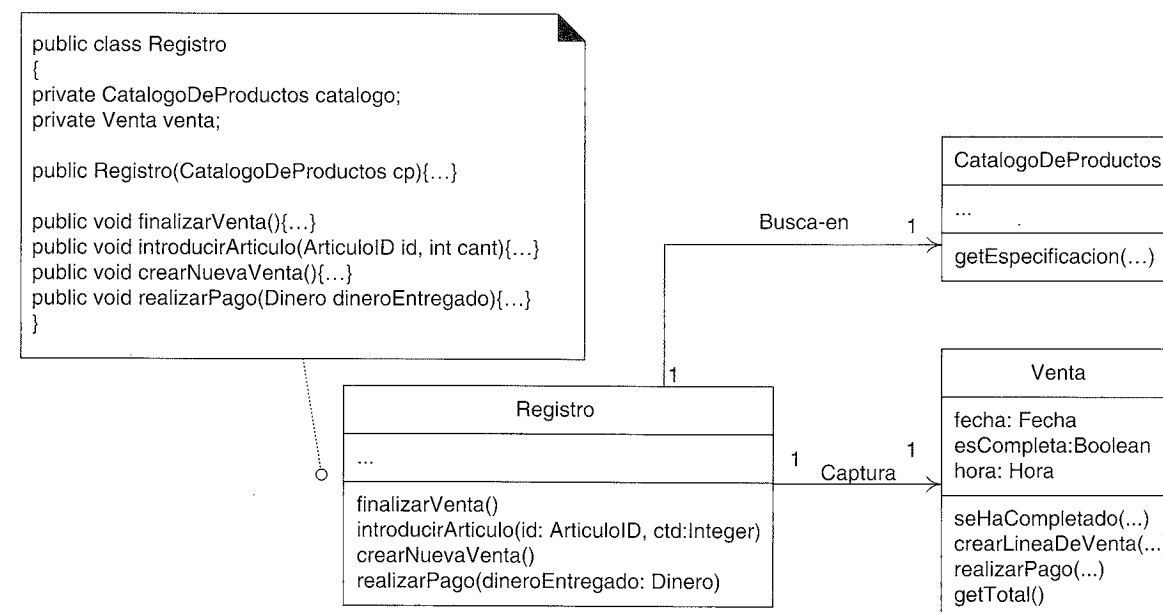


Figura 20.7. La clase *Registro*.

El método Registro--introducirArticulo

Se envía el mensaje *introducirArticulo* a una instancia de *Registro*; por tanto, se define el método *introducirArticulo* en la clase *Registro*.

```
public void introducirArticulo(ArticuloID articuloID, int ctd)
```

Mensaje 1: Se envía el mensaje *getEspecificacion* al *CatalogoDeProductos* para recuperar una *EspecificacionDelProducto*.

```
EspecificacionDelProducto espec =  
catalogo.getEspecificacion(articuloID);
```

Mensaje 2: Se envía el mensaje *crearNuevaVenta* a la *Venta*.

```
venta.crearLineaDeVenta(espec, ctd);
```

Resumiendo, cada mensaje de la secuencia del método, que se muestra en el diagrama de interacción, se transforma en una sentencia del método Java.

La Figura 20.8 muestra el método completo *introducirArticulo* y su relación con el diagrama de interacción.

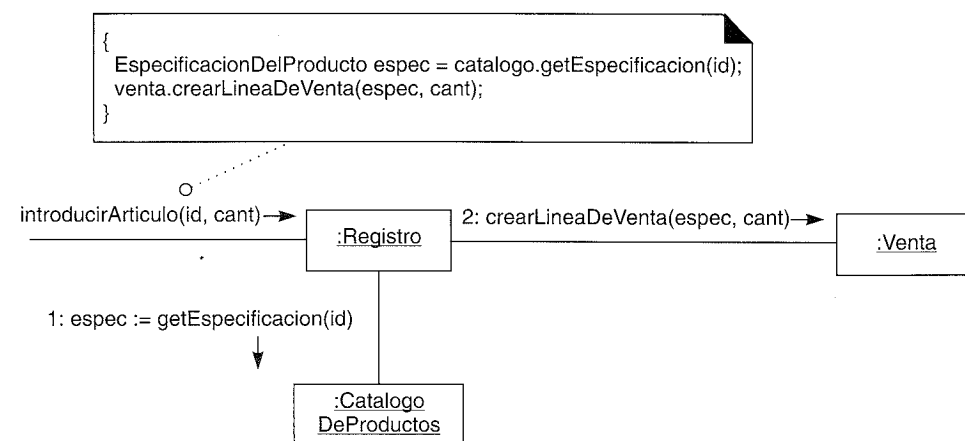


Figura 20.8. El método introducirArticulo.

20.5. Clases contenedoras/colecciones en el código

A menudo es necesario que un objeto mantenga la visibilidad a un grupo de otros objetos; normalmente esta necesidad es evidente a partir del valor de la multiplicidad en el diagrama de clases —podría ser mayor que uno—. Por ejemplo, una *Venta* debe mantener la visibilidad a un grupo de instancias de *LineaDeVenta*, como se muestra en la Figura 20.9.

En los lenguajes de programación OO, con frecuencia se implementan estas relaciones introduciendo un contenedor o colección intermedia. La clase del lado del uno de-

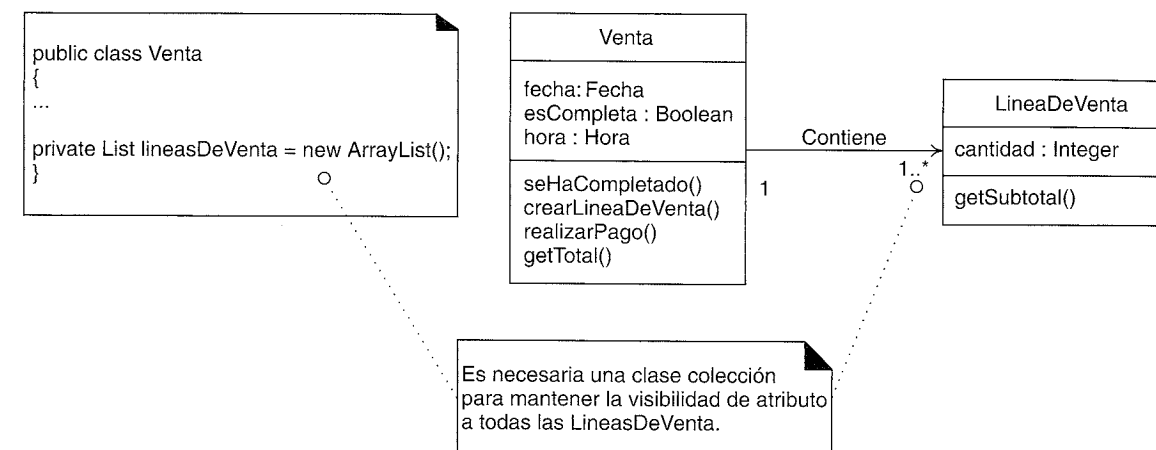


Figura 20.9. Incorporación de una colección.

fine un atributo de referencia que apunta a la instancia del contenedor/colección, que contiene instancias de la clase del lado de muchos.

Por ejemplo, las librerías de Java contienen las clases de colección como *ArrayList* y *HashMap*, que implementan las interfaces de *List* y *Map*, respectivamente. Utilizando *ArrayList*, la clase *Venta* puede definir un atributo que mantenga una lista ordenada de las instancias de *LineaDeVenta*.

Por supuesto, los requisitos influyen en la elección de la clase colección; las búsquedas por claves requieren el uso de un *Map*, una lista ordenada creciente requiere una *List*, etcétera.

20.6. Manejo de excepciones y de errores

Hasta el momento se ha ignorado el manejo de las excepciones en el desarrollo de una solución. Esto se hizo a propósito para centrarnos en las cuestiones básicas de la asignación de responsabilidades y el diseño de objetos. Sin embargo, en el desarrollo de la aplicación, es aconsejable tener en cuenta el manejo de las excepciones durante el trabajo de diseño y, por supuesto, durante la implementación.

Brevemente, en UML, las excepciones se representan como mensajes asíncronos en los diagramas de interacción. Esto se estudiará en el Capítulo 33.

20.7. Definición del método Venta--crearLineaDeVenta

Como último ejemplo, también se puede escribir el método *crearLineaDeVenta* de la clase *Venta* inspeccionando el diagrama de colaboración *introducirArticulo*. La Figura 20.10 muestra una versión abreviada del diagrama de interacción, con el método Java acompañante.

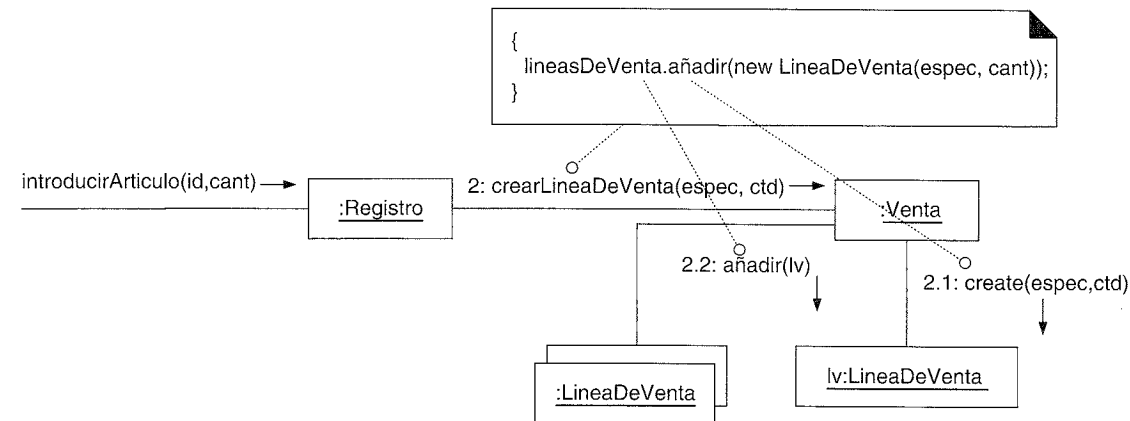


Figura 20.10. Método Venta--crearLineaDeVenta.

20.8. Orden de implementación

Es necesario implementar las clases (e idealmente, hacer las pruebas de unidad completamente) desde la menos a la más acoplada (ver Figura 20.11). Por ejemplo, las posibles primeras clases que podríamos implementar son *Pago* o *EspecificacionDelProducto*; a continuación las clases que sólo dependen de la implementación anterior —*CatalogoDeProductos* o *LineaDeVenta*—.

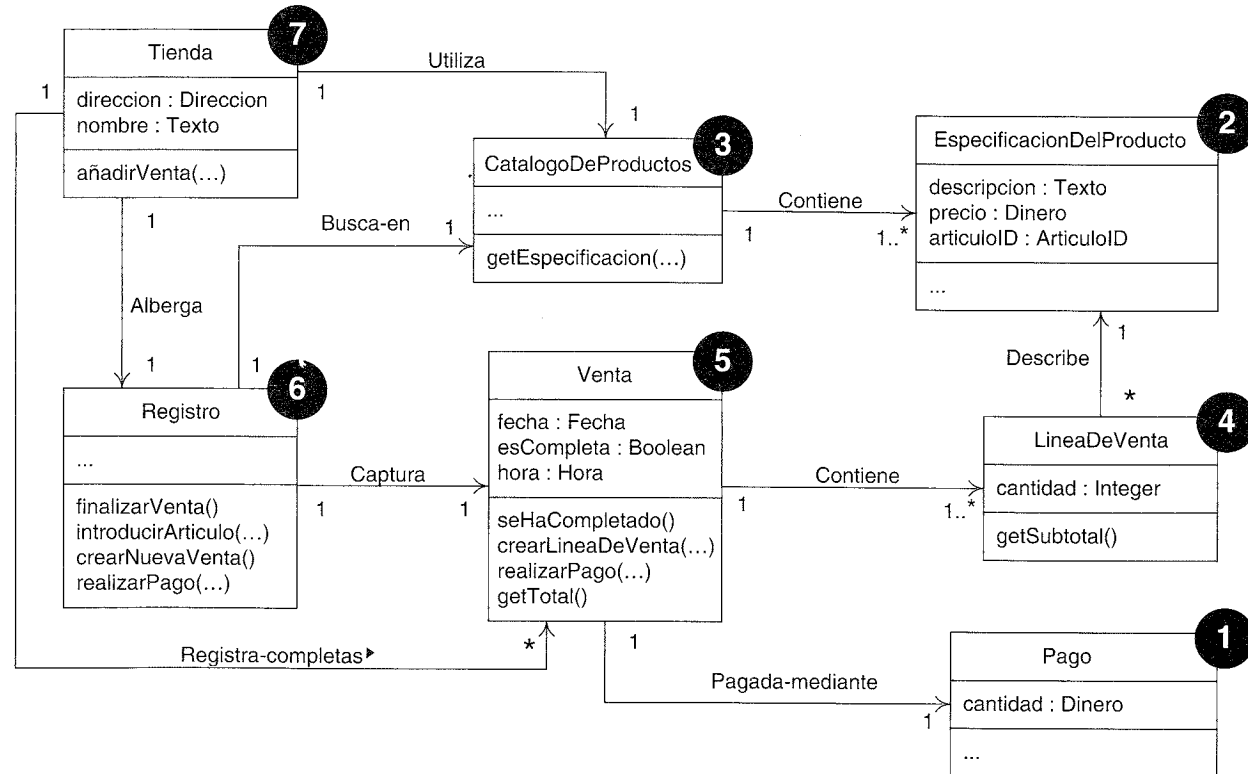


Figura 20.11. Posible orden para la implementación y prueba de las clases.

20.9. Programar probando primero

Una práctica excelente promovida por el método de Programación Extrema (XP, *eXtreme Programming*) [Beck00], y aplicable al UP (como son la mayoría de las prácticas de XP), es **programar probando primero**. En esta práctica, se escribe el código de las pruebas de unidad *antes* del código que se va a probar, y los desarrolladores escriben el código de las pruebas de unidad para *todo* el código de producción. La secuencia básica es escribir un poco del código de prueba, luego escribir un poco del código de producción, hacer que pase las pruebas, y entonces escribir más código de prueba, y así sucesivamente.

Entre las ventajas encontramos:

- **Se escriben realmente las pruebas de unidad:** La naturaleza humana (o al menos la de los programadores) es de tal manera que es muy común evitar la escritura de las pruebas de unidad, si se dejó como una idea adicional.
- **Satisfacción de los programadores:** Si un desarrollador escribe el código de producción, depura de manera informal, y entonces como ocurrencia tardía añade pruebas de unidad, no se siente muy satisfecho. Sin embargo, si se escriben primero las pruebas, y entonces se crea y refina el código de producción para que pase las pruebas, existe un sentimiento de logro —de pasar una prueba—. No se puede ignorar el aspecto psicológico del desarrollo —la programación es un esfuerzo humano—.
- **Aclaración de las interfaces y el comportamiento:** A menudo, la interfaz pública exacta y el comportamiento de una clase no están perfectamente claros hasta que se implementan. Escribiendo las pruebas de unidad primero, uno aclara el diseño de la clase.
- **Verificación demostrable:** Obviamente, el contar con cientos o miles de pruebas de unidad proporciona una verificación significativa de la corrección.
- **La confianza en cambiar cosas:** Si programamos probando primero, tendremos de cientos o miles de pruebas de unidad, y una clase de pruebas de unidad por cada clase de producción. Cuando un desarrollador necesita cambiar el código existente —escrito por ellos mismos o por otros— existe un conjunto de pruebas de unidad que pueden ejecutarse, proporcionando una retroalimentación inmediata sobre si el cambio causó un error.

Como ejemplo, JUnit (www.junit.org) es un marco de pruebas de unidad para Java gratuito, simple y popular. Suponga que estamos utilizando JUnit y seguimos la estrategia de programar probando primero, para crear la clase *Venta*. *Antes* de programar la clase *Venta*, escribimos un método de prueba de unidad en la clase *PruebaVenta* que hace lo siguiente:

- Crea una nueva venta.
- Le añade algunas líneas de venta.
- Solicita el total y comprueba si tiene el valor esperado.

Por ejemplo,

```

public class PruebaVenta extends TestCase
{
    //...
    public void pruebaTotal()
    {
        //inicializa la prueba
        Dinero total = new Dinero(7.5);
        Dinero precio = new Dinero (2.5);
        ArtículoID id = new ArtículoID (1);
        EspecificacionDelProducto espec;
        espec = new EspecificacionDelProducto(id, precio, "producto 1");
        Venta venta = new Venta();

        //añade los artículos
        venta.crearLineaDeVenta(espec, 1);
        venta.crearLineaDeVenta(espec, 2);

        //Comprobar que el total es 7.5
        assertEquals(venta.getTotal(), total);
    }
}

```

Sólo después de crear esta clase *PruebaVenta* será cuando escribamos la clase *Venta* que pase esta prueba. Sin embargo, no es necesario escribir todos los métodos de prueba de antemano. Un desarrollador escribe un método de prueba, después el código de producción que la satisface, a continuación otro método de prueba, y así sucesivamente.

20.10. Resumen de la transformación de los diseños en código

El proceso de traducción de los DCD a las definiciones de las clases, y de los diagramas de interacción en los métodos, es relativamente directo. Durante el trabajo de programación todavía hay cabida para tomar decisiones, realizar cambios en el diseño y explorar, pero algunas de las grandes ideas del diseño se tuvieron en cuenta antes de la programación.

20.11. Introducción a la solución del programa

Esta sección presenta una muestra de la solución del programa de la capa de los objetos del dominio en Java para esta iteración. La generación del código se deriva casi por completo de los diagramas de clases del diseño y los diagramas de interacción que se definieron en el trabajo de diseño, y se basa en los principios de transformación de los diseños a código que se han estudiado previamente.

Lo más importante de este listado es mostrar que se traducen los artefactos de diseño a una base de código. Este código define un caso sencillo; la intención no es ilustrar un programa Java robusto y totalmente desarrollado con sincronización, manejo de excepciones, etcétera.

Clase Pago

```

public class Pago
{
    private Dinero cantidad;
    public Pago(Dinero dineroEntregado){ cantidad = dineroEntregado; }
    public Dinero getCantidad() { return cantidad; }
}

```

Clase CatalogoDeProductos

```

public class CatalogoDeProductos
{
    private Map especificacionesDeProductos = new HashMap();

    public CatalogoDeProductos()
    {
        // datos de ejemplo
        ArtículoID id1 = new ArtículoID( 100 );
        ArtículoID id2 = new ArtículoID( 200 );
        Dinero precio = new Dinero( 3 );

        EspecificacionDelProducto ep;
        ep = new EspecificacionDelProducto( id1, precio, "producto 1");
        especificacionesDeProductos.put( id1, ep );
        ep = new EspecificacionDelProducto( id2, precio, "producto 2");
        especificacionesDeProductos.put( id2, ep );
    }

    public EspecificacionDelProducto getEspecificacion (ArtículoID id)
    {
        return (EspecificacionDelProducto)especificacionesDeProductos.get(id);
    }
}

```

Clase Registro

```

public class Registro{
{
    private CatalogoDeProductos catalogo;
    private Venta venta;

    public Registro(CatalogoDeProductos catalogo)
    {
        this.catalogo = catalogo;
    }

    public void finalizarVenta()
    {
        venta.seHaCompletado();
    }

    public void introducirArticulo(ArtículoID id, int cantidad)
    {
        EspecificacionDelProducto espec = catalogo.getEspecificacion(id);
    }
}

```

```

    venta.crearLineaDeVenta( espec, cantidad);
    }

    public void crearNuevaVenta()
    {
        venta = new Venta();
    }

    public void realizarPago(Dinero dineroEntregado)
    {
        venta.realizarPago(dineroEntregado);
    }
}

```

Clase EspecificacionDelProducto

```

public class EspecificacionDelProducto
{
    private ArtículoID id;
    private Dinero precio;
    private String descripcion;

    public EspecificacionDelProducto
        (ArtículoID id, Dinero precio, String descripcion)
    {
        this.id = id;
        this.precio = precio;
        this.descripcion = descripcion;
    }

    public ArtículoID getArtículoID() { return id; }

    public Dinero getPrecio() { return precio; }

    public String getDescripcion() { return descripcion; }
}

```

Clase Venta

```

public class Venta
{
    private List lineasDeVenta = new ArrayList();
    private Date fecha = new Date();
    private boolean esCompleta() = false;
    private Pago pago;

    public Dinero getDevolucion()
    {
        return pago.getCantidad().minus( getTotal() );
    }

    public void seHaCompletado(){ esCompleta = true; }

    public boolean esCompleta(){ return esCompleta;}
}

```

```

public void crearLineaDeVenta
    (EspecificacionDelProducto espec, int cantidad)
{
    lineasDeVenta.add( new LineaDeVenta(espec, cantidad) );
}

public Dinero getTotal()
{
    Dinero total = new Dinero();
    Iterator i = lineasDeVenta.iterator();
    while (i.hasNext())
    {
        LineaDeVenta ldv = (LineaDeVenta)i.next();
        total.add(ldv.getSubtotal() );
    }
    return total;
}

public void realizarPago(Dinero dineroEntregado)
{
    pago = new Pago( dineroEntregado);
}
}

```

Clase LineaDeVenta

```

public class LineaDeVenta
{
    private int cantidad;
    private EspecificacionDelProducto especProducto;

    public LineaDeVenta (EspecificacionDelProducto espec, int cantidad)
    {
        this.especProducto = espec;
        this.cantidad = cantidad;
    }

    public Dinero getSubtotal()
    {
        return especProducto.getPrecio().times( cantidad );
    }
}

```

Clase Tienda

```

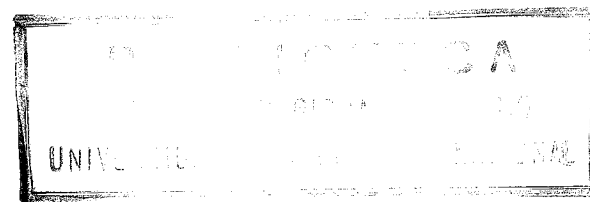
public class Tienda
{
    private CatalogoDeProductos catalogo = new CatalogoDeProductos();
    private Registro registro = new Registro(catalogo);

    public Registro getRegistro() { return registro; }
}

```

Parte 4

ELABORACIÓN EN LA ITERACIÓN 2



LA ITERACIÓN 2 Y SUS REQUISITOS

21.1. Énfasis de la Iteración 2: diseño de objetos y patrones

Los capítulos de la fase de concepción y los de la Iteración 1 de la fase de elaboración han hecho hincapié en un conjunto de técnicas fundamentales del análisis y diseño de objetos, con el objeto de compartir información sobre una amplia gama de pasos comunes en la construcción de sistemas de objetos.

En esta iteración, el caso de estudio se centrará en:

- El diseño de objetos esencial.
- El uso de patrones para crear un diseño sólido.
- La aplicación de UML para visualizar los modelos.

Estos son objetivos básicos del libro y habilidades necesarias.

Hay una discusión mínima acerca del análisis de requisitos o el modelado del dominio, y la explicación del diseño es más concisa, ahora que (en la Iteración 1) ya se ha presentado una explicación detallada sobre las bases de cómo pensar en objetos.

Por supuesto, en esta iteración tendrían lugar muchas otras actividades de análisis, diseño e implementación, pero se les presta menos atención para centrarnos en proporcionar información sobre cómo hacer el diseño de objetos.

21.2. De la Iteración 1 a la 2

Cuando termina la Iteración 1, se debería cumplir lo siguiente:

- Se ha dedicado bastante esfuerzo a probar todo el software: unidad, aceptación, carga, facilidad de uso, etcétera. La idea en el UP es llevar a cabo verificaciones de calidad y corrección tempranas, realistas y continuas, de manera que la pronta retroalimentación guíe a los desarrolladores a adaptar y mejorar el sistema, encontrando su “camino verdadero”.
- Los clientes se han involucrado regularmente en la evaluación del sistema parcial, para obtener retroalimentación con el objeto de adaptar y clarificar los requisitos. Y los clientes consiguen ver pronto un progreso visible del sistema.
- El sistema, a través de todos los subsistemas, se ha integrado completamente y estabilizado como versión base interna.

En aras de la brevedad, se han omitido muchas actividades del final de la Iteración 1 e inicio de la Iteración 2, puesto que esta presentación concede mucha importancia a introducir el A/DOO. A continuación presentamos comentarios sobre unas pocas de las muchas actividades que se han saltado:

- Al comienzo de la nueva iteración, utilice una herramienta CASE para llevar a cabo ingeniería inversa y generar los diagramas UML a partir del código fuente de la última iteración (los resultados forman parte del Modelo de Diseño del UP). Éstos se pueden imprimir a gran tamaño con un plotter y colgarse en las paredes de la sala del proyecto, lo que nos ayudaría a comunicar la representación del punto de partida del diseño lógico de la siguiente iteración.
- Se ha comenzado el análisis e ingeniería de usabilidad para la UI. Ésta es una técnica y una actividad extraordinariamente importantes para el éxito de muchos sistemas. Sin embargo, se trata de un tema extenso y no trivial, y queda fuera del alcance de este libro.
- Se ha comenzado el modelado e implementación de la base de datos.
- Se eligen los requisitos de la siguiente iteración cerca del final de la iteración anterior.
- Tiene lugar otro taller de requisitos de dos días (por ejemplo), en el que se escriben más casos de uso en formato completo. Durante la elaboración, mientras quizás se han diseñado e implementado el 10% de los requisitos de más riesgo, hay una actividad *paralela* para estudiar y definir en profundidad quizás el 80% de los casos de uso del sistema, aunque la mayoría de los requisitos no se implementarán hasta la construcción.
 - Entre los participantes se incluirán unos pocos desarrolladores (como el arquitecto de software) de la primera iteración, de manera que, durante este taller, la investigación y las preguntas se documentan a partir de la comprensión (y confusión) adquirida con la construcción rápida y real de parte del software. No hay nada como construir algo del software para descubrir qué es lo que realmente no

conocemos acerca de los requisitos —ésta es una idea clave en el UP y en el desarrollo iterativo—.

Simplificaciones en el caso de estudio

En un proyecto UP bien hecho, los requisitos que se eligen para las primeras iteraciones se organizan de acuerdo con el riesgo y el alto valor para el negocio, de manera que se identifican y se resuelven primero las cuestiones de más riesgo. Sin embargo, si este caso de estudio siguiera exactamente esta estrategia, no ayudaría a explicar las ideas y principios fundamentales del A/DOO en las primeras iteraciones. Por tanto, se han tomado algunas licencias con respecto a la prioridad de los requisitos, prefiriendo aquellos que apoyan los objetivos pedagógicos, en lugar de los objetivos de riesgo del proyecto.

21.3. Requisitos de la Iteración 2

La Iteración 2 de la aplicación del PDV NuevaEra maneja varios requisitos interesantes:

1. Soporte a las variaciones de servicios externos de terceras partes. Por ejemplo, se deben conectar al sistema diferentes calculadores de impuestos, cada uno con su propia interfaz. Del mismo modo con diferentes sistemas de contabilidad, etcétera. Cada uno ofrecerá un API y protocolo diferentes para un núcleo de las funciones comunes.
2. Reglas de fijación de precios complejas.
3. Reglas de negocio conectables.
4. Un diseño para actualizar una ventana del GUI cuando cambia el total de la venta.

Sólo se tendrán en cuenta estos requisitos (en esta iteración) en el contexto de los escenarios del caso de uso *Procesar Venta*.

Nótese que no se han descubierto requisitos nuevos; se identificaron durante la fase de inicio. Por ejemplo, el caso de uso *Procesar Venta* original señala el problema de la fijación de precios:

Escenario principal de éxito:

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
2. El Cajero le dice al sistema que cree una nueva venta.
3. El Cajero introduce el identificador del artículo.
4. El Sistema registra la línea de la venta y presenta la descripción del artículo, precio y suma parcial. ***El precio se calcula a partir de un conjunto de reglas de precios.***

Además, las secciones en la Especificación Complementaria registran los detalles de las reglas del dominio para los precios, y señalan la necesidad de soportar varios sistemas externos:

Especificación Complementaria			
...			
Interfaces			
Interfaces software			
Para la mayoría de los sistema externos de colaboración (calculador de impuestos, contabilidad, inventario,...) necesitamos ser capaces de conectar diversos sistemas y, por tanto, diversas interfaces.			
...			
Reglas de Dominio (Negocio)			
ID	Regla	Grado de variación	Fuente
REGLA4	Reglas de descuento al comprador. Ejemplos: Empleado—20% de descuento Clientes preferentes—10% de descuento Antiguos—15% de descuento	Alto. Cada tienda utiliza reglas diferentes.	Política de la tienda.
...	...	...	...
...			
Información en dominios de interés			
Fijación de precios			
Además de las reglas de fijación de precios que se describen en la sección de reglas del dominio, nótese que los artículos tienen un <i>precio original</i> , y opcionalmente, un <i>precio rebajado</i> . El precio de los artículos (antes de los descuentos adicionales) es el precio rebajado, si existe. Las organizaciones mantienen el precio original incluso si hay un precio rebajado, por razones de contabilidad e impuestos.			
...			

Desarrollo incremental para el mismo caso de uso a lo largo de la iteraciones

Debido a estos requisitos, en la Iteración 2 estamos volviendo sobre el caso de uso *Procesar Venta*, pero implementando más escenarios, de manera que el sistema crece incrementalmente. Es habitual trabajar en varios escenarios o características del mismo caso de uso a lo largo de varias iteraciones y extender el sistema gradualmente para finalmente manejar toda la funcionalidad requerida. Por otro lado, los casos de uso breves y simples, podrían implementarse completamente en una iteración.

La Iteración 1 hizo simplificaciones de manera que el problema y la solución no fueran excesivamente complejas de abordar. Una vez más —por la misma razón— se considera una cantidad relativamente pequeña de funcionalidad adicional.

En un proyecto de desarrollo real los requisitos que se han escogido en el libro para esta iteración no serían la elección indiscutible —otra posibilidad es actualizar el inventario—, gestionar los pagos a crédito, o un caso de uso completamente diferente. Sin embargo, ésta es una elección de suficiente riqueza con oportunidades de aprendizaje valiosas.

21.4. Refinamiento de los artefactos orientados al análisis en esta iteración

Modelo de casos de uso: casos de uso

Como resultado de los requisitos escogidos para esta iteración no se requiere ningún refinamiento de los casos de uso, aunque podrían cambiar como resultado de otras percepciones.

Sin embargo, además del diseño de objetos y la programación, en esta iteración tendrá lugar una actividad paralela de un breve taller de requisitos, con el que se investigarán y escribirán en detalle más casos de uso. Se revisarán los anteriores casos de uso en formato completo (por ejemplo, *Procesar Venta*) y probablemente se refinarán en base al conocimiento adquirido a partir de la Iteración 1. Algunas de estas actualizaciones de los casos de uso se podrían tener en cuenta en la siguiente iteración de la fase de elaboración, pero muchas se pospondrán hasta la construcción (porque no son significativas o arriesgadas para la arquitectura).

Modelo de casos de uso: DSS

Esta iteración abarca la inclusión del soporte para sistemas externos de terceras partes con varias interfaces, como un calculador de impuestos. El sistema del PDV NuevaEra se comunicará de manera remota con los sistemas externos. En consecuencia, deberían actualizarse los DSS para reflejar al menos algunas de las colaboraciones inter-sistemas, para clarificar cuáles son los nuevos eventos de nivel del sistema.

La Figura 21.1 ilustra un DSS para un escenario de pago a crédito, que requiere la colaboración con varios sistemas externos. Aunque en esta iteración no se maneja el pago a crédito, el diseñador (yo) ha dibujado un DSS basado en él (y probablemente varios otros también), para entender mejor la colaboración entre los sistemas y, por tanto, el soporte que se requiere para distintas interfaces en los sistemas externos.

Modelo del Dominio

Después de haber adquirido un poco de experiencia en el modelado del dominio, un modelador puede estimar si un conjunto de requisitos nuevos tendrán un mayor o menor impacto en el Modelo del Dominio, en términos de muchos nuevos conceptos, asociaciones y atributos. A diferencia de la iteración anterior, los requisitos que se van a abordar esta vez no comprenden muchos conceptos del dominio nuevos. Un breve estudio de los nuevos requisitos sugiere algo como *ReglaDePrecio* como concepto del dominio, pero probablemente no hay docenas de cosas nuevas.

En esta situación, es bastante razonable saltarse el refinamiento del Modelo del Dominio, y pasar rápidamente al trabajo de diseño, y dejar que tenga lugar el descubrimiento de nuevos conceptos del dominio durante el diseño de objetos, cuando los diseñadores están considerando detalladamente una solución. Una señal de madurez en el proceso con el UP es entender cuándo la creación de un nuevo artefacto añadirá un

valor significativo, o es una especie de paso mecánico “hacer trabajo” y es mejor saltarlo.

Esta flexibilidad es un arma de doble filo. Con mucha frecuencia, la flexibilidad de saltarse las actividades anteriores a la programación tiene lugar dentro de una creencia excesivamente optimista de que el problema se puede solucionar simplemente precipitándose hacia el código. Si verdaderamente puede, estupendo, porque la programación es el trabajo que realmente importa, no dibujar modelos del dominio. Por otro lado, la mayoría de los desarrolladores tienen historias donde un poco de reflexión, investigación y previsión antes de programar habrían reducido dolor y sufrimiento.

Modelo de caso de uso: contratos de las operaciones del sistema

En esta iteración no se van a considerar nuevas operaciones del sistema y, por tanto, no se necesitan los contratos. En cualquier caso, los contratos son únicamente una opción a tener en cuenta cuando la precisión detallada que ofrecen mejora las descripciones de los casos de uso.

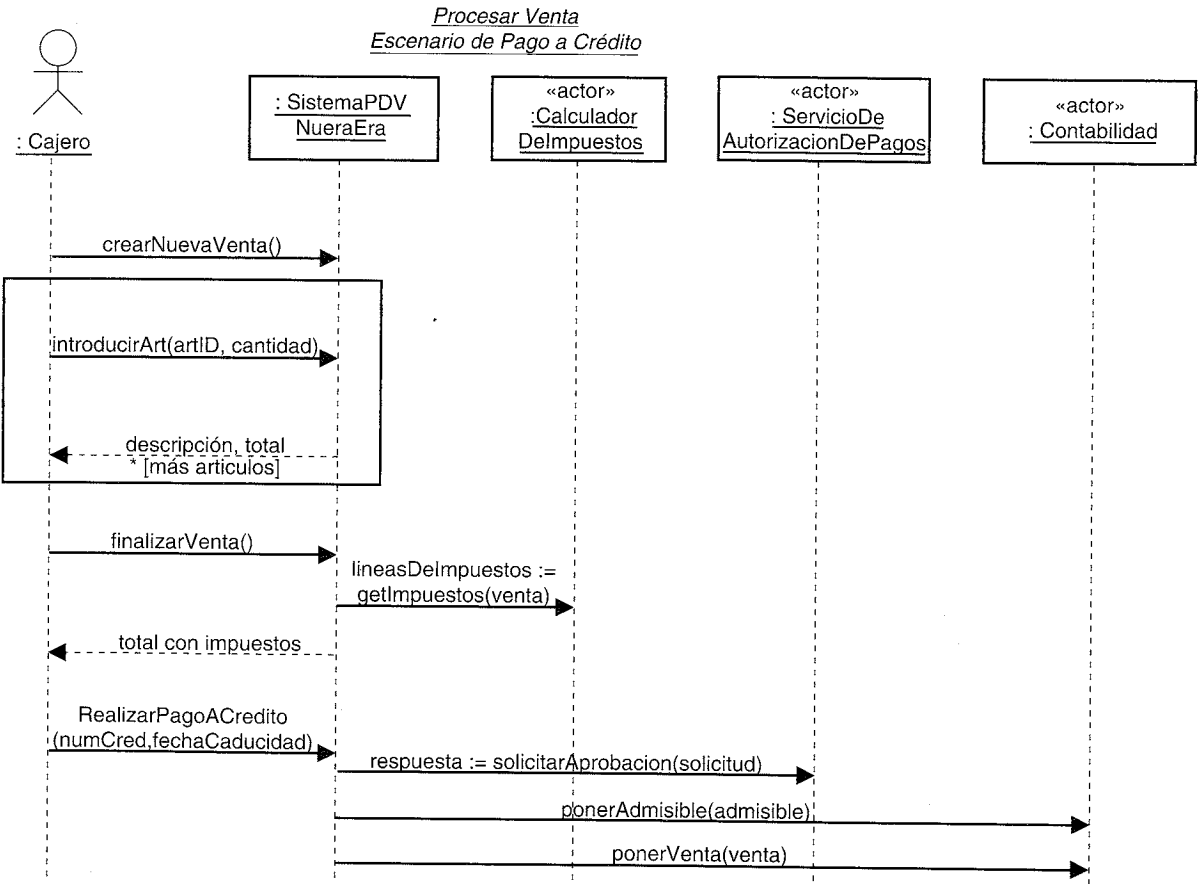


Figura 21.1. Un escenario DSS que ilustra algunos eventos externos.

GRASP: MÁS PATRONES PARA ASIGNAR RESPONSABILIDADES

La suerte es el residuo del diseño.
Branch Rickey

Objetivos

- Aprender a aplicar el resto de los patrones GRASP.

Introducción

- Anteriormente, estudiamos la aplicación de los cinco primeros patrones GRASP:
- Experto en Información, Creador, Alta Cohesión, Bajo Acoplamiento y Controlador.
- Los cuatro últimos patrones GRASP son:
- Polimorfismo.
 - Indirección.
 - Fabricación Pura.
 - Variaciones Protegidas.

Una vez que se hayan explicado, tendremos un vocabulario rico y de uso común con el que discutir los diseños. Y este vocabulario crecerá puesto que también se presentan (en capítulos siguientes) algunos de los patrones de diseño de la “pandilla de los cuatro” (GoF, *Gang-of-Four*) (como Estrategia y Factoría). Una frase corta como, “Sugiero una Estrategia generada a partir de una Factoría para soportar Variaciones Protegidas y Bajo Acoplamiento con respecto a <X>” revela mucha información sobre el diseño, puesto que el nombre de los patrones transmite de manera concisa un concepto de diseño complejo.

Este capítulo presenta el resto de los patrones GRASP, una ayuda para aprender los principios fundamentales según los cuales se asignan las responsabilidades a los objetos y se diseñan éstos.

Los capítulos siguientes introducen otros patrones útiles y los aplican al desarrollo de la segunda iteración de la aplicación del PDV NuevaEra.

22.1. Polimorfismo

Solución Cuando las alternativas o comportamientos relacionados varían según el tipo (clase), asigne la responsabilidad para el comportamiento —utilizando operaciones polimórficas— a los tipos para los que varía el comportamiento¹.

Corolario: No haga comprobaciones acerca del tipo de un objeto y no utilice la lógica condicional para llevar a cabo alternativas diferentes basadas en el tipo.

Problema ¿Cómo manejar las alternativas basadas en el tipo? ¿Cómo crear componentes software conectables (*pluggable*)?

Alternativas basadas en el tipo: La variación condicional es un tema fundamental en los programas. Si se diseña un programa utilizando sentencias de lógica condicional *if-then-else* o *case*, entonces, si aparece una nueva variación, requiere la modificación de la lógica de casos. Este enfoque dificulta que el programa se extienda con facilidad con nuevas variaciones porque se tiende a necesitar los cambios en varios sitios —en donde quiera que exista la lógica condicional.

Componentes software conectables: Viendo los componentes en las relaciones cliente-servidor, ¿cómo podemos sustituir un componente del servidor por otro, sin afectar al cliente?.

Ejemplo En la aplicación del PDV NuevaEra, se deben soportar diferentes sistemas externos de cálculo de impuestos de terceras partes (como Master-En-Impuestos e Impuestos-Pro); el sistema necesita ser capaz de integrarse con distintos de ellos. Cada calculador de impuestos tiene un interfaz diferente, con lo que hay un comportamiento parecido pero que varía para adaptarse a cada uno de estos interfaces externos o APIs. Un producto podría soportar un simple protocolo de sockets TCP, otro podría ofrecer un interfaz SOAP, y un tercero podría ofrecer un interfaz RMI de Java.

¿Qué objetos deberían ser los responsables de manejar estas interfaces diferentes de los calculadores de impuestos externos?

Puesto que el comportamiento para la adaptación del calculador varía según el tipo de calculador, según el Polimorfismo deberíamos asignar la responsabilidad de la adaptación a los diferentes objetos calculadores (o adaptadores de calculador), implementada con una operación polimórfica *getImpuestos* (ver Figura 22.1).

¹ El **polimorfismo** tiene varios significados relacionados. En este contexto, significa “asignar el mismo nombre a servicios en diferentes objetos” [Coad95] cuando los servicios son parecidos o están relacionados. Los diferentes tipos de objetos normalmente implementan un interfaz común o están relacionados en una jerarquía de implementación con una superclase común, pero esto depende del lenguaje; por ejemplo, los lenguajes con ligadura dinámica como Smalltalk no requieren esto.

Estos objetos adaptadores de los calculadores no son los calculadores externos, sino objetos software locales que representan los calculadores externos, o el adaptador para el calculador. Enviando un mensaje al objeto local, finalmente se hará una llamada al calculador externo a través de su API.

Cada método *getImpuestos* toma el objeto *Venta* como parámetro, de manera que el calculador pueda analizar la venta. La implementación de cada método *getImpuestos* será diferente: el *AdaptadorMasterEnImpuestos* adaptará la solicitud al API del Master-En-Impuestos, y así sucesivamente.

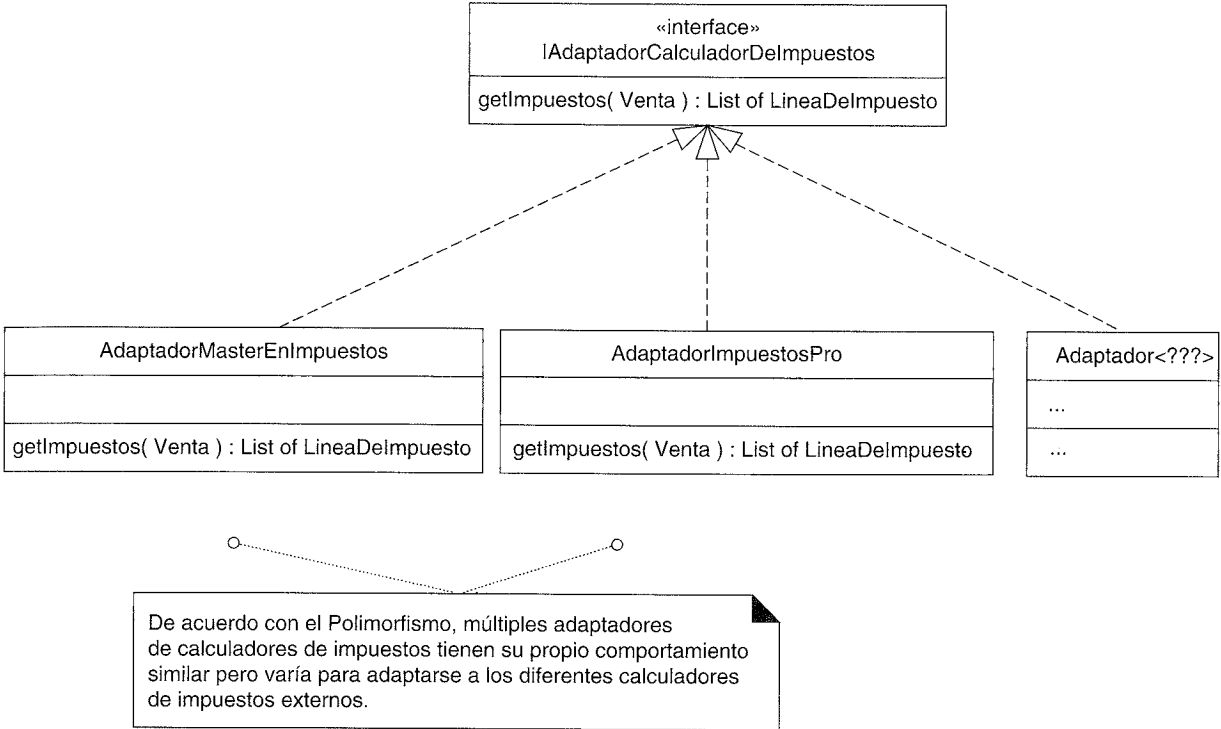


Figura 22.1. Polimorfismo en la adaptación a diferentes calculadores de impuestos externos.

Notación UML: La Figura 22.1 presenta alguna notación nueva para especificar las **interfaces** (un descriptor de operaciones sin implementación), la implementación de las interfaces, y los valores de retorno de tipo “colección”; la Figura 22.2 lo explica en detalle. Se utiliza un **estereotipo** UML para designar una interfaz; un estereotipo es un mecanismo para crear una nueva categoría de elementos. El nombre de un estereotipo se pone entre comillas francesas, como en «interface». Las comillas francesas son un signo ortográfico de un único carácter conocido sobre todo por su uso en la tipografía francesa para indicar una cita; pero citando a Rumbaugh, “quienes tengan problemas tipográficos pueden utilizar dobles corchetes angulares (<<>>)” [RJB99].

Discusión

El polimorfismo es un principio fundamental para diseñar cómo se organiza el sistema para gestionar variaciones similares. Según el Polimorfismo, un diseño basado en la asignación de responsabilidades puede extenderse fácilmente para manejar nuevas variaciones. Por ejemplo, la inclusión de una nueva clase adaptador de calculador con su propio método polimórfico *getImpuestos* tendrá un impacto menor en el diseño existente.

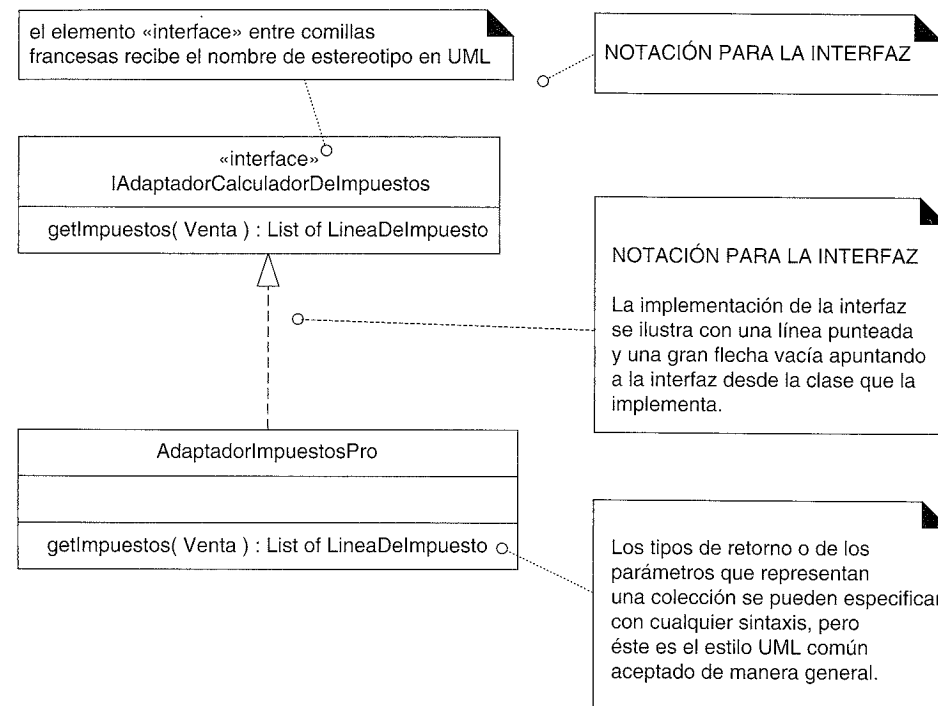


Figura 22.2. Notación UML para las interfaces y los tipos de retorno.

Contraindicaciones

Algunas veces, los desarrolladores diseñan sistemas con interfaces y polimorfismo para “futuras necesidades” especulativas frente a posibles variaciones desconocidas. Si el punto de variación está motivado sin ninguna duda por una variabilidad inmediata o muy probable, entonces, por supuesto, es razonable el esfuerzo de añadir la flexibilidad por medio del polimorfismo. Pero es conveniente una evaluación crítica, puesto que no es raro que se dedique un esfuerzo innecesario a preparar para el futuro un diseño con polimorfismo en puntos de variación que en realidad son improbables y nunca aparecerán realmente. Sea realista sobre la verdadera probabilidad de la posible variación antes de invertir en incrementar la flexibilidad.

Beneficios

- Se añaden fácilmente las extensiones necesarias para nuevas variaciones.
- Las nuevas implementaciones se pueden introducir sin afectar a los clientes.

Patrones Relacionados

- Variaciones Protegidas.
- Varios de los populares patrones de diseño GoF [GHJV95], que se estudiarán en este libro dependen del polimorfismo, entre los que se encuentran los patrones Adaptador, Command, Composite, Proxy, Estado y Estrategia.

También conocido como; parecido a

Elección del Mensaje, No preguntes “¿Qué clase?”.

22.2. Fabricación Pura

Solución

Asigne un conjunto de responsabilidades altamente cohesivo a una clase artificial o de conveniencia que no representa un concepto del dominio del problema —algo inventado para soportar alta cohesión, bajo acoplamiento y reutilización—.

Problema

Tal clase es una *fabricación* de la imaginación. Idealmente, las responsabilidades asignadas a esta fabricación soportan alta cohesión y bajo acoplamiento, de manera que el diseño de la fabricación es muy limpio, o *puro* —de ahí que sea una fabricación pura—.

Finalmente, una fabricación pura implica construir algo, ¡lo que hacemos cuando estamos desesperados!

¿Qué objetos deberían tener la responsabilidad cuando no quiere violar los objetivos de Alta Cohesión y Bajo Acoplamiento, u otros, pero las soluciones que ofrece el Experto (por ejemplo) no son adecuadas?

Los diseños orientados a objetos algunas veces se caracterizan por implementar como clases software las representaciones de los conceptos del dominio del mundo real para reducir el salto en la representación; por ejemplo, una clase *Venta* y *Cliente*. Sin embargo, hay muchas situaciones en las que la asignación de las responsabilidades sólo a las clases software de la capa del dominio da lugar a problemas en cuanto a cohesión y acoplamiento pobres, o que el potencial para reutilizar sea bajo.

Ejemplo

Por ejemplo, suponga que se necesita soporte para almacenar las instancias de *Venta* en una base de datos relacional. Según el Experto en Información, se puede justificar la asignación de esta responsabilidad a la propia clase *Venta*, puesto que la venta tiene los datos que se necesitan almacenar. Pero considere las siguientes implicaciones:

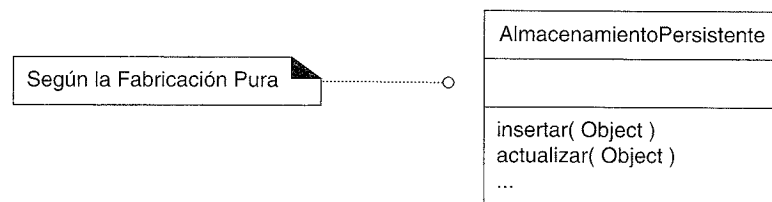
- La tarea requiere un número relativamente amplio de operaciones orientadas al soporte de la base de datos, ninguna relacionada con el concepto de ventas, luego la clase *Venta* deja de tener cohesión.
- La clase *Venta* tiene que acoplarse a la interfaz de la base de datos relacional (como JDBC en las tecnologías Java), luego el acoplamiento aumenta. E incluso el acoplamiento no se produce con otro objeto del dominio, sino con un tipo particular de interfaz de base de datos.
- Almacenar los objetos en una base de datos relacional es una tarea muy general que muchas clases necesitan soportar. La colocación de esta responsabilidad en la clase *Venta* da a entender que la reutilización va a ser pobre o que va a haber mucha duplicación en otras clases que hagan lo mismo.

Por tanto, aunque la *Venta* es una candidata lógica en virtud del Experto en Información para almacenarse a sí misma en una base de datos, nos lleva a un diseño con baja cohesión, alto acoplamiento y bajo potencial para reutilizar —exactamente el tipo de situación desesperada que pide que se cree algo—.

Una solución razonable es crear una nueva clase que es responsable únicamente de almacenar los objetos en algún tipo de medio de almacenamiento persistente, como una base de datos relacional; llamémosla *AlmacenamientoPersistente*². Esta clase es una *Fabricación Pura* —una invención de la imaginación—.

Obsérvese el nombre: *AlmacenamientoPersistente*. Es un concepto que se entiende, pero el nombre o concepto de “almacenamiento persistente” no es algo que uno encon-

² En un *framework* de persistencia real, se necesita finalmente más de una única clase de fabricación pura para crear un diseño razonable. Este objeto será una fachada *front-end* sobre un gran número de objetos de ayuda *back-end*.



traría en el Modelo del Dominio. Y si un diseñador le pregunta a una persona del negocio en una tienda, “¿Trabaja con objetos en almacenamiento persistente?” no le entendería. Ellos entienden conceptos como “venta” y “pago”. El *AlmacenamientoPersistente* no es un concepto del dominio, sino algo creado o fabricado para facilitar las cosas al desarrollador de software.

Esta Fabricación Pura soluciona los siguientes problemas de diseño:

- La *Venta* permanece bien diseñada, con alta cohesión y bajo acoplamiento.
- La clase *AlmacenamientoPersistente* es relativamente cohesiva, teniendo como único objetivo el almacenamiento o inserción de los objetos en un medio de almacenamiento persistente.
- La clase *AlmacenamientoPersistente* es un objeto muy genérico y reutilizable.

La creación de una fabricación pura en este ejemplo es exactamente la situación en la que se requiere su uso —sustituir un mal diseño basado en el Experto, con cohesión y acoplamiento pobres, por un buen diseño en el que el potencial para reutilizar es mayor—.

Observe, como con todos los patrones GRASP, se concede mucha importancia al lugar donde se debería colocar la responsabilidad. En este ejemplo las responsabilidades se cambian de la clase *Venta* (motivada según el Experto) a una Fabricación Pura.

Discusión

El diseño de los objetos se puede dividir en general en dos grupos:

1. Los escogidos de acuerdo a una **descomposición de la representación**.
2. Los escogidos según una **descomposición del comportamiento**.

Por ejemplo, se crean las clases software como la *Venta* de acuerdo a una descomposición de la representación; la clase software está relacionada o representa una cosa en un dominio. La descomposición de la representación es una estrategia común en el diseño de objetos y favorece el objetivo de salto en la representación reducido. Pero algunas veces, deseamos asignar responsabilidades agrupando comportamientos o algoritmos, sin que interese crear una clase con un nombre u objetivo que esté relacionado con un concepto del dominio del mundo real.

Un buen ejemplo es un objeto “algoritmo” como un *GeneradorDeTablaDeContenidos*, cuyo objetivo es (sorpresa) generar una tabla de contenidos y lo creó un desarrollador como clase de ayuda o de conveniencia, sin preocuparse por elegir un nombre del vocabulario del dominio de libros y documentos. Existe como clase de conveniencia concebida por el desarrollador para agrupar algún comportamiento o métodos relacionados y, por tanto, está motivada según la *descomposición de comportamiento*.

En cambio, una clase software denominada *TablaDeContenidos* está inspirada siguiendo una *descomposición de la representación*, y debería contener información consistente con nuestra concepción del dominio real (como los nombres de los capítulos).

No es crítico identificar una clase como Fabricación Pura. Es un objetivo educativo transmitir la idea general de que algunas clases software se inspiran de acuerdo con las representaciones del dominio, y otras simplemente se “crean” por conveniencia del diseñador de objetos. Estas clases de conveniencia normalmente se crean para agrupar algún comportamiento común y, por tanto, se inspiran según una descomposición de comportamiento en lugar de una descomposición de la representación.

Dicho de otro modo, una Fabricación Pura normalmente se organiza en base a funcionalidad relacionada, y de este modo es un objeto centrado en la función o de comportamiento.

Muchos de los patrones de diseño orientados a objetos existentes son ejemplo de Fabricación Pura: Adaptador, Estrategia, *Command*, etcétera [GHJV95].

Como comentario final merece la pena reiterar: Algunas veces la solución que ofrece el Experto en Información no es conveniente. Aunque el objeto sea un candidato porque tiene mucha de la información relacionada con la responsabilidad, en otros aspectos, su elección nos lleva a un diseño pobre, normalmente debido a problemas en la cohesión o el acoplamiento.

Beneficios

- Se soporta Alta Cohesión puesto que las responsabilidades se factorizan en una clase de grano fino que sólo se centra en un conjunto muy específico de tareas relacionadas.
- El potencial para reutilizar podría aumentar debido a la presencia de clases de Fabricación Pura de grano fino cuyas responsabilidades tienen aplicación en otras aplicaciones.

Contraindicaciones

Algunas veces, los inexpertos en el diseño de objetos y que conocen bien la descomposición u organización del software en base a sus funciones, abusan de la descomposición de comportamiento en los objetos de Fabricación Pura. Exagerando, sucede que las funciones se convierten en objetos. No existe nada inherentemente incorrecto en la creación de objetos “función” o “algoritmo”, pero necesita equilibrarse con la capacidad de diseñar con descomposición de la representación, como la capacidad de aplicar el Experto en Información de manera que una clase de la representación como la *Venta* también tenga responsabilidades. El Experto en Información enuncia el objetivo de colocar las responsabilidades con los objetos que conocen la información necesaria para esas responsabilidades, lo cual tiende a favorecer la disminución del acoplamiento. Si se abusa, la Fabricación Pura podría dar lugar a demasiados objetos de comportamiento con responsabilidades que *no* se colocaron con la información necesaria para su realización, lo que puede afectar negativamente al acoplamiento. El síntoma habitual es que la mayoría de los datos contenidos en los objetos se pasan a otros objetos para trabajar con ellos.

Patrones y Principios Relacionados

- Bajo Acoplamiento.
- Alta Cohesión.
- Usualmente una Fabricación Pura asume responsabilidades de las clases del dominio a las que se les asignaría esas responsabilidades en base al patrón Experto.
- Todos los patrones de diseño GoF [GHJV95], como el Adaptador, *Command*, Estrategia, etcétera, son Fabricaciones Puras.
- Prácticamente el resto de patrones de diseño son Fabricaciones Puras.

22.3. Indirección

Solución Asigne la responsabilidad a un objeto intermedio que medie entre otros componentes o servicios de manera que no se acoplen directamente.

El intermediario crea una *indirección* entre los otros componentes.

Problema ¿Dónde asignar una responsabilidad, para evitar el acoplamiento directo entre dos (o más) cosas? ¿Cómo desacoplar los objetos de manera que se soporte el bajo acoplamiento y el potencial para reutilizar permanezca más alto?

Ejemplos **AdaptadorCalculadorDeImpuestos**

Estos objetos actúan como intermediarios con los calculadores de impuestos externos. Mediante el polimorfismo, proporcionan una interfaz consistente para los objetos internos y ocultan las variaciones en las APIs externas. Añadiendo un nivel de indirección y el polimorfismo, los objetos adaptador protegen el diseño interno frente a las variaciones de las interfaces externas (ver Figura 22.3).

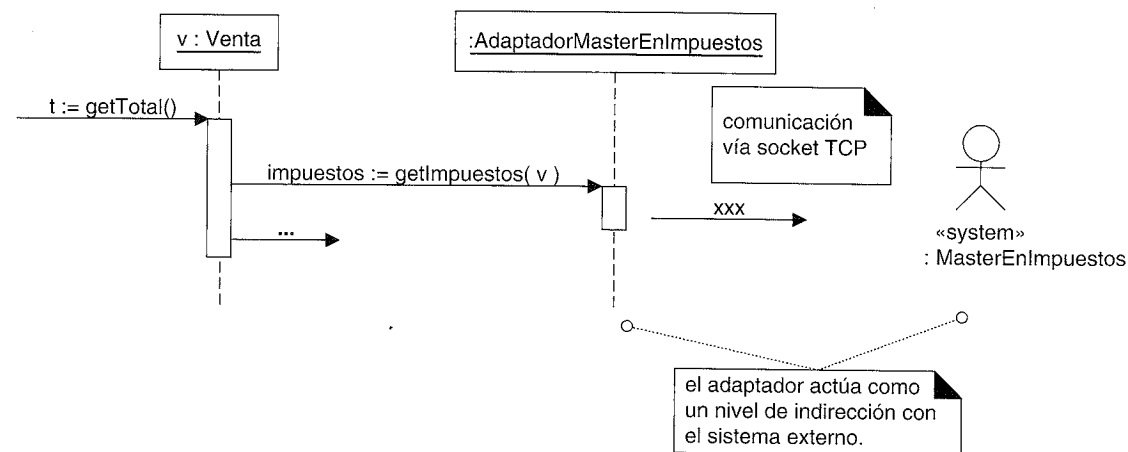


Figura 22.3. Indirección por medio del adaptador.

AlmacenamientoPersistente

El ejemplo de Fabricación Pura de desacoplar la *Venta* de los servicios de base de datos relacional mediante la incorporación de una clase *AlmacenamientoPersistente*, también es un ejemplo de asignación de responsabilidades para soportar la Indirección. El *AlmacenamientoPersistente* actúa como intermediario entre la *Venta* y la base de datos.

Discusión “La mayoría de los problemas en informática se pueden resolver mediante otro nivel de indirección” es un viejo dicho de particular relevancia en los diseños orientados a objetos³.

³ ¿Si algún dicho es viejo en informática! He olvidado la fuente (¿Parnas?). Nótese que también existe el dicho contrario: “¡La mayoría de los problemas en ejecución se pueden resolver eliminando alguna capa de indirección!”.

Del mismo modo que muchos de los patrones de diseño existentes son especializaciones de Fabricación Pura, muchos son también especializaciones de Indirección. Algunos ejemplos son Adaptador, Fachada y Observador. Además, muchas Fabricaciones Puras se generan debido a la Indirección. El motivo para la Indirección normalmente es el Bajo Acoplamiento; se añade un intermediario para desacoplar otros componentes o servicios.

Beneficios

- Disminuir el acoplamiento entre los componentes.

Patrones y Principios Relacionados

- Variaciones Protegidas.
- Bajo Acoplamiento.
- Muchos patrones GoF como Adaptador, Puente, Fachada, Observador y Mediador [GHJV95].
- Muchos intermediarios de Indirección son Fabricaciones Puras.

22.4. Variaciones Protegidas

Solución

Identifique los puntos de variaciones previstas o de inestabilidad; asigne responsabilidades para crear una interfaz estable alrededor de ellos.

Nota: El término “interfaz” se utiliza en el sentido más general de una vista de acceso; no sólo significa literalmente algo como una interfaz de Java o COM.

Problema

¿Cómo diseñar objetos, subsistemas y sistemas de manera que las variaciones o inestabilidades en estos elementos no tengan un impacto no deseable en otros elementos?

Ejemplo

Por ejemplo, el problema anterior del calculador de impuestos externo y su solución con el Polimorfismo ilustra las Variaciones Protegidas (Figura 22.1). El punto de inestabilidad o variación lo forman las diferentes interfaces o APIs de los calculadores de impuestos externos. El sistema de PDV necesita ser capaz de integrarse con muchos sistemas de cálculo de impuestos existentes, y también con futuros calculadores de terceras partes que no existen todavía.

Añadiendo un nivel de indirección, una interfaz, y utilizando el polimorfismo con varias implementaciones de *IAdaptadorCalculadorDeImpuestos*, se consigue proteger al sistema de las variaciones en las APIs externas. Los objetos internos colaboran con una interfaz estable; las distintas implementaciones del adaptador ocultan las variaciones de los sistemas externos.

Discusión

Cockburn publicó por primera vez las Variaciones Protegidas (VP) como patrón en [VCK96], aunque este principio de diseño tan fundamental ha estado por ahí durante décadas bajo distintos términos.

Mecanismos motivados por VP

VP es un principio fundamental que motiva la mayoría de los mecanismos y patrones en la programación y el diseño destinados a proporcionar flexibilidad y protección frente a las variaciones.

A un nivel, se puede ver la madurez de un desarrollador o arquitecto en su conocimiento creciente de mecanismos de amplia aplicación para conseguir VP, elegir las batallas VP apropiada que merecen la pena pelear, y su capacidad para escoger una solución de VP adecuada. En las primeras etapas, uno aprende acerca de la encapsulación de datos, interfaces y polimorfismo —todos ellos mecanismos básicos para alcanzar VP—. Más tarde, uno aprende técnicas como los lenguajes basados en reglas, intérpretes de reglas, diseños basados en reflexión y metadatos, máquinas virtuales, etcétera —todas ellas se pueden aplicar para proteger contra alguna variación—.

Por ejemplo:

Mecanismos básicos de Variaciones Protegidas

VP motiva la encapsulación de datos, interfaces, polimorfismo, indirección y estándares. Nótese que los componentes como gestores (*brokers*) y máquinas virtuales son ejemplos complejos de indirección para conseguir VP.

Diseños dirigidos por los datos

Los diseños dirigidos por los datos cubren una amplia familia de técnicas entre las que se encuentran la lectura de códigos, valores, rutas de ficheros de clase, nombres de clase, etcétera, procedentes de una fuente externa para cambiar el comportamiento, o “parametrizar” un sistema de alguna manera en tiempo de ejecución. Otras variantes pueden ser hojas de estilo, metadatos para la correspondencia objeto-relacional, ficheros de propiedades, lectura de distribución de los elementos de una ventana y muchos más. Se protege al sistema del impacto de variaciones de los datos, metadatos o declarativas registrando externamente la variante, leyéndola, y razonando sobre ella.

Búsqueda de servicios

La búsqueda de servicios incluye técnicas como la utilización de servicios de nombres (por ejemplo, JNDI de Java) o intermediarios (*traders*) para obtener un servicio (por ejemplo, Jini de Java, o UDDI para los servicios Web). Se protegen a los clientes de las variaciones en la ubicación de los servicios, utilizando la interfaz estable de la búsqueda del servicio. Es un caso especial de diseño dirigido por los datos.

Diseños dirigidos por un intérprete

Los diseños dirigidos por un intérprete comprenden los intérpretes de reglas que ejecutan las reglas leídas de una fuente externa, intérpretes de script o lenguajes que leen y ejecutan programas, máquinas virtuales, motores de redes neuronales que ejecutan redes, motores lógicos de restricciones que leen y razonan con conjuntos de restricciones, etcétera. Este enfoque permite cambiar o parametrizar el comportamiento de un sistema por medio de expresiones lógicas externas. Se protege al sistema del impacto de las variaciones lógicas registrando externamente la lógica, leyéndola, y utilizando un intérprete.

Diseños reflexivos o de meta-nivel

Un ejemplo de este enfoque es utilizar el *java.beans.Introspector* para obtener un objeto *BeanInfo*, solicitar el objeto *Method* para el método de acceso (*getter*) para la propiedad del *bean* X, e invocar *Method.invoke*. Se protege al sistema del impacto de la lógica o variaciones externas del código mediante algoritmos reflexivos que utilizan introspección y servicios de metalenguaje. Podría considerarse un caso especial de diseño dirigido por los datos.

Acceso uniforme

Algunos lenguajes como Ada, Eiffel y C# soportan un constructor sintáctico de manera que se expresan de la misma manera el acceso a un método y a un campo. Por ejemplo, *unCirculo.radio* podría invocar al método *radio():float* o referirse directamente al campo público, dependiendo de la definición de la clase. Podemos cambiar de campos públicos a métodos de acceso, sin cambiar el código del cliente.

Principio de Sustitución de Liskov (PSL)

PSL [Liskov88] formaliza el principio de protección contra las variaciones en implementaciones diferentes de una interfaz, o una subclase que extiende una superclase.

Citando textualmente:

Lo que se quiere aquí es algo como la siguiente propiedad de sustitución: si para cada objeto *o1* de tipo *S* hay un objeto *o2* de tipo *T* tal que para todos los programas *P* definidos en términos de *T*, el comportamiento de *P* no varía cuando se sustituye *o1* por *o2* entonces *S* es un subtipo de *T* [Liskov88].

Informalmente, el software (métodos, clases,...) que hace referencia a un tipo *T* (alguna interfaz o superclase abstracta) debería trabajar correctamente o como se espera con cualquier implementación o subclase de *T* que la sustituya —llamémosla *S*—. Por ejemplo:

```
public void añadirImpuestos
    (IAdaptadorCalculadorDeImpuestos calculador, Venta venta)
{
    List lineasDeImpuestos = calculador.getImpuestos( venta );
    //...
}
```

Para este método *añadirImpuestos*, no importa la implementación de *IAdaptadorCalculadorDeImpuestos* que se pase como argumento real, el método debería continuar funcionando “como se espera”. PSL es una idea simple, intuitiva para la mayoría de los desarrolladores de software, que formaliza esta intuición.

Diseños de ocultación de la estructura

En la primera edición de este libro, se enunció un importante principio de diseño de objetos denominado **No Hable con Extraños** o la **Ley de Demeter** [Lieberherr88] como uno de los nueve patrones GRASP. Brevemente, significa evitar crear diseños que recorren largos caminos de la estructura de objetos y envía mensajes (o habla) con objetos distantes, indirectos (extraños). Tales diseños son frágiles con respecto a los cambios en las estructuras de los objetos —un punto frecuente de inestabilidad—. Pero en la segunda edición el principio más general VP sustituye a No Hable con Extraños, porque el segundo es un caso especial del primero. Es decir, un mecanismo para lograr proteger de los cambios en la estructura es aplicar las reglas de No Hable con Extraños.

No Hable con Extraños restringe los objetos a los que deberías enviar los mensajes dentro de un método. Establece que en un método, sólo deberían enviarse mensajes a los siguientes objetos:

1. El objeto *this* (o *self*).
2. Un parámetro del método.
3. Un atributo de *this*.
4. Un elemento de una colección que es un atributo de *this*.
5. Un objeto creado en el método.

La intención es evitar el acoplamiento de un cliente con conocimiento de objetos indirectos y las conexiones entre objetos.

Los objetos directos son “conocidos” del cliente, los objetos indirectos son “extraños”. Un cliente debería hablar con conocidos y evitar hablar con extraños.

A continuación presentamos un ejemplo que viola (ligeramente) el principio No Hables con Extraños. Los comentarios explican la violación.

```
class Registro
{
    private Venta venta;

    public void metodoAlgoFragil()
    {
        //venta.getPago() envía un mensaje a un "conocido" (según #3)

        //pero en venta.getPago().getCantidadEntregada()
        //el mensaje getCantidadEntregada() se aplica a un Pago "extraño"

        Dinero cantidad = venta.getPago().getCantidadEntregada();

        //...
    }
    //...
}
```

Este código recorre conexiones estructurales a partir de un objeto conocido (la *Venta*) a un objeto extraño (el *Pago*), y entonces le envía el mensaje. Es ligeramente frágil, ya que depende del hecho de que los objetos *Venta* se conecten a los objetos *Pago*. En realidad, no es probable que esto sea un problema.

Pero, considere el siguiente fragmento, que llega más lejos a lo largo del camino de la estructura:

```
public void metodoMasFragil()
{
    TitularCuenta titular =
        venta.getPago().getCuenta().getTitularCuenta();

    //...
}
```

El ejemplo es artificial, pero puede ilustrar el patrón: llegar más lejos a lo largo de un camino de conexiones de objetos para enviar un mensaje a un objeto distante e indirecto —hablando con un extraño alejado—. El diseño está acoplado a una estructura particular de cómo están conectados los objetos. Cuanto más lejos a lo largo de un camino recorre el programa, más frágil es.

Karl Lieberherr y sus colegas han investigado sobre buenos principios de diseño de objetos, en el marco del proyecto Demeter. Esta Ley de Demeter (No Hable con Extraños) se identificó debido a que vieron que con frecuencia se producían cambios e inestabilidad en la estructura de los objetos, y como resultado frecuentes roturas en el código que estaba acoplado con el conocimiento de las conexiones de los objetos.

Pero, como examinaremos en la siguiente sección “VP especulativo y escoja sus batallas”, no siempre es necesario proteger contra esto; depende de la inestabilidad de la estructura de los objetos. En las librerías estándar (como las librerías de Java) las conexiones estructurales entre las clases de los objetos son relativamente estables. En sistemas maduros, la estructura es más estable. En sistemas nuevos en las primeras iteraciones, no es estable.

En general, cuanto más lejos se recorre a lo largo de un camino, más frágil es y, por tanto, es más útil ser conforme con el principio No Hable con Extraños.

Si obedecemos estrictamente esta ley —protección contra las variaciones estructurales— es necesario que se añadan nuevas operaciones públicas a los “conocidos” de un objeto; estas operaciones proporcionan en último término la información deseada, y ocultan cómo se obtuvo. Por ejemplo, para soportar No Hable con Extraños en los dos casos anteriores:

```
//caso 1
Dinero cantidad = venta.getCantidadEntregadaEnPago();

//caso 2
TitularCuenta titular = venta.getTitularCuentaDelPago();
```

Contraindicaciones**Advertencia: VP especulativo y escoja sus batallas**

En primer lugar, merece la pena definir dos puntos de cambio:

- **Punto de variación:** variaciones en el sistema actual, existente o en los requisitos, como las múltiples interfaces de los calculadores de impuestos que se deben soportar.
- **Puntos de evolución:** puntos especulativos de variación que podrían aparecer en el futuro, pero que no están presentes en los requisitos actuales⁴.

VP se aplica tanto a los puntos de variación como de evolución

Una advertencia: algunas veces el coste de “futuras necesidades” especulativas en los puntos de evolución pesa más que el coste en el que se incurre por un diseño simple, más “frágil” que se revisa cuando sea necesario en respuesta a las verdaderas presiones de cambio. Es decir, el coste de diseñar la protección en los puntos de evolución puede ser más alto que rehacer un diseño simple.

Por ejemplo, recuerdo un sistema de manejo de mensajes del busca en el que el arquitecto incluyó un lenguaje de script y un intérprete para dar soporte a la flexibilidad y variaciones protegidas en un punto de evolución. Sin embargo, durante la revisión en una versión incremental, se eliminó el complejo (e ineficiente) script —simplemente no era necesario—. Y cuando empecé con la programación orientada a objetos (al principio de los ochenta) padecí la enfermedad de “generalizitis” en la que solía pasar muchas horas creando superclases de las clases que realmente necesitaba escribir. Yo haría todo muy general y flexible (y protegido contra las variaciones), para esas situaciones futuras en las que realmente sería beneficioso —lo que no sucedía nunca—. Era un pobre juez de cuando merecía la pena el esfuerzo.

La idea no es abogar por diseños frágiles que hay que rehacer. Si la necesidad de flexibilidad y protección de cambios es realista, entonces está motivada la aplicación de VP. Pero si es para futuras necesidades o supuesta “reutilización” con probabilidades muy inciertas, entonces se requiere un modo de ver las cosas restringido y crítico.

Los desarrolladores sin experiencia tienden a realizar diseños frágiles, los desarrolladores de nivel intermedio tienden a unos diseños excesivamente flexibles y generalizados (de unas formas que nunca se utilizan). Los diseñadores expertos eligen con perspicacia; quizás un diseño simple y frágil en el que existe un equilibrio entre el coste del cambio y su probabilidad.

- Se añaden fácilmente las extensiones que se necesitan para nuevas variaciones.
- Se pueden introducir nuevas implementaciones sin afectar a los clientes.
- Se reduce el acoplamiento.
- Puede disminuirse el impacto o coste de los cambios.

Beneficios

⁴ En el UP, los puntos de evolución se pueden documentar formalmente en **Casos de Cambio**; cada uno describe aspectos relevantes de un punto de evolución a beneficio de un futuro arquitecto.

Patrones y Principios Relacionados

- La mayoría de los patrones y principios de diseño son mecanismos para variaciones protegidas, entre los que se encuentran el polimorfismo, las interfaces, indirectión, encapsulación de datos, la mayoría de los patrones de diseño GoF, etcétera.
- En [Pree95] los puntos de variación y evolución reciben el nombre de “puntos calientes”.

También conocido como; Parecido a

VP es esencialmente lo mismo que los principios de ocultación de la información y de abierto-cerrado, que son términos más antiguos. Como patrón “oficial” en la comunidad de los patrones, fue Cockburn en [VCK96] quien lo denominó “Variaciones Protegidas” en 1996.

Ocultación de la Información

El famoso artículo de David Parnas *On the Criteria To Be Used in Decomposing Systems Into Modules* (Criterios a Utilizar para Descomponer los Sistemas en Módulos) [Parnas72] es un ejemplo de los clásicos que siempre se citan pero que rara vez se leen. En él, Parnas introduce el concepto de **ocultación de información**. Quizás debido a que el término suena como la idea de encapsulación de datos, se ha mal interpretado como eso, y algunos libros definen erróneamente los conceptos como sinónimos. Más bien, lo que Parnas quiso decir con la ocultación de información era ocultar información sobre los diseños a otros módulos, en los puntos de cambio difíciles o probables. Citando literalmente su explicación sobre la ocultación de información como principio de diseño conductor:

Proponemos en lugar de eso que uno comience con una lista de decisiones de diseño difíciles o decisiones de diseño que es probable que cambien. Cada módulo entonces se diseña para ocultar tal decisión a los otros.

Esto es, el principio de Parnas de ocultación de información es el mismo que se enuncia en VP, y no es simplemente encapsulación de datos —que no es más que una de las muchas técnicas para ocultar información acerca del diseño—. Sin embargo, el término ha sido tan ampliamente reinterpretado como sinónimo de encapsulación de datos que ya no es posible utilizarlo en el sentido original sin malentendidos.

Principio Abierto-Cerrado

El **Principio Abierto-Cerrado** (PAC), descrito por Bertrand Meyer en [Meyer88] es esencialmente equivalente al patrón VP y a la ocultación de información. Una definición del PAC es:

Los módulos deberían ser tanto abiertos (para extenderse; adaptables) como cerrados (el módulo está cerrado a las modificaciones que afecten a los clientes).

El PAC y las VP son esencialmente dos expresiones del mismo principio, que resaltan diferentes puntos: protección en los puntos de variación y evolución. En el PAC, los “módulos” comprenden todos los elementos de software discretos, como los métodos, clases, subsistemas, aplicaciones, etcétera.

En el contexto del PAC, la frase “cerrado con respecto a X” significa que los clientes no se ven afectados si X cambia. Por ejemplo, “la clase está cerrada con respecto a las definiciones de los campos de instancia” mediante el mecanismo de encapsulación de datos con campos privados y métodos de acceso públicos. Al mismo tiempo, están abiertas a la modificación de las definiciones de los datos privados, porque los clientes externos no están acoplados directamente a los datos privados.

Otro ejemplo podría ser, “los adaptadores de los calculadores de impuestos están cerrados con respecto a su interfaz pública” mediante la implementación de la interfaz estable *IAdaptadorCalculadorDeImpuestos*. Sin embargo, los adaptadores están abiertos a la extensión siendo modificados de manera privada en respuesta a los cambios en las APIs de los calculadores de impuestos externos, de manera que no afectan a sus clientes.

DISEÑO DE LAS REALIZACIONES DE CASOS DE USO CON LOS PATRONES DE DISEÑO GoF

Cualquier cosa que tú puedas hacer, yo puedo hacerla meta.
Daniel Dennett

Objetivos

- Aplicar los patrones de diseño GRASP y GoF al diseño del caso de estudio NuevaEra.

Introducción

Este capítulo estudia el diseño de objetos para las realizaciones de los casos de usos de la siguiente iteración del caso de estudio NuevaEra, que aborda el soporte para el acceso a los servicios externos de terceras partes cuyas interfaces podrían variar, de reglas más complejas para establecer el precio de los productos y de reglas del negocio conectables.

En el contexto de los problemas de diseño que se discutirán, también se introducirá nueva notación UML muy utilizada.

Lo importante es mostrar cómo aplicar los patrones GoF y los GRASP más básicos. Se pretende ilustrar que el diseño de objetos y la asignación de responsabilidades se puede explicar y aprender en base a la aplicación de los patrones —un vocabulario de principios y estilos que se pueden combinar para diseñar los objetos—.

Los patrones de la “pandilla de los cuatro” (Gang-of-Four)

Los patrones adicionales que se presentan en este capítulo proceden de *Design Patterns* [GHJV95], un libro básico y muy popular que presenta 23 patrones que son útiles durante el diseño de objetos. Puesto que el libro fue escrito por cuatro autores, estos patrones se conocen como los patrones de la “pandilla de los cuatro” o patrones “GoF”¹.

Este capítulo proporciona una introducción a algunos de los patrones GoF más utilizados; capítulos posteriores presentarán más². Se recomienda un estudio completo y cuidadoso del libro *Design Patterns* para iniciarse como diseñador de objetos, aunque el libro asume que el lector ya es un diseñador con alguna experiencia; este libro ofrece una introducción.

Un vocabulario compartido

Además del vocabulario visual de la notación UML, al final de este capítulo tendremos un vocabulario de diseño compartido más rico, en términos de nombres de patrones. Por tanto, será posible comunicar progresivamente ideas de diseño del software sobre todo mediante diagramas UML, con algunas notas conectadas que indican los patrones (Indirección, Estrategia,...) que se van a aplicar.

23.1. Adaptador (GoF)

El problema que se presentó en el capítulo anterior para motivar el patrón Polimorfismo, y su solución, es más concretamente un ejemplo del patrón GoF **Adaptador**.

Adaptador (<i>Adapter</i>)
<i>Contexto/Problema</i> ¿Cómo resolver interfaces incompatibles, o proporcionar una interfaz estable para componentes parecidos con diferentes interfaces?
<i>Solución</i> Convierta la interfaz original de un componente en otra interfaz, mediante un objeto adaptador intermedio.

Recordemos que: el sistema de PDV NuevaEra necesita soportar diferentes tipos de servicios externos de terceras partes, entre los que se encuentran los calculadores de impuestos, servicios de autorización de pagos, sistemas de inventario, y sistemas de contabilidad, entre otros. Cada uno tiene un API diferente, que no puede ser cambiada.

Una solución es añadir un nivel de indirección con objetos que adapten las distintas interfaces externas a una interfaz consistente que es el que se utiliza en la aplicación. La solución se ilustra en la Figura 23.1.

¹ Con una referencia tangencial a la política China.

² En la práctica, quizás unos 15 de estos 23 patrones son los que se utilizan con frecuencia.

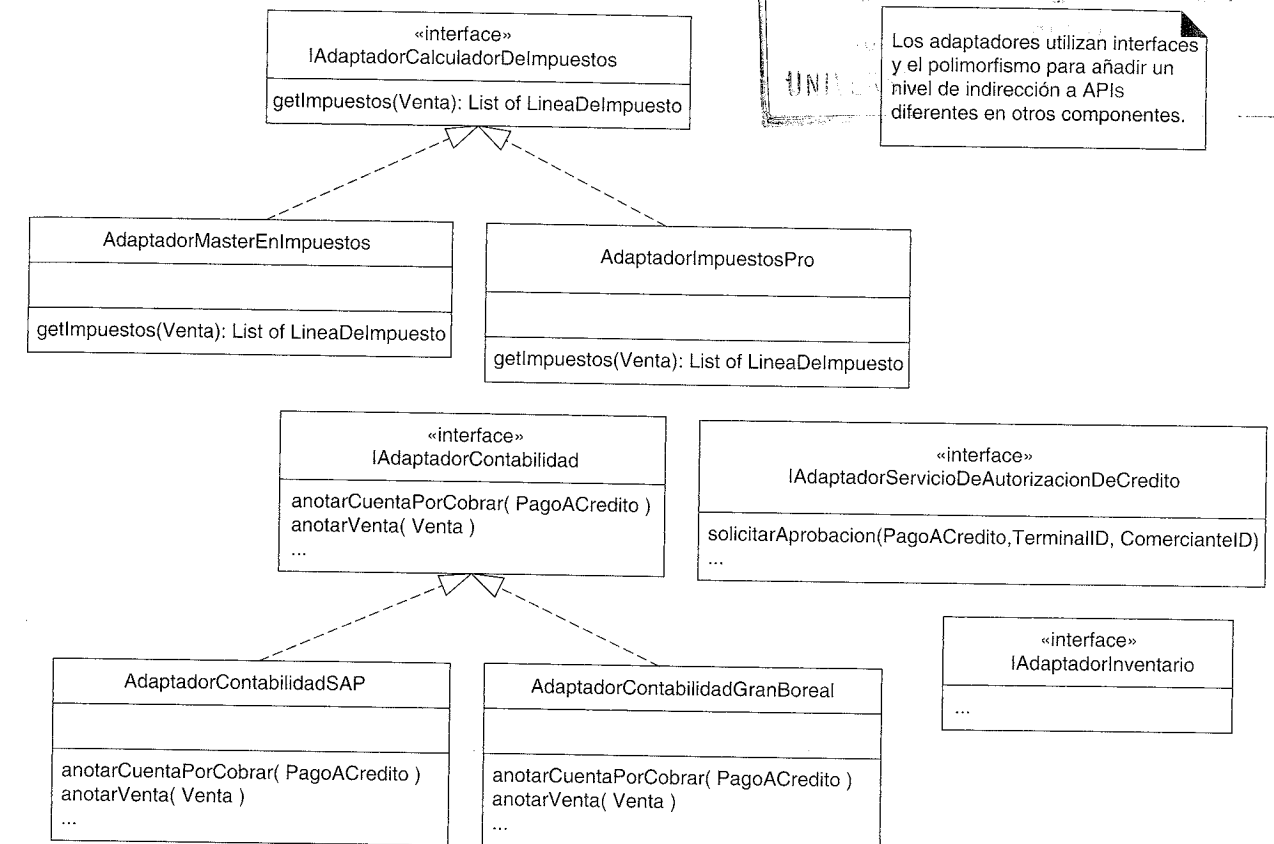


Figura 23.1. El patrón Adaptador.

Como se ilustra en la Figura 23.2, una instancia de un adaptador particular se instanciará para el servicio externo elegido³, como SAP para contabilidad, y adaptará la solicitud *anotarVenta* a la interfaz externa, tal como una interfaz SOAP XML sobre HTTPS para un servicio Web de intranet ofrecido por SAP.

Notación UML: Obsérvese en la Figura 23.2 el uso de un “piruli” de interfaz para indicar que la instancia *AdaptadorContabilidadSAP* implementa un interfaz relevante.

Polimorfismo, Indirección y Variaciones Protegidas (GRASP)

La aplicación anterior del patrón Adaptador es una especialización de los componentes básicos GRASP. Ofrece Variaciones Protegidas de los cambios en las interfaces externas o paquetes de terceras partes mediante el uso de un objeto de Indirección que aplica interfaces y Polimorfismo.

³ En la Arquitectura de Conexión de J2EE, estos adaptadores a los servicios externos se denominan de manera más específica **adaptadores de recursos**.

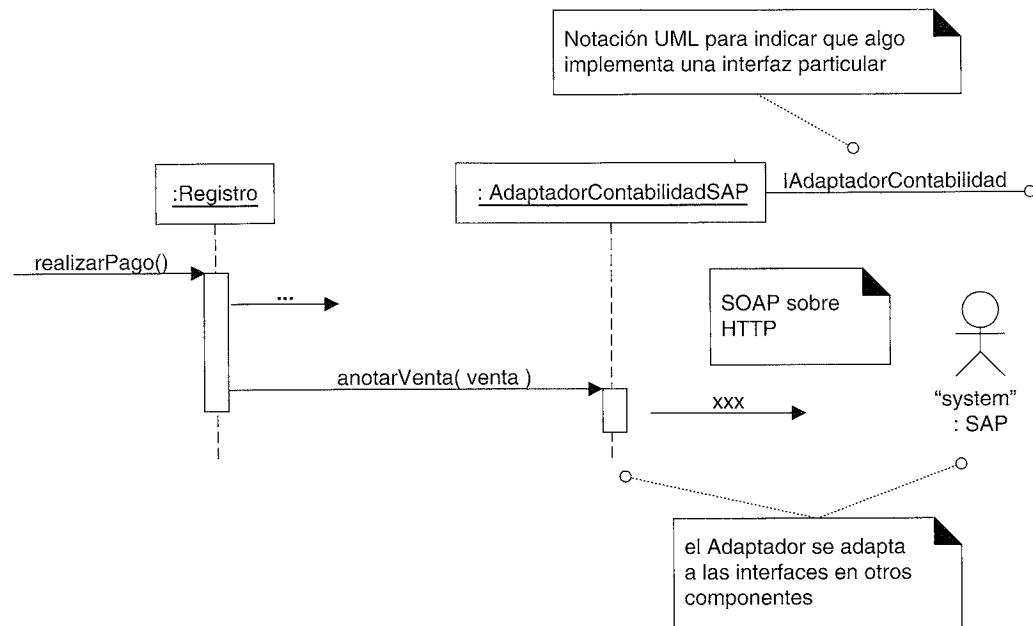


Figura 23.2. Utilización de un adaptador.

Nótese que se pueden analizar patrones mucho más complejos y especializados en función de la familia GRASP básica. Existen cientos de patrones de diseño publicados, y aunque es útil estudiarlos para acelerar el aprendizaje, entender sus conceptos básicos subyacentes (Variaciones Protegidas, Bajo Acoplamiento, Polimorfismo, Indirección...) nos ayudan a abrirnos camino a través de los innumerables detalles y ver el “alfabeto” esencial de las técnicas de diseño que se van a aplicar.

Convención de nombres: ¿incluimos el nombre del patrón en el nombre del tipo?

Nótese que los nombres de los tipos incluyen el nombre del patrón “Adaptador”. Éste es un estilo relativamente común y tiene la ventaja de poder comunicar fácilmente a los que leen el código o los diagramas el patrón de diseño que se va a utilizar.

23.2. Descubrimientos del “análisis” durante el diseño: Modelo del Dominio

Observe que en el diseño del Adaptador en la Figura 23.1, la operación *getImpuestos* devuelve una lista de *LineaDeImpuesto*. Es decir, al reflexionar e investigar en profundidad el modo de gestionar los impuestos y cómo funcionan los calculadores de impuestos, el diseñador (yo) se dio cuenta de que se asocia una lista de líneas de impuestos con una venta, como impuesto del gobierno central, impuesto del gobierno regional, etcétera (¡siempre que exista la oportunidad los gobiernos inventarán nuevos impuestos!).

Además de ser una clase software recién creada en el Modelo de Diseño, se trata de un concepto del dominio. Es normal y común descubrir conceptos del dominio relevantes y refinar el conocimiento de los requisitos durante el diseño o la programación —el desarrollo iterativo apoya esta clase de descubrimiento incremental—.

¿Debería reflejarse este descubrimiento en el Modelo del Dominio (o Glosario)? Si el Modelo del Dominio se utilizará en el futuro como fuente de inspiración para el trabajo de diseño posterior, o como ayuda visual para comunicar los conceptos claves del dominio, entonces podría ser útil añadirlo. La Figura 23.3 presenta un Modelo del Dominio actualizado.

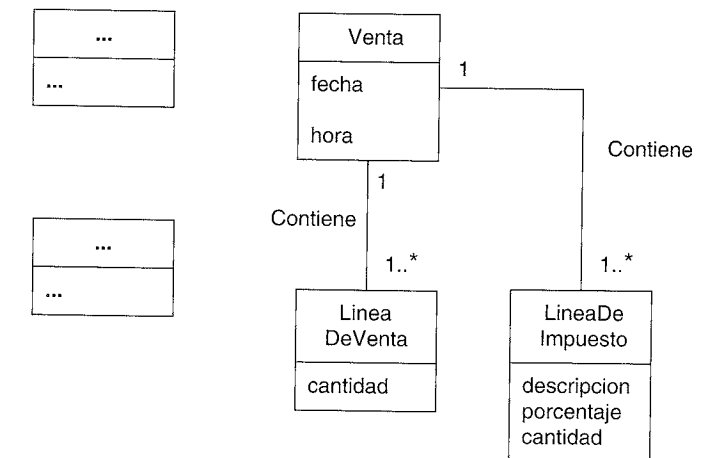


Figura 23.3. Modelo del Dominio parcial actualizado.

¿Mantenemos el Modelo del Dominio?

Puntualizando el comentario anterior sobre la actualización del Modelo del Dominio: obsérvese que la arquitectura del Modelo de Diseño normalmente se organizará en capas (lo que se discutirá con más detalle en un capítulo posterior). Una de estas capas de clases de diseño se denominará **capa del dominio**; esta capa contendrá las clases software cuyos nombres y estructuras se inspiraron en el vocabulario y los conceptos del dominio (*Venta*, *LineaDeImpuesto*, etcétera).

Sugerencia

Después de algunas iteraciones, el Modelo del Dominio —como primera fuente de inspiración para las clases del diseño en la capa del dominio del Modelo de Diseño— podría dejar de tener utilidad. Si la actualización del Modelo del Dominio para reflejar los cambios en el Modelo de Diseño deja de tener un valor práctico, piense en eliminarlo. Más bien, únicamente lleve a cabo un trabajo de ingeniería inversa (con una herramienta CASE de UML) para generar un diagrama de clases de la capa del dominio a partir de las clases de diseño del Modelo de Diseño. Aunque son clases software en lugar de clases conceptuales del dominio puras, reflejan el vocabulario del dominio más relevante que ha surgido en el diseño del software y, por tanto, un diagrama de clases UML de las clases del diseño en la capa del dominio del Modelo de Diseño puede ser una “aproximación” útil al verdadero Modelo del Dominio.

Por favor, no lo mal interprete: no sugerimos que se elimine definitivamente un Modelo del Dominio, sino más bien que considere si merece la pena mantenerlo, o es simplemente documentación para generar trabajo, y que conozca qué alternativas pueden ser útiles.

Patrones Relacionados

Un adaptador de recursos que oculta un sistema externo podría también ser considerado un objeto Fachada (otro patrón GoF que se expone en este capítulo), puesto que envuelve el acceso al subsistema o sistema con un único objeto (que es la esencia de la Fachada). Sin embargo, el nombre de adaptador está motivado especialmente cuando el objeto que envuelve facilita la adaptación a diversas interfaces externas.

23.3. Factoría (GoF)

El adaptador da lugar a un nuevo problema en el diseño: en la solución del patrón Adaptador anterior para servicios externos con diversas interfaces, ¿quién crea el adaptador? ¿Y cómo determinar qué clase de adaptador crear, como *AdaptadorMasterE-Impuestos* o *AdaptadorImpuestosPro*?

Si los crea algún objeto del dominio, las responsabilidades de los objetos del dominio exceden la lógica pura de la aplicación (como el cálculo del total de la venta) y entran en otras cuestiones relacionadas con la conexión con componentes software externos.

Este punto subraya otro principio de diseño fundamental (normalmente considerado un principio de diseño de arquitectura): diseño para mantener una **separación de intereses** (*separation of concerns*). Es decir, divida en módulos o separe intereses distintos en áreas diferentes, de manera que cada una tenga un propósito cohesivo. Por ejemplo, la capa del dominio de los objetos software destaca las responsabilidades centradas relativamente en la lógica de la aplicación, mientras que un grupo diferente de objetos es responsable de las cuestiones de conectividad con sistemas externos.

Por tanto, la elección de un objeto del dominio (como el *Registro*) para crear los adaptadores no está de acuerdo con el objetivo de separación de intereses, y disminuye su cohesión.

Notación UML: Observe el estilo del diagrama UML de la Figura 23.4 que incluye una nota que muestra el pseudocódigo detallado de *getAdaptadorCalculadorDeImpuestos*. Este estilo nos permite incluir detalles dinámicos sobre algoritmos en los diagramas de clases estáticos de manera que podría disminuir la necesidad de utilizar diagramas de interacción.

Una alternativa típica en este caso es aplicar el patrón **Factoría** (o **Factoría Concreta**), en el que se define un objeto Fabricación Pura “factoría” para crear los objetos.

Los objetos factoría tienen varias ventajas:

- Separan la responsabilidad de la creación compleja en objetos de apoyo (*helper*) cohesivos.
- Ocultan la lógica de creación potencialmente compleja.
- Permiten introducir estrategias para mejorar el rendimiento de la gestión de la memoria, como objetos caché o de reciclaje.

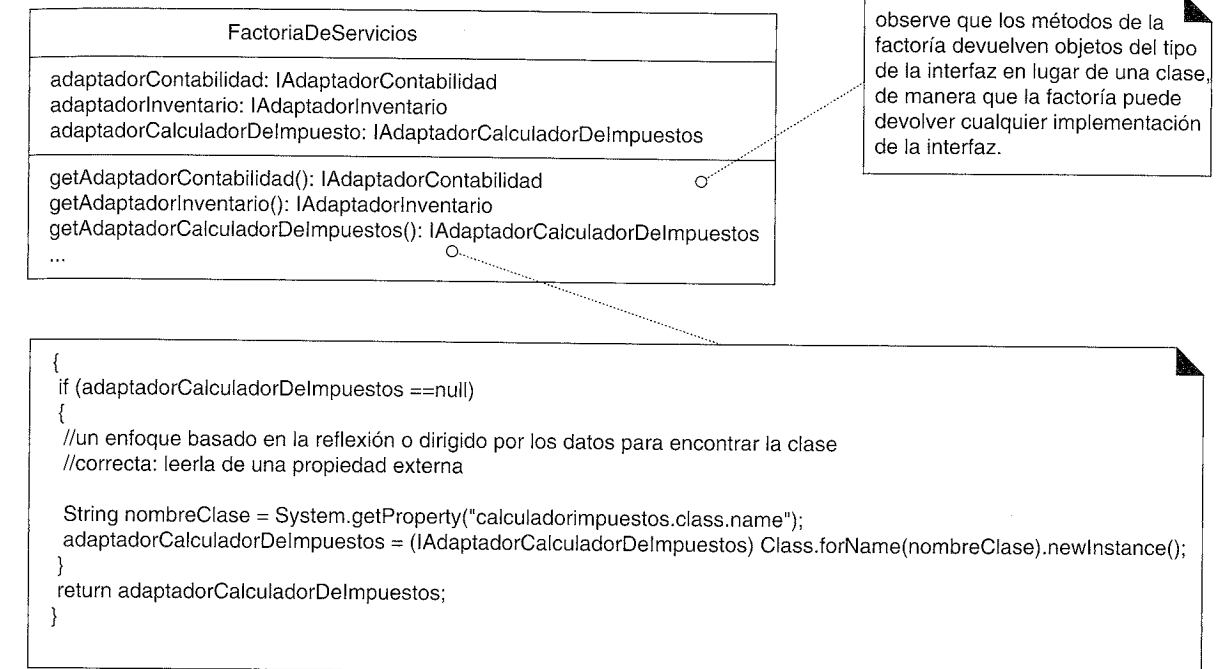
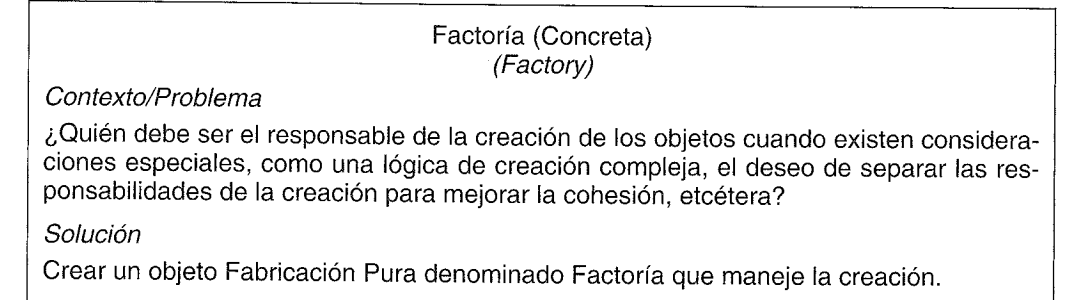


Figura 23.4. El patrón Factoría.



La solución de la Factoría se ilustra en la Figura 23.4.

Nótese que en la *FactoriaDeServicios*, la lógica para decidir qué clase se crea se resuelve leyendo el nombre de la clase desde una fuente externa (por ejemplo, por medio de una propiedad del sistema, si se utiliza Java) y después se carga la clase dinámicamente. Éste es un ejemplo de **diseño dirigido por los datos** parcial. El diseño consigue Variaciones Protegidas con respecto a los cambios en la clase de implementación del adaptador. Sin cambiar el código fuente en la clase factoría, podemos crear instancias de nuevas clases adaptador cambiando el valor de la propiedad y asegurando que la nueva clase es visible en la ruta de clases de Java para que pueda cargarse.

A menudo se accede a las factorías con el patrón Singleton⁴.

Patrones Relacionados

⁴ N. del T.: Al igual que para otros patrones, no hemos traducido el nombre de este patrón ya que a diferencia de otros (como Factoría o Adaptador) normalmente se utiliza el nombre original en inglés tanto en la expresión oral como escrita en castellano.

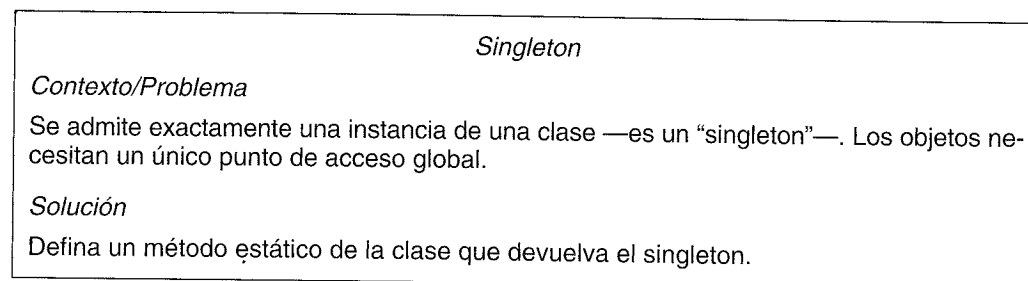
23.4. Singleton (GoF)

Con la *FactoriaDeServicios* surge un nuevo problema de diseño: ¿quién crea la propia factoría y cómo se accede?

En primer lugar, observe que sólo se necesita una instancia de la factoría en el proceso. Segundo, una rápida reflexión sugiere que los métodos de esta factoría podrían necesitar que se invoquen desde varios sitios del código, puesto que en diferentes lugares se necesita acceder a los adaptadores para solicitar los servicios externos. Por tanto, existe un problema de visibilidad: ¿cómo conseguir la visibilidad a esta única instancia de *FactoriaDeServicios*?

Una solución es pasar la instancia de *FactoriaDeServicios* como parámetro donde quiera que se descubra que se necesita que sea visible, o inicializar los objetos que necesitan que la factoría sea visible, con una referencia permanente. Esto es posible pero no es conveniente; una alternativa es el patrón **Singleton**.

Ocasionalmente, es conveniente mantener visibilidad global o un único punto de acceso a una única instancia de una clase, en lugar de cualquier otra forma de visibilidad. Esto se cumple para la instancia de *FactoriaDeServicios*.



Por ejemplo, la Figura 23.5 muestra una implementación del patrón Singleton.

Por tanto, la idea clave es que la clase X defina un método estático *getInstancia* que él mismo proporciona una única instancia de X.

Con este enfoque, un desarrollador tiene visibilidad global a esta instancia única, por medio del método estático de la clase *getInstancia*, como vemos en este ejemplo:

```
public class Registro
{
    public void inicializar()
    {
        ... hace algún trabajo ...

        //acceso a la Factoría Singleton mediante la llamada a
        getInstancia adaptadorContabilidad =
            FactoriaDeServicios.getInstancia().getAdaptadorContabilidad();
    }
}
```

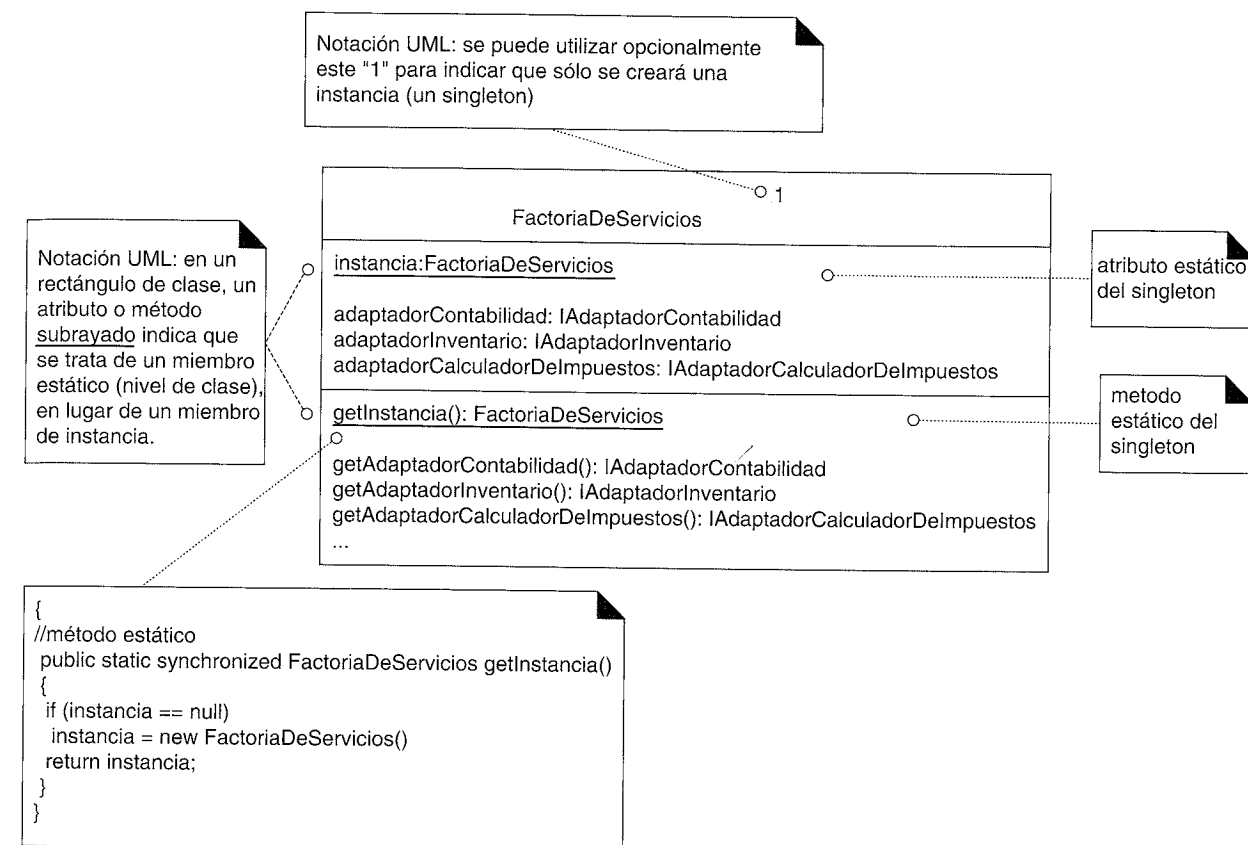


Figura 23.5. El patrón Singleton en la clase *FactoriaDeServicios*.

```
... hace algún trabajo ...
}

// otros métodos...
}
```

Puesto que la visibilidad de las clases públicas tiene un alcance global (en la mayoría de los lenguajes), en cualquier punto del código, en cualquier método de cualquier clase, uno puede escribir *ClaseSingleton.getInstancia()* para obtener la visibilidad de la instancia del singleton, y entonces enviarle un mensaje, como *ClaseSingleton.getInstancia().hacerFoo()*. Es difícil superar la emoción de ser capaces de *hacerFoo* globalmente.

Abreviatura UML para el acceso a un Singleton en los diagramas de interacción

La notación UML que implica —pero no muestra explícitamente— el mensaje *getInstancia* en un diagrama de interacción es añadir el estereotipo «singleton» a la instancia, como en la Figura 23.6. Este enfoque evita tener que mostrar explícitamente el mensaje (sin interés) *getInstancia* a la clase antes de enviar el mensaje a la instancia singleton.